



Department of Computer Science

COMP4300 - Graduation Project

Fall 2025/2026

Birzeit Municipality Archiving System



Authors

Malak Mustafa	ID: 1212986
Rama Shaheen	ID: 1211153
Issam Abdelkareem	ID: 1211821

Supervisor: Dr. Yousef Hassouneh

Section – B

Title of Project: Birzeit Municipality Archiving System: A digital solution for Birzeit municipality's document management

Project No: 20

Supervisor: Dr. Yousef Hassouneh

Key Areas: Document Management System (DMS), Search - Retrieval, Workflow Automation, Security - Data Privacy, and User Interface - Usability

Section – C

Student Signature: _____

Date: _____

Student Signature: _____

Date: _____

Student Signature: _____

Date: _____

Submitted: _____

First Supervisor Name: _____

First Supervisor Signature: _____ **Date:** _____

Approved: ____ / ____ / _____

Abstract

This project examines the Birzeit municipality problems with the archiving system they have, this includes poor classification, document loss, low knowledge on how to archive documents and the tools used to archive the documents in the municipality, such as excel sheet and word document, which are not designed or recommended for archiving purposes leading to inefficiencies. So the aim of this project, is to design and create a efficient, easy to use and secure digital archiving system for the Birzeit municipality to improve document management by building, alongside the paper-based archiving, a digital-based archiving system.

To develop a clear process to build the system, beginning with a thorough understanding and study of the municipality's specific needs. Built with HTML5, CSS3, and JavaScript for the front-end, and python, flask with MySQL, RDF triples with graphDB for the backend, however, the system does have key features, like using semantic search for context-based retrieval beyond keywords and role-based access control to provide a secure access based on the user role, along with the support of metadata indexing for better categorization and retrieval finishing with an easy to use interface for all municipal employees.

The implementation of the digital archiving system for the Birzeit municipality is expected to accomplish significant improvements in document archiving of the municipality. By addressing the previously identified challenges, the integration of semantic search capabilities will enable more accurate and context-based document retrieval, letting staff members find information more naturally and going beyond basic keyword searches. Role-based access control will also guarantee data confidentiality and integrity by limiting access privileges according to user roles, preventing sensitive documents from being manipulated or accessed by unauthorized parties. Superior document classification and retrieval will be made possible by the strong metadata indexing, which will make it simpler for municipal staff to arrange and locate information. Last but not least, the intuitive interface, made for all municipal workers, will encourage smooth adoption and lower the learning curve for new technologies. Through the use of digital archiving in conjunction with current paper-based techniques, the Birzeit Municipality will be able to create a robust and all-encompassing archiving solution, which will ultimately result in more efficient operations, better decision-making, and increased productivity all around.

Contents

Acknowledgements	8
1 Introduction	9
1.1 General overview	9
1.2 Motivation	9
1.3 Method	10
1.4 Aim and Objectives	11
1.5 Overview of the technical area	13
1.6 Overview of the report	13
2 Background	15
2.1 Details of relevant theory	15
2.2 Review of past/reported work	16
2.3 Tools and Technology	18
3 System Analysis and Design	19
3.1 System Model and Architecture	19
3.1.1 Product Description	19
Non-Functional Requirements	23
3.2 Functional Decomposition	25
3.2.1 Actors	25
3.2.2 Use Cases	26
3.2.3 Use Cases Diagram	27
3.3 System Models	28
3.3.1 Class Diagram	28
3.3.2 Sequence Diagram	29
3.3.3 Activity Diagram	38
3.3.4 State Chart Diagram	49
3.4 System Architecture	51
3.4.1 Sub-System	51
3.4.2 Software Architecture	52
3.4.3 Deployment diagram	53
3.5 Data Management and ERD	54
4 Testing	56
4.1 Testing methodology	56
4.1.1 Black-Box Testing	56
4.1.2 White-Box Testing	56
4.2 Integration testing	57

4.2.1	Document Management Section	57
4.2.2	Search and Retrieval Section	62
4.2.3	Editing and version control	66
4.3	Testing results and analysis	69
5	Implementation and UI	70
5.1	Implementation plan	70
5.1.1	System architecture implementation	72
5.1.2	Security and Access control implementation	72
5.1.3	Semantic search implementation	73
5.2	Implementation of backend	74
5.2.1	System File Structure	74
5.2.2	Back-end architecture	75
5.2.3	Database implementation	76
5.2.4	File storage management	77
5.2.5	Semantic layer implementation	78
5.2.6	Archive document implementation	79
5.3	User Interface	80
5.3.1	Login & Register	80
5.3.2	Home Pages	81
5.3.3	View Documents	82
5.3.4	Search For Document	83
5.3.5	Archive Document	84
5.3.6	Metadata Management	85
5.3.7	Departments and Categories Management	86
5.3.8	Requests	86
5.3.9	Report Generation	87
6	Conclusion	89
6.1	Review of the project	89
A	Appendices	91
A.1	Use Case Specifications	91
A.2	Glossary (or data dictionary)	107
A.3	Code	108
A.4	System modifications	110

List of Tables

3.1	Authentication Requirements	21
3.2	Document Archiving Requirements	21
3.3	Search for a Document Requirements	21
3.4	Role Management Requirements	21
3.5	version control Requirement	22
3.6	Audit log Requirement	22
3.7	Report generate Requirement	22
3.8	Workflow Requirement	23
3.9	Performance Requirements	23
3.10	Security Requirements	23
3.11	Usability Requirements	23
3.12	Reliability Requirements	24
3.13	Scalability Requirements	24
3.14	Compliance Requirements	24
3.15	Actors and their description	25
3.16	Use case summary	26
4.1	TC-01: Archive Document Test – Positive Flow	58
4.2	TC-02: Archive Document Test – Negative Flow (Input Validation) . . .	60
4.3	TC-03: Archive Document Test – positive Flow	61
4.4	TC-04: Search Document Test – Positive Flow	63
4.5	TC-05: Search Document Test – Negative Flow (Invalid query input) . .	64
4.6	TC-06: Search Document Test – Positive flow (filter-based search)	65
4.7	TC-07: Edit Document Test – Positive Flow	66
4.8	TC-08: Edit Document Test – Negative Flow	67
4.9	TC-09: Edit Document Test – Positive Flow	68
A.1	UC-01: Login/Authentication	91
A.2	UC-02: Archive	92
A.3	UC-03: Edit document	94
A.4	UC-04: Search for a Document	95
A.5	UC-05: View Document	97
A.6	UC-06: Manage User Roles and Permissions	99
A.7	UC-07: Document Version Control	100
A.8	UC-08: Metadata Management	101
A.9	UC-09: Manage Document Categories	102
A.10	UC-10: Manage Municipal Departments	103
A.11	UC-11: Log Activity	104
A.12	UC-12: Generate Document Report	106

List of Figures

2.1	Role Management interface in OpenKM	16
2.2	Document structure and interface in SeedDMS	17
3.1	Use case diagram	27
3.2	Class diagram	28
3.3	Archive sequence diagram	29
3.4	Edit sequence diagram	30
3.5	Search sequence diagram	31
3.6	View sequence diagram	31
3.7	Manage user role sequence diagram	32
3.8	version control sequence diagram	32
3.9	Metadata schema management sequence diagram	33
3.10	Manage document categories sequence diagram	34
3.11	Manage municipal departments sequence diagram	35
3.12	Audit log sequence diagram	36
3.13	Generate report sequence diagram	37
3.14	Archive activity diagram	38
3.15	Edit activity diagram	39
3.16	Search activity diagram	40
3.17	View activity diagram	41
3.18	Manage user role activity diagram	42
3.19	version control activity diagram	43
3.20	Metadata schema management activity diagram	44
3.21	Manage document categories activity diagram	45
3.22	Manage municipal departments activity diagram	46
3.23	Audit log activity diagram	47
3.24	Generate report activity diagram	48
3.25	Document state chart diagram	49
3.26	Version control state chart diagram	50
3.27	Sub System	51
3.28	Software architecture	52
3.29	Deployment Diagram	53
3.30	ERD	54
5.1	RDF-based semantic knowledge graph	73
5.2	JavaScript Files Structure	74
5.3	Python Backend Files Structure	75
5.4	Login Page	80
5.5	Register Form	80

5.6	Administrative Affairs Dashboard	81
5.7	Secretary Dashboard	81
5.8	Department Head Dashboard	82
5.9	Municipal Employee Dashboard	82
5.10	View Documents	83
5.11	Search For Document	83
5.12	Step 1: Department and Category Selection	84
5.13	Step 2: Metadata Entry and File Upload	84
5.14	Successful Archiving Confirmation	84
5.15	Metadata Management Interface	85
5.16	Departments and Categories Management Interface	86
5.17	Pending Requests - Department Head View	87
5.18	My Requests - Employee View	87
5.19	Report Generation Interface	88
A.1	Archive Pseudo Code	108
A.2	Semantic Search Pseudo Code	109

Acknowledgements

We want to express our sincere gratitude to everyone who supported us throughout this project. First and foremost, we would like to thank our supervisor, Dr. Yousef Hassouneh, for his invaluable guidance, continuous support, and insightful feedback throughout the development of this project. His expertise and encouragement were instrumental in helping us overcome challenges and achieve our goals. We are also deeply grateful to the staff of Birzeit Municipality for their cooperation and assistance. Their willingness to share their knowledge about the current document management processes and their valuable input on the system requirements were essential to the success of this project. Their feedback helped us design a system that truly meets the needs of the municipality. We extend our heartfelt thanks to our families and friends for their unwavering support, patience, and encouragement throughout this journey. Their understanding and motivation kept us going during the most challenging times. Finally, we extend our thanks to everyone who contributed to making this project a reality.

Chapter 1

Introduction

1.1 General overview

The Birzeit municipality's archiving system is an advanced digital platform created to simplify and improve the way documents are handled within the municipality. It works in together with the municipality's current traditional paper-based files, which are at risk of loss, damage, and a lack of organization. The system improves archiving, organizing, and retrieval procedures for municipal employees by safely keeping all documents in a single, central location, increasing the efficiency of daily tasks.

Employees can easily find documents without having to search through physical files due to key features like semantic search, metadata indexing, and a user-friendly interface. To protect sensitive information, the system also includes secure login processes and role-based access, guaranteeing that only individuals with the proper authorization can access or handle particular documents.

One of the key benefits of this system is its ability to store documents in a structured digital format, which greatly reduces the chance of loss and speeds up retrieval. This modification represents a modern, reliable approach to document management that helps municipal workers.

1.2 Motivation

The reason we thought of this project is that there are several challenges and general gaps in the current system at the municipality such as

1. Unwieldy classification and categorization: Despite some classifications, the system is complicated and overlapping, making it difficult to find and access information quickly.
2. Loss of some documents: Sometimes documents do not reach the archiving department, leading to document loss.
3. Employee ignorance: The majority of staff members are not well-versed in the fundamentals and significance of archiving, which leads to disarray in the distribution and arrangement of files.

4. Use of non-specialized tools: Relying on Word and Excel does not provide effective document management.
5. A large number of documents are stored in paper archives without electronic archiving.
6. A large number of licenses are issued but not electronically archived.

1.3 Method

To build the Birzeit Municipality Archiving System, we followed a practical and structured development methodology, starting from understanding the municipality's needs and ending with a working digital solution. Our goal was to create a system that solves real problems, is easy to use, and fits smoothly into the daily work of municipal employees.

We began by meeting with people who work at the municipality, employees, supervisors, and technical staff to learn more about how they currently manage documents and where the biggest issues lie. These conversations helped us identify specific problems, such as missing documents, poor organization, and the time wasted searching through paper records. Based on these findings, a detailed set of functional and non-functional requirements was derived to clearly define system behavior, usability expectations, and security constraints.

Following the requirements analysis, we began with the designing the system phase. This phase involved developing architectural diagrams and database schemas to illustrate system components and their interactions. Using MySQL to design the relational database to store structured data and document records, while **GraphDB** combined with the **Resource description framework(RDF)** as adopted to model semantic relationships among documents. During document archiving, metadata attributes and contextual information are extracted and transformed into **RDF triples**, forming a semantic knowledge graph that captures conceptual connections between archived records. This dual-database architecture enables efficient keyword-based retrieval alongside advanced semantic reasoning.

Meanwhile, in the front-end interface, a clean and intuitive design was implemented using **HTML, CSS, and JavaScript**, ensuring ease of use and minimal training requirements for the municipal employees. The backend system was developed using **Python and the Flask framework**, providing a modular and scalable architecture that integrates MySQL for transactional processing and GraphDB for semantic operations.

An important feature we plan to implement is semantic search. This feature will allow users to find documents based on their meaning and context, not just exact keywords. By leveraging RDF-based semantic modeling and SPARQL queries, the system can interpret user intent and retrieve semantically related documents through the knowledge graph. For example, a user could search by department name or category, or can search using the metadata; either way, a result will be displayed. This approach significantly improves

retrieval accuracy, enhances contextual relevance, and reduces search time.

Security was a key consideration in the design; we will build a system that is role-based access that gives different permissions to different types of employees, meaning different levels of access are assigned based on user roles, such as municipal employees, department heads, and secretaries. This approach ensures both data confidentiality and integrity.

In conclusion, the adopted methodology emphasizes user-centered design, semantic modeling, and secure system architecture. By collaborating with the municipal staff and following a structured development lifecycle, the project delivers a robust digital archiving system that enhances organization, accessibility, and efficiency. Feedback from end users is incorporated throughout the development process to refine system features and ensure alignment with real operational needs.

1.4 Aim and Objectives

The main goal of this project is to build a new, simple, and safe system for Birzeit municipality to save and manage documents. This new system will work alongside the old paper-based way of working. It will help make the work faster, more organized, and easier for everyone to use. The system will also be able to grow in the future as more documents are added. The objectives include:

Semantic search: We want to make it easier for employees to find documents. They won't need to remember the exact file name or words. They can search using simple terms, like the department or topic, and still find the right document. This is done using semantic search with GraphDB and RDF triples, which understand the meaning behind search words, not just exact matches.

Digitize and organize: A lot of documents are still stored on paper or in random Microsoft Word/Excel files. Our system will fix that by converting everything into digital files and organizing them in a smart way. Documents will be sorted by department and category, making it easier for employees to find the right file when needed and reducing the need for physical storage space.

Role-Based Access Control (RBAC): Not everyone in the birzeit municipality is allowed to view or edit all documents. That's why our system will have role-based access control, so users only see what they are allowed to. The system has four roles: Administrative Affairs (full access), Department Heads (approve and manage their department), Municipal Employees (archive and view), and Secretary (view and search only). This ensures privacy and that sensitive information is safe.

User-Friendly Design: When we make the system, we should remember that not everyone knows a lot about computers. The screen should look clean and simple, so anyone can use it without needing much help. Each role gets their own dashboard with only the tools they need. This way, more workers will be able to use it easily without

training.

Metadata management: When a user archives a document, useful details are added like title, author, date, and type. This is called metadata, and it helps keep things organized. Users can also edit the metadata if something needs to be changed or updated. Administrative affairs can even create new metadata fields for different document types to match what the municipality needs.

Prevent document loss: One of the big challenges is documents missing or never being archived in the first place. Our system will track everything the moment it's uploaded, which helps prevent loss. It also makes it clear who uploaded the document and when. Every action is saved in a log that cannot be changed or deleted.

Approval workflow: Not all employees can archive documents directly. Regular municipal employees need to send their documents for approval first. The system automatically sends these requests to the department head or administrative affairs depending on the document type. This makes sure that only authorized people can add documents to the archive and keeps everything organized and controlled.

Version control: When someone edits a document, the system doesn't delete the old version. Instead, it saves a new version and keeps the old one for reference. This way, if there's a mistake or someone needs to see what changed, they can look at the history. Each version has its own ID and date so it's easy to track changes over time.

Audit logging: The system automatically records every action that happens, like who viewed a document, who edited it, and when it was archived. Only administrative affairs and the mayor can see these logs. This creates a clear record of all activities and helps with accountability. If something goes wrong, the logs show exactly what happened and who was responsible.

Report generation: Administrative affairs and department heads can create reports to see how the system is being used. Reports show things like how many documents were archived in a certain time period, which departments are most active, and how many requests are pending. These reports help managers make better decisions and see if the system is working well.

Department and category management: Administrative affairs can add or change departments and document categories in the system. This lets the system match the actual structure of the municipality. If a new department is created or a category name needs to change, it can be updated easily. This flexibility means the system can adapt as the municipality grows or changes.

1.5 Overview of the technical area

The Birzeit Municipality Archiving System (BMAS) is an electronic platform developed to modernize the management and retrieval of municipal documents. The system integrates multiple technical areas, including document management, semantic search, role-based access control, and secure database systems.

The system will be built using HTML5, CSS3, Bootstrap5, and JavaScript for the front-end and python and flask with MySQL and graphdb for the back-end. This technology package ensures a secure and user-friendly system. The system allows users to archive PDF files, which are automatically tagged with metadata such as title, date, and section, and stored in a central database for quick and organized access.

A standout feature of the system is its use of semantic search, which goes beyond basic keyword matching by considering the meaning behind a user's query. By applying natural language processing and smart indexing methods, it can find relevant documents even when search terms are vague or incomplete.

A key component of the system is maintaining data security and privacy through the implementation of a Role-Based Access Control (RBAC) system, which restricts access to documents based on employee roles.

1.6 Overview of the report

This report provides a comprehensive and detailed account of the development process of the Birzeit Municipality Archiving System, a digital solution designed to modernize document management for the municipality.

The report begins with:

- **1. Literature review** showing existing systems and scholarly articles to establish a theoretical foundation for the project.
- **2. Semantic search and indexation** combine semantic search and indexation with municipal workflows.
- **3. System design** architecture and functionality, outlining how the system will be built.
- **4. Comparison to systems** highlighting how our system stands out from other and similar systems about archiving.
- **5. Testing** phase for testing the website features and make sure that it works as expected.
- **6. implementation** the phase of implementing the project giving it user-friendly interfaces and RBAC.
- **7. Conclusion** an overview of our project conclusion, what we have achieved and creating a functional website for the municipality.

Summary

This chapter introduce the Birzeit Municipality Archiving System and identified the main problems with the current paper-based system: poor organization, document loss, use of Excel and Word for archiving, and lack of staff training. We explained our goal to create a digital archiving system using HTML, CSS, JavaScript, Python, flask, GraphDB, and MySQL. The system will include semantic search, role-based access control, metadata management, and an easy-to-use interface. We also outlined the project plan and described the technical areas involved.

Chapter 2

Background

This chapter explains the theories and background information behind our system design. First, we discuss important concepts like document management, semantic search, metadata indexing, and role-based access control. Then, we look at similar systems like OpenKM and SeedDMS to learn from their features and find gaps our system can fill. Finally, we present the tools and technologies we chose and explain why they work well together. This background prepares you for the detailed system design in Chapter 3.

2.1 Details of relevant theory

Introduction

As with many other municipalities, Birzeit Municipality relies on the traditional archiving system, but it wants to develop the archiving process into a digital archiving systems that allow for the transformation of physical records into protected and easily searchable files. Semantic search and metadata indexing are useful features that ease document management, retrieval, and preservation in any organization. For Birzeit Municipality, role-based access control modern services, alongside automated record classification and access logs, ensure compliance within an organization while maintaining maximum protection to the system's integrity.

Problem

Traditional archiving documents, as practiced by Birzeit Municipality, prove to be highly inefficient. Because of having hundreds of thousands of physical records, most of which are prone to damage and loss, employees have a routinely difficult task locating important files. The disorganization results in delays and interruptions across various departments, leading to poor and inconsistent access to vital southern mesopotamian municipal information. The described situation reflects bigger challenges that exist globally, such as reliance on keyword-based searches, which ignore user needs. As a result, the search becomes irrelevant and time-consuming [1]. Moreover, attempts to digitize documents often create “digital stacks of paper” instead of well-organized digital archives due to a lack of proper structural metadata [2].

Solution

The integrated solution brings together semantic technologies and metadata management,

thus solving the problems caused by Birzeit Municipality Archiving System. Within the system, document retrieval is performed intelligibly through reasoning about the contextually related entities' interconnections using RDF (Resource Description Framework) triples [3], which is a framework that represents data as subject–predicate–object relationships to form a semantic graph, enabling flexible data linking, better search accuracy and interoperability. For search functionality, we implement a hybrid approach merging traditional keyword indexing with content analysis and semantics of the documents and their metadata. During upload, classification into structured metadata fields like department, document type, date of creation, and relevant words is conclusively made and indexed for precise retrieval [4]. Users are aided through non-technical interfaces involving faceted search coupled with document previews and visual relationship mapping to archival systems. Role-based access controls [5] applied to secure servers provide bounded enclosure of documents while sustaining workflows enabling comprehensive digital archiving system solutions enhanced around municipal expectations.

Conclusion

This document demonstrates the potential of advanced semantic search capabilities, thorough metadata cataloguing, and multi-tiered security access to revolutionize municipal document management systems. In solving Birzeit's concerns, the system simultaneously integrates best practices from prior research in archival science. Collection and retention objectives combined with operational efficiency will be enhanced through technology integration which supports long-term digital service development within the municipality

2.2 Review of past/reported work

In reviewing past work, we found two systems stand out for their relevance to our project:

OpenKM Document Management System

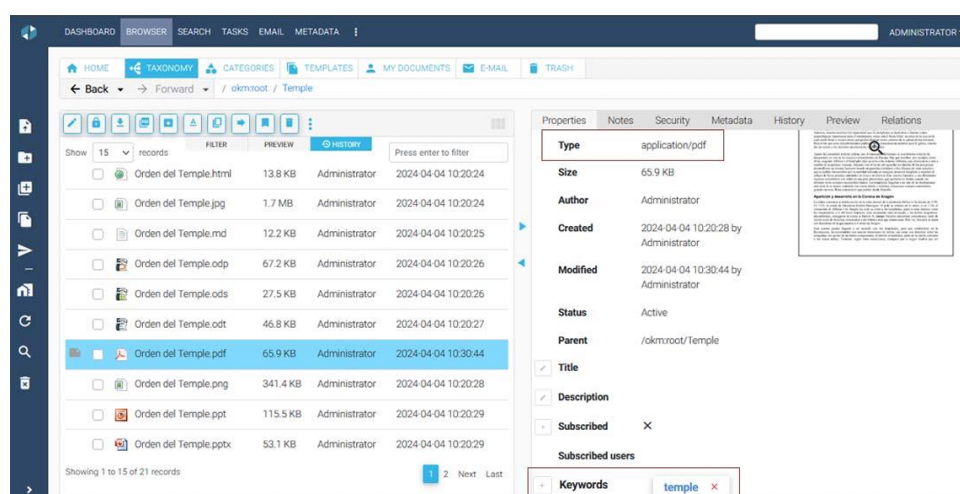


Figure 2.1: Role Management interface in OpenKM

As shown in Figure 2.1, OpenKM is a web-based document management system that supports both large and small organizations in storing and retrieving documents

efficiently.

Similarities to our system:

- **Semantic Search:** Supports metadata-based and full-text search, helping users find documents quickly without the need to know the file names exactly.
- **User-Friendly Interface:** Designed to be accessible for users with minimal tech skills.
- **Role Management:** Enables administrators to define detailed user roles and document access roles.

SeedDMS (Seed Document Management System)

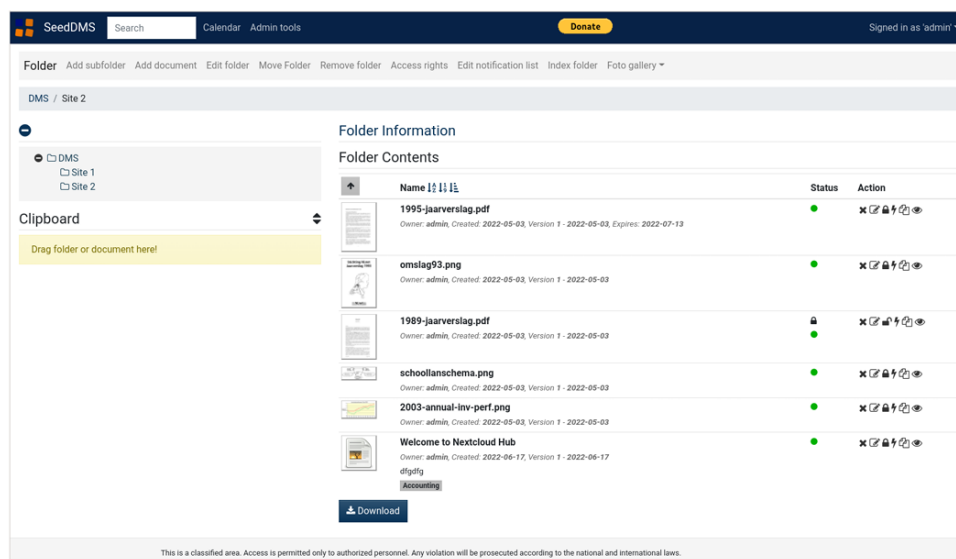


Figure 2.2: Document structure and interface in SeedDMS

SeedDMS is an open-source document management system that offers a full-featured platform for storing and retrieving documents in a secure and user-friendly environment. It supports a hierarchical structure for document organization and integrates essential features such as metadata tagging, version control, and role-based access.

The system includes search functionality that allows users to search not only by filename but also by metadata and content, making retrieval faster and more precise. Furthermore, it supports document versioning and audit trails, which are essential for maintaining document integrity and accountability — a key concern for municipal operations.

Figure 2.2 shows the seedDMS structure and how documents are arranged within folders, as well as the actions that can be performed on documents.

2.3 Tools and Technology

To build and develop the Birzeit Municipality Archiving system, we will use some of the following important tools and technologies to support the design objectives.

1. Web Technologies:

to enhance the system's usability and provide a clean, user-friendly interface, the following web technologies were used to design and implement the front-end of the system:

- Figma: A tool for implementing our ideas as a raw design.
- HTML5, CSS3, and Bootstrap5: For structuring and styling the user interface.
- JavaScript: Adds interactivity, form validation, and dynamic content rendering.

2. Development Tools:

to implement a good back-end logic that supports the system development:

- VS Code: Primary IDE for coding and debugging.
- GitHub: Version control and team collaboration.
- Python: Backend programming language used to implement system logic.
- Flask: Python web framework used to build the server and handle HTTP requests.

3. Database and data storage:

A combination of databases and relations to ensure the efficiency of data management and document storage:

- MySQL (via Xampp): the original database used for storing the structured data.
- GraphDB: supports the semantic search by storing the relationships between documents and related entities in triples.
- Local file storage: to store the documents.

4. Security and Access Control:

to have more control on sensitive information and protect it:

- Role-Based Access Control (RBAC): Guarantees that only authorized documents and actions are accessible to each user.
- Password Encryption: To protect the storage of credentials

Summary

This chapter explains the theories behind our system design. We discussed how semantic search, metadata indexing, and role-based access control work together to solve document management problems. We reviewed two existing systems: OpenKM, which has strong role management and metadata search features, and SeedDMS, which offers hierarchical organization and version control. We also presented the tools and technologies we selected, including HTML5, CSS3, Bootstrap5, and JavaScript for the front-end, Python and Flask for the back-end, and additional tools like Figma for design and GitHub for version control.

Chapter 3

System Analysis and Design

This chapter presents the detailed design of our system. We start by describing the system's main features, objectives, and requirements. Then, we show the actors who will use the system and the use cases that describe how they interact with it. After that, we present UML diagrams including class diagrams, sequence diagrams, activity diagrams, and state charts that show how the system works step by step. We also explain the system architecture, software design, and database structure. By the end of this chapter, you will have a complete understanding of how the Birzeit Municipality Archiving System is designed and how all its parts work together.

3.1 System Model and Architecture

3.1.1 Product Description

System Objectives

- **Enhance Document Accessibility:** The ability to retrieve data quickly and accurately through semantic and metadata searches.
- **Ensure Document Security:** Protect data by controlling access to documents based on user role.
- **Improve document Organization and Categorization:** Place documents in specific categories and add metadata to organize them and make them easier to find.
- **Optimize archiving workflow:** Replace traditional document archives with a digital archiving system to reduce the effort and time involved in document archiving and retrieval.
- **Promote Staff Awareness:** The system provides clear instructions and guidelines on how to use it, enabling all employees to understand and apply archiving effectively.

System Main Features

- **Document management system:** a core platform for storing, organizing, and managing all municipal documents digitally.
- **Semantic search:** to understand the context and meaning of words, allowing users to retrieve documents using meaningful queries beyond simple keyword matching.
- **Role-based access control:** a system to restrict access to documents and system functionalities based on predefined user roles, ensuring data privacy and preventing unauthorized access.
- **User Interface:** A user-friendly interface designed for ease of use by various staff members.
- **Document store format:** For storing documents in PDF formats.
- **Metadata:** Manual tagging of documents with metadata, for easier categorization and retrieval.
- **Centralized database:** A database to store both the files and their related metadata.

Operating Environments

Software

- It can be run on Windows, and Linux devices.
- It can be run on most browsers like Chrome, Opera, etc.

Hardware

- Minimum 8 GB RAM.
- Minimum 1280 * 720 screen size of the device

Network

- Minimum 10 Mbps for optimal performance.

Constraints

- The system must fully comply with local data privacy, it should handle sensitive documents securely.
- The system must work with the municipality's current IT setup and store documents in the local file server.
- Access to documents and actions within the system must be controlled based on user roles.
- Documents must be correctly tagged and indexed with the appropriate metadata.
- The system should run in a secure and reliable file server with regular backups in place to prevent any risk of data loss.
- The system will not work offline, users must be connected to the municipal network to access and interact with the platform.

Functional Requirements**SR1: Authentication**

SR1.1 Authentication.Login	The system shall allow municipal employees to log in using valid credentials (email and password)
SR1.2 Authentication.Login.Invalid	If invalid credentials are entered, the system shall display an error message and prompt the user to try again.
SR1.3 Authentication.Login.Failure	If the login service is unavailable, the system shall display an error message and advise the user to try again later.

Table 3.1: Authentication Requirements

SR2: Document Archiving

SR2.1 Archive.document	The system shall allow authorized users to upload and archive documents with metadata and keywords.
SR2.2 Archive.document.approval	Archived documents by regular municipal employees shall require approval from the department head, and the department head shall then submit to the administrative affairs for administrative documents.
SR2.3 Archive.document.DocType	The system shall support file type: .pdf
SR2.4 Archive.document.failure	The system shall display an error message if metadata is missing or invalid, and prompt the municipal employee to correct the input.

Table 3.2: Document Archiving Requirements

SR3: Search for a Document

SR3.1 Search.Document	The system shall allow authorized users to search for documents using metadata criteria.
SR3.2 Search.Document.Results	The system shall display matching documents with options to view or edit permissions.
SR3.3 Search.Document.NoResults	If no results are found, the system shall notify the user and allow them to refine their search or exit.
SR3.4 Search.Document.Permission	If the user lacks permission to view a document, the system shall prompt them to request access from the department head.

Table 3.3: Search for a Document Requirements

SR4: Role Management

SR4.1 Role.Change	Administrative affairs shall be able to change permissions for all roles.
SR4.2 Role.Log	All role changes shall be recorded in the audit log.

Table 3.4: Role Management Requirements

SR5: Version control

SR5.1 Version.control	The system shall track and manage multiple versions of the same document.
SR5.2 Version.tag	Municipal employees shall be able to manually tag document versions with comments.
SR5.3 Version.History	The system shall allow municipal employees to view the history of document changes.
SR5.4 Version.failure	If a version cannot be saved due to a system error, the system shall notify the user.

Table 3.5: version control Requirement

SR6: Audit log

SR6.1 Log.activity	The system shall log all municipal employees activities, including data and time stamp, user id, document id, and action type .
SR6.2 Log.access	Only administrative affairs shall have access to view the audit logs for all users.
SR6.3 Log.filter	The system shall allow filtering audit logs by specific metadata (e.g., date range, action type).
SR6.4 Log.failure	If the audit log database is unavailable, the system shall generate an alert and retry the connection.

Table 3.6: Audit log Requirement

SR7: Report generation

SR7.1 Report.Generate	The system shall allow authorized users to show the report of document activities within a specified time range.
SR7.2 Report.Export	Users shall be able to export reports in PDF format.
SR7.3 Report.NoData	If no documents were uploaded in the specified period, the system shall display a message indicating no activity.
SR7.4 Report.Failure	If the report generation fails, the system shall display an error message.

Table 3.7: Report generate Requirement

SR8: Workflow

SR8.1 Workflow.Approval	The system shall follow the workflow for document approval based on the user role.
SR8.2 Workflow.Reject	If a document is rejected, the system shall show the requester with comments of why it's rejected.
SR8.3 Workflow.Failure	If a workflow step is missing due to a system error, the system shall log the error.

Table 3.8: Workflow Requirement

Non-Functional Requirements**Performance Requirements**

Requirement	Description
Performance.Search	Search results shall load within 15 seconds.
Performance.Concurrency	The system shall handle concurrent searches and edits from multiple users without degradation in performance.

Table 3.9: Performance Requirements

Security Requirements

Requirement	Description
Security.Access	Only authorized users shall access specific functionalities based on their roles.
Security.Audit	Audit logs shall be immutable and fully accessible only to administrative affairs.
Security.data	All document uploads and edits shall be validated to prevent malicious input.

Table 3.10: Security Requirements

Usability Requirements

Requirement	Description
Usability.Training	The system shall be intuitive, with training provided to municipal employees for proper use.
Usability.Error	Clear error messages shall guide users in case of invalid inputs or system failures.

Table 3.11: Usability Requirements

Reliability Requirements

Requirement	Description
Reliable.Logging	The system shall ensure all activities are consistently and correctly logged, even during partial failures.
Reliable.Backup	Document versions and audit logs shall be backed up regularly to prevent data loss.

Table 3.12: Reliability Requirements

Scalability Requirements

Requirement	Description
Scalability.Documents	The system shall handle a growing volume of documents and users without performance degradation.
Scalability.Reports	Report generation shall efficiently process large datasets for extended time periods.

Table 3.13: Scalability Requirements

Compliance Requirements

Requirement	Description
Compliance.Metadata	Metadata schemas shall enforce consistency and support regulatory requirements.
Compliance.Audit	Audit logs shall meet municipal compliance standards for record-keeping.

Table 3.14: Compliance Requirements

3.2 Functional Decomposition

3.2.1 Actors

Actors	Description
Mayor	Requests documents from the secretary.
Administrative affairs	Has the ability to approve requests, manage user roles and their permission, manage department and categories hierarchy, search and views documents, can archive and edit documents.
Department head	Searches and views, approves, and manages documents within their department; has extended access over staff.
Municipal employee	Archives and categorizes documents using metadata, performs searches, views, edits, and requests to view documents from other departments.
Secretary	Has the ability to view and search documents.

Table 3.15: Actors and their description

3.2.2 Use Cases

Use case Id	Use case name	Use case description
UC-01	Login / Authenticate	Allows users to log in to the system with appropriate authentication.
UC-02	Archive	Enable users to store and archive documents with metadata.
UC-03	Edit	Allows authorized users to edit document metadata and document.
UC-04	Search	Provides users with the ability to search for documents using metadata.
UC-05	View	Enables users to view documents, with permission access by department head or administrative affairs.
UC-06	User role and permissions	Allows administrative affairs to modify user permissions and assign roles.
UC-07	Version control	Tracks and manages multiple versions of documents.
UC-08	Metadata	Enables administrative affairs to define and manage metadata schema for documents.
UC-09	Manage categories	Allows administrative affairs to create category, edit category to maintain hierarchy.
UC-10	Manage departments	Allows administrative affairs to create, edit municipal departments to maintain hierarchy.
UC-11	Audit log	Automatically logs system activities, available only to the administrative affairs.
UC-12	Generate report	Allows the administrative affairs and the department head to create reports on document activities within a specified time frame.

Table 3.16: Use case summary

3.2.3 Use Cases Diagram

A visual representation of the interactions between users (actors) and the system, showing the system's functionality from a user's perspective.

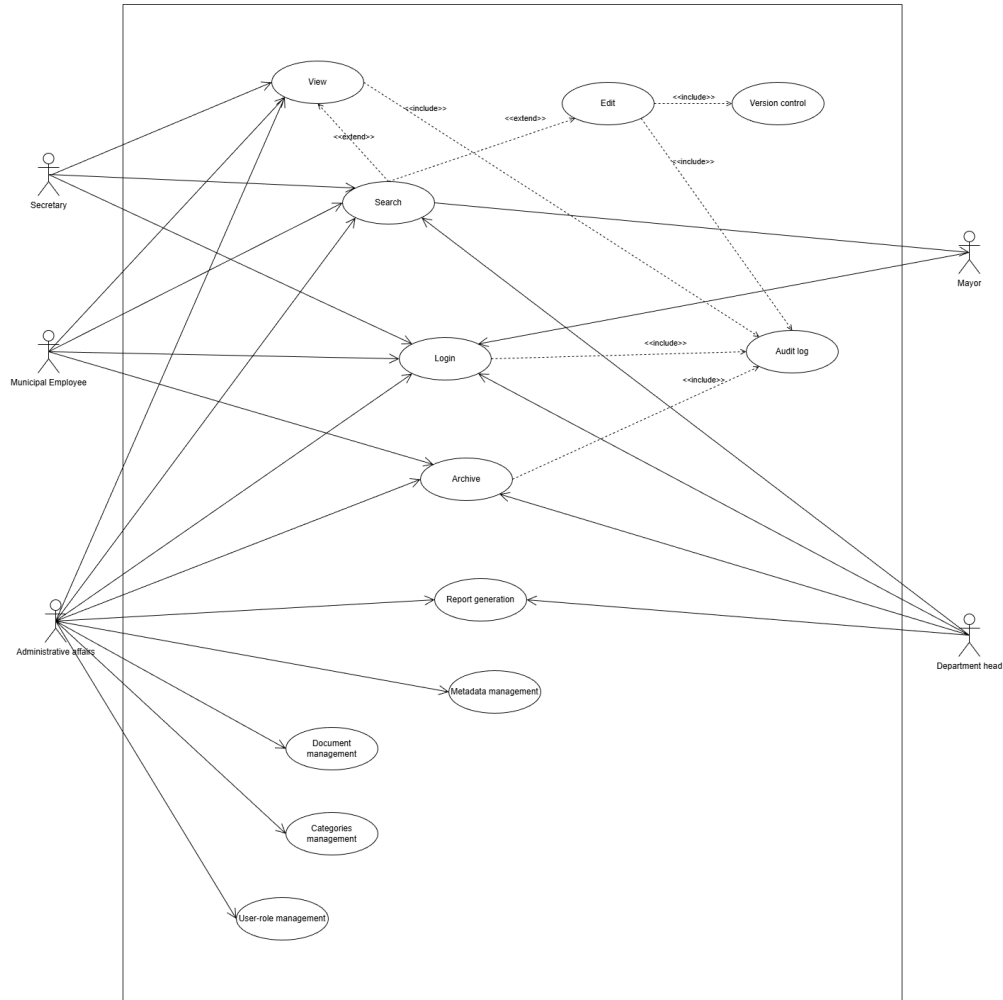


Figure 3.1: Use case diagram

Figure 3.1 shows the use case diagram that has the interactions between different users and core system functionalities.

3.3.2 Sequence Diagram

shows how objects interact in a specific scenario by depicting the sequence of messages exchanged over time. It visually represents the flow of operations from one object to another, typically for one use case.

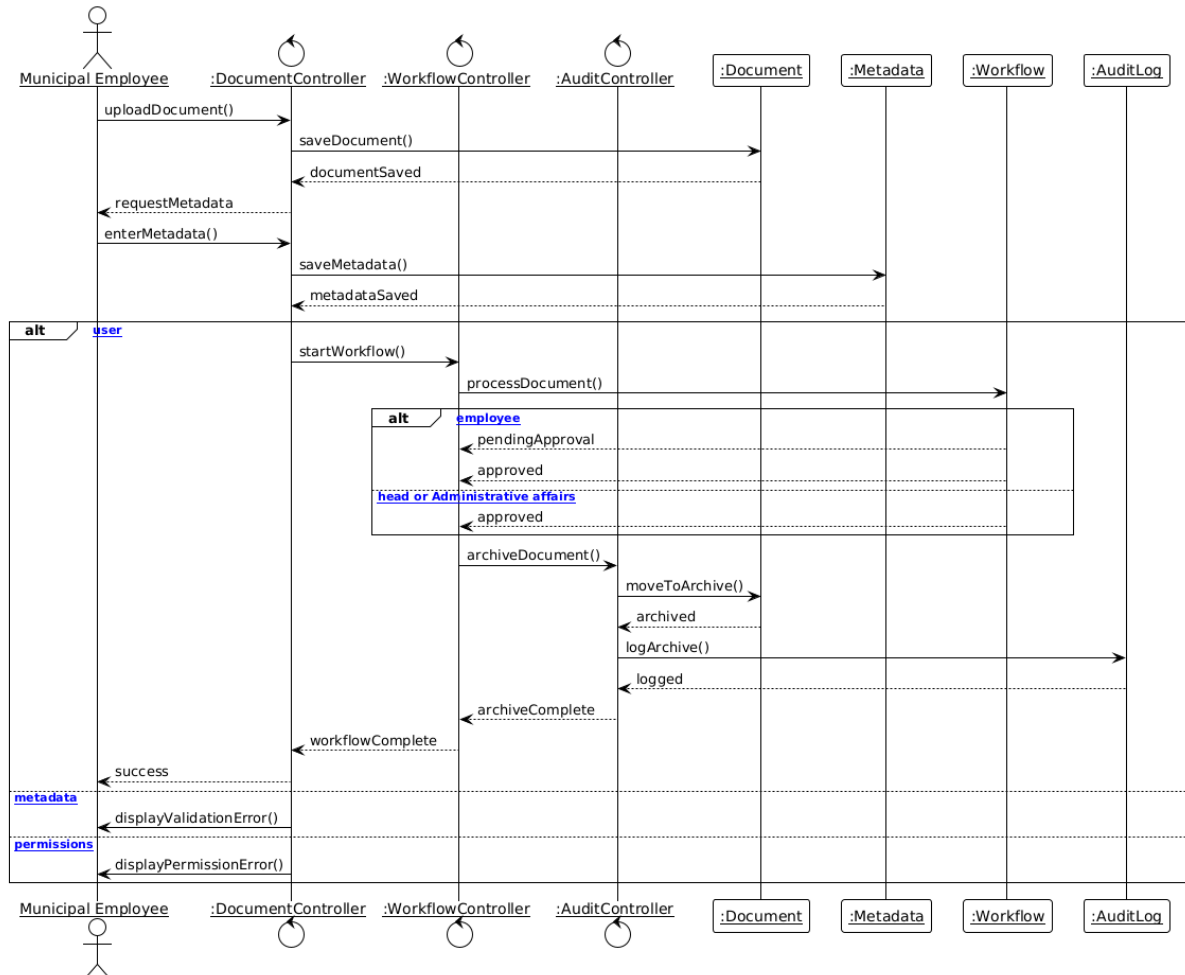


Figure 3.3: Archive sequence diagram

Figure 3.3 of the archive sequence diagram shows the flow that involves archiving a document by the municipal employee.

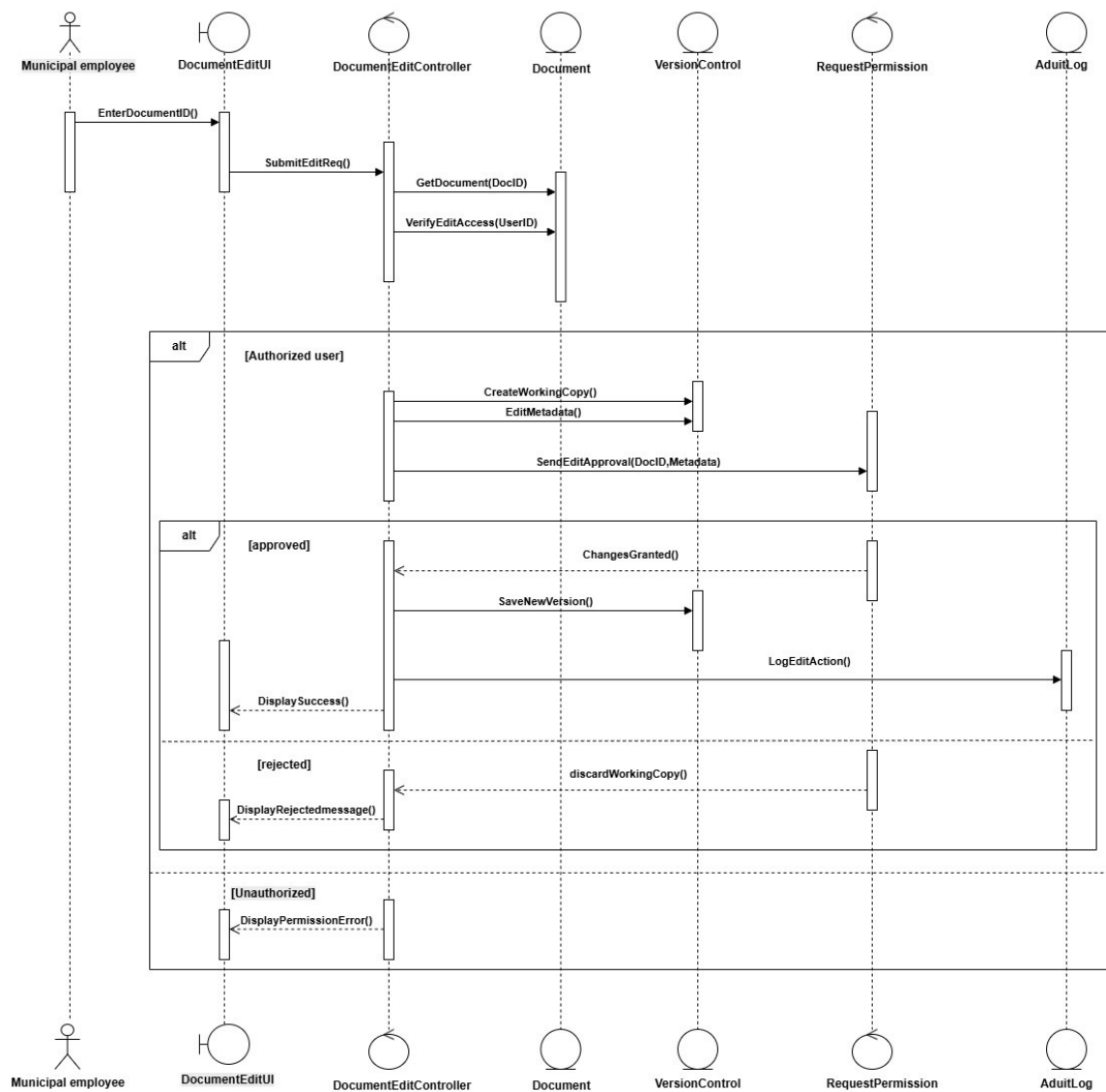


Figure 3.4: Edit sequence diagram

Figure 3.4 of the edit sequence diagram shows the flow that involves editing a document by the municipal employee.

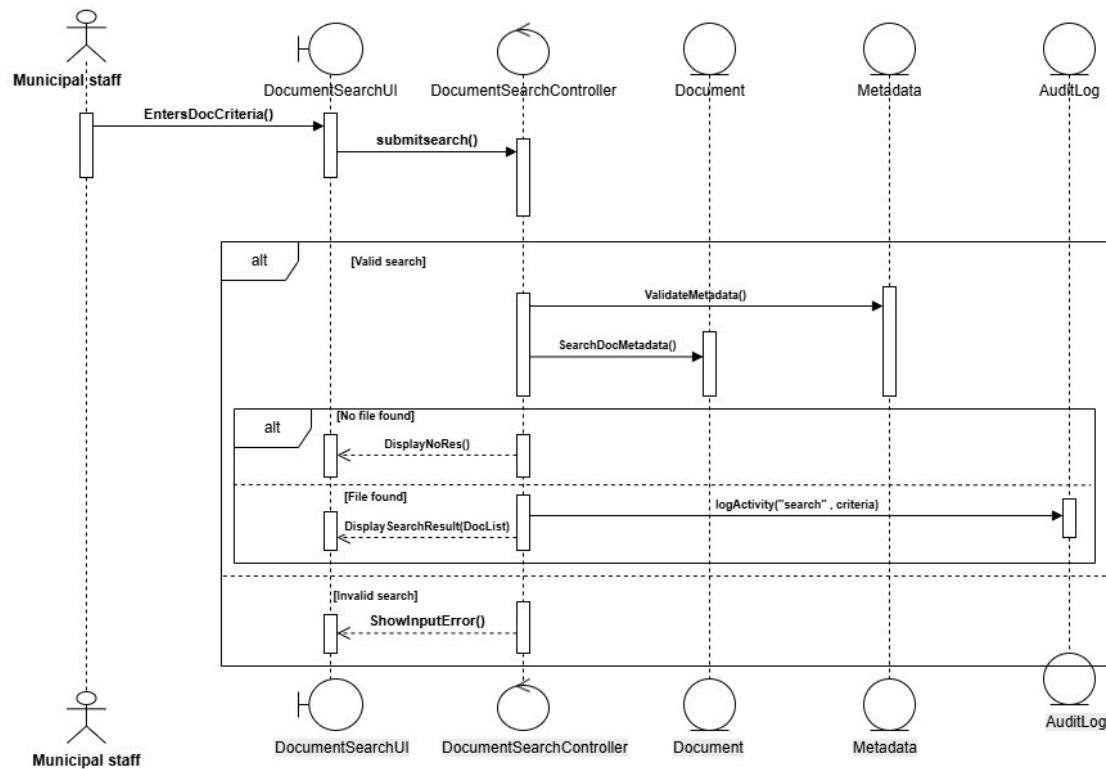


Figure 3.5: Search sequence diagram

Figure 3.5 of the search sequence diagram shows how the search for a document by the municipal employee is done.

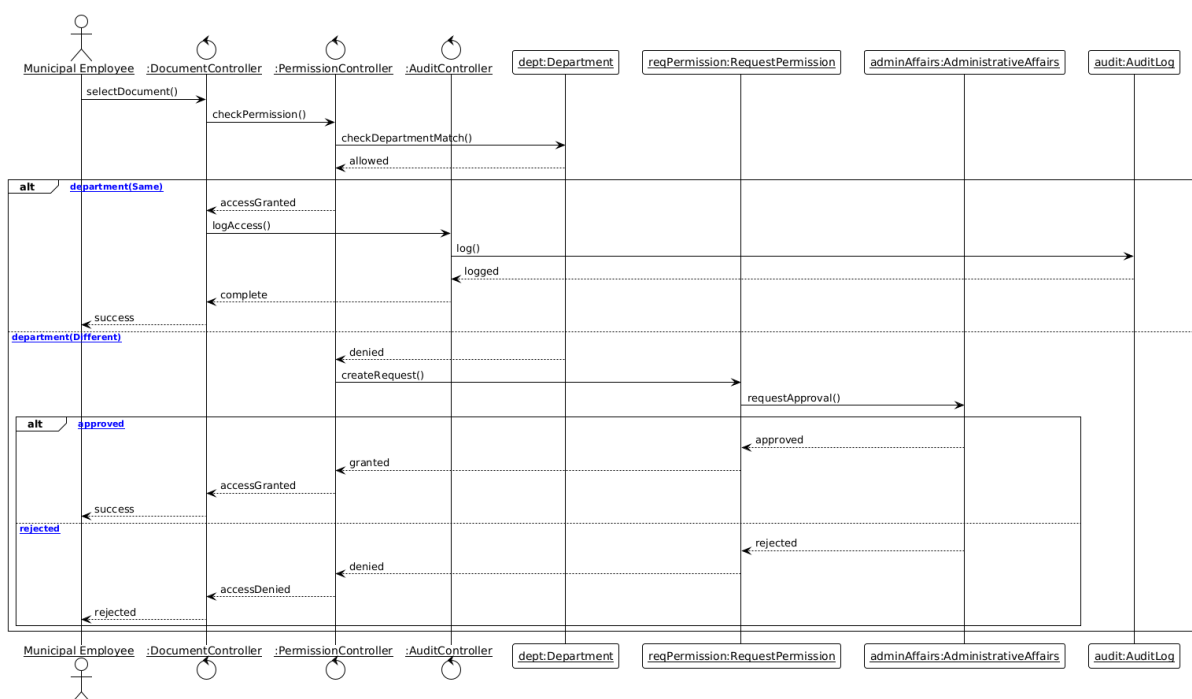


Figure 3.6: View sequence diagram

Figure 3.6 of the view sequence diagram shows the process of viewing the document by municipal employee.

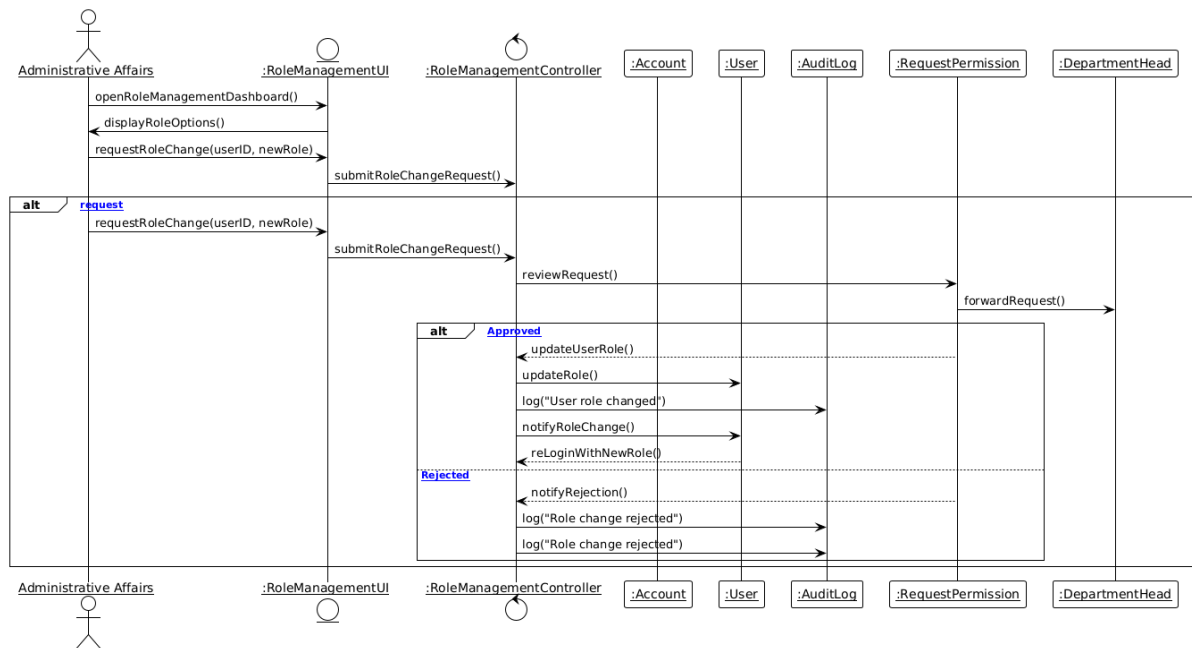


Figure 3.7: Manage user role sequence diagram

Figure 3.7 of the manage user role sequence diagram shows how the administrative affairs can manage and control the roles and permissions for each role.

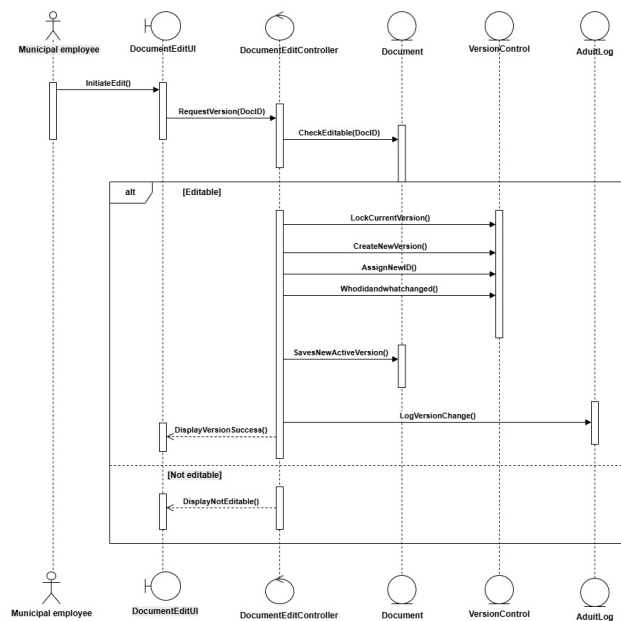


Figure 3.8: version control sequence diagram

Figure 3.8 for the version control sequence diagram shows how the system handles document editing and version control. The user searches for a document, edits it, and saves the changes. The system then creates a new version while keeping previous versions , and updates the version history for future reference.

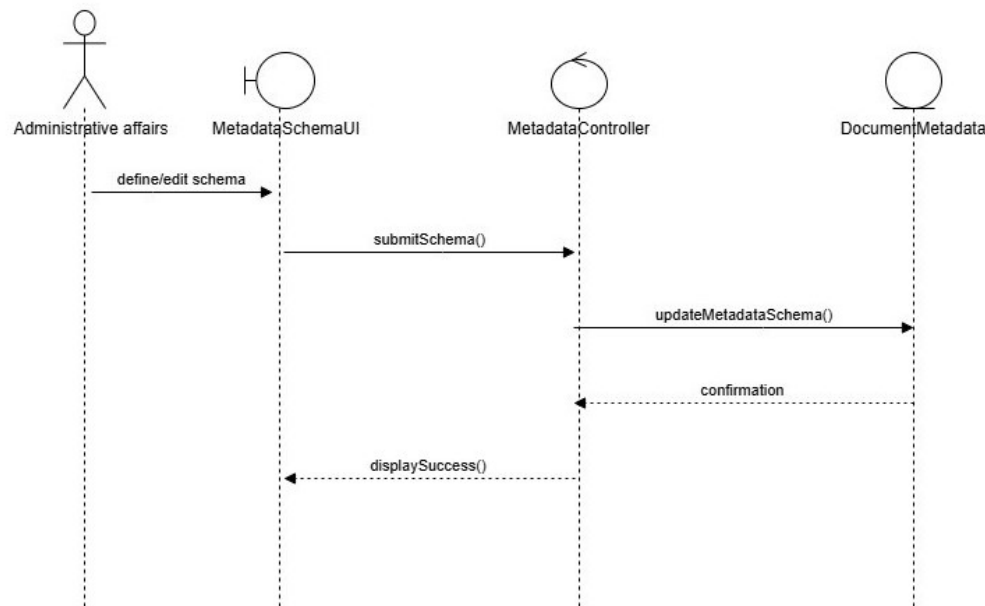


Figure 3.9: Metadata schema management sequence diagram

Figure 3.9 for the metadata management sequence diagram shows that the administrative affairs can add or edit the metadata from each category.

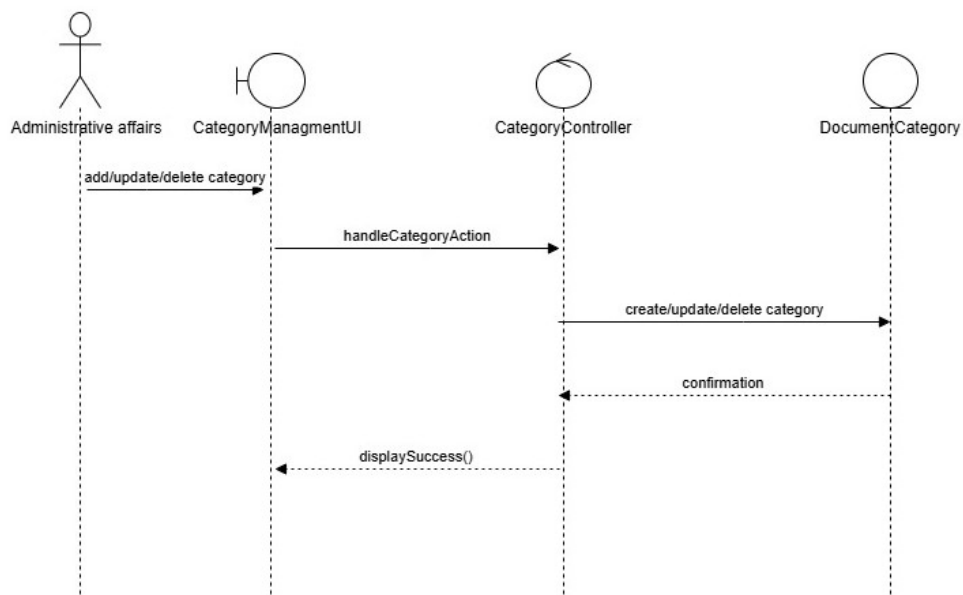


Figure 3.10: Manage document categories sequence diagram

Figure 3.10 for the categories management sequence diagram shows that the administrative affairs can add or edit categories from each department.

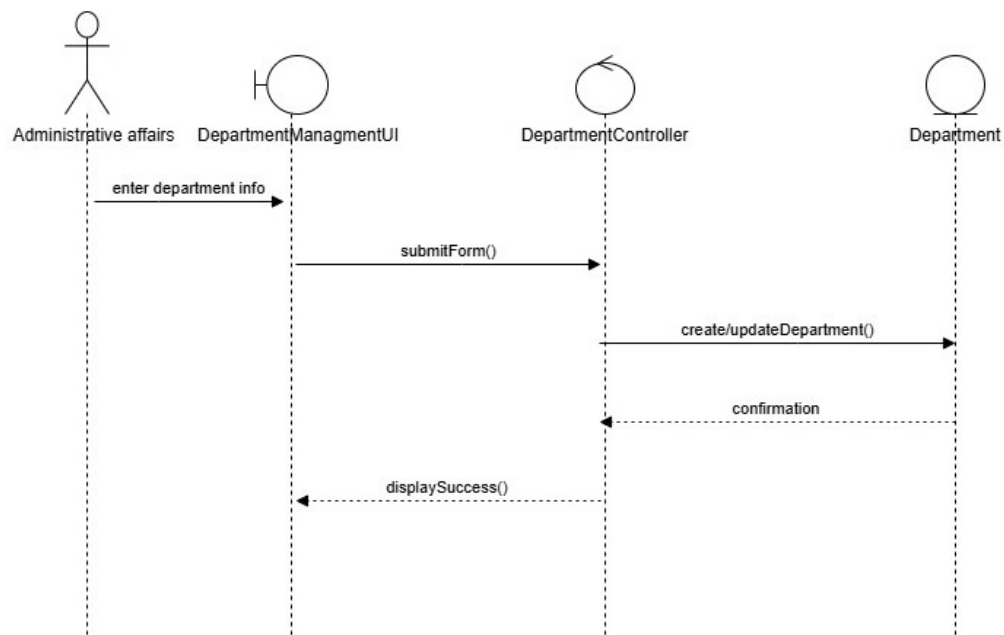


Figure 3.11: Manage municipal departments sequence diagram

Figure 3.11 for the department management sequence diagram shows that the administrative affairs can add or edit the departments from the system.

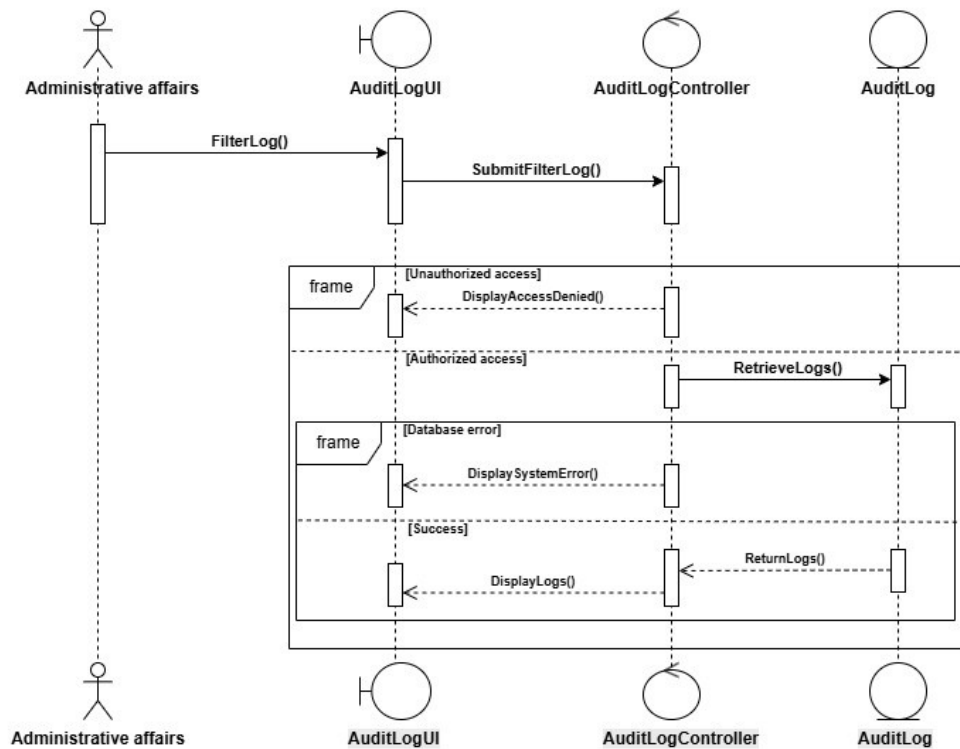


Figure 3.12: Audit log sequence diagram

Figure 3.12 for the auditlog sequence diagram shows that each step that the user takes in the system is saved in the database.

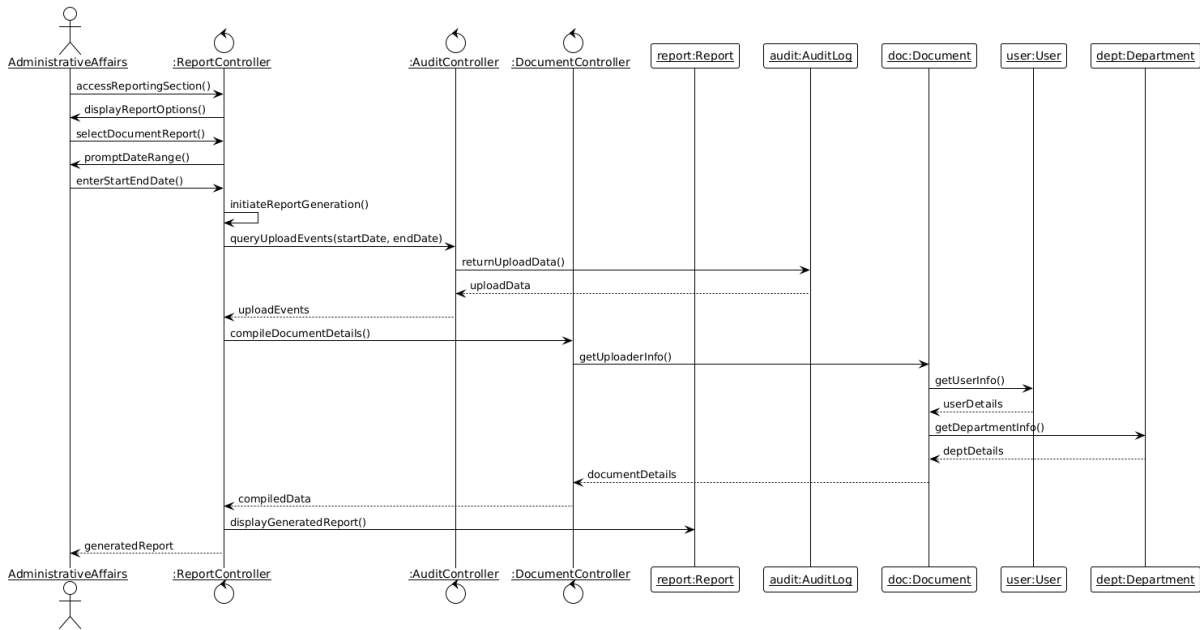


Figure 3.13: Generate report sequence diagram

Figure 3.13 for the report sequence diagram shows that the administrative affairs can generate a report for the system’s database, and they can filter by day.

3.3.3 Activity Diagram

shows the workflows of stepwise activities and actions with support for choice, iteration, and concurrency.

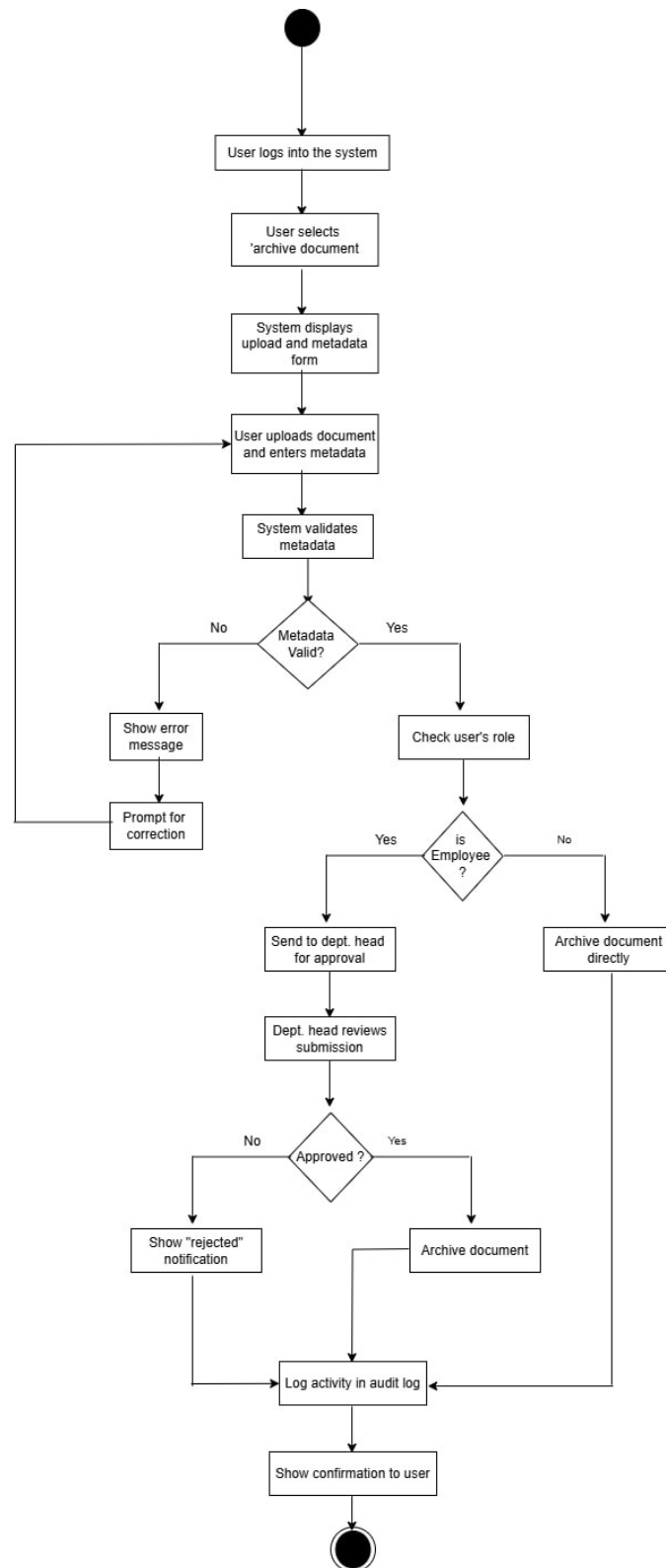


Figure 3.14: Archive activity diagram

Figure 3.14 shows the steps that a role could do to archive a document, from entering metadata, keywords, and the document pdf, if the role archiving a document is a municipal employee permission must be granted.

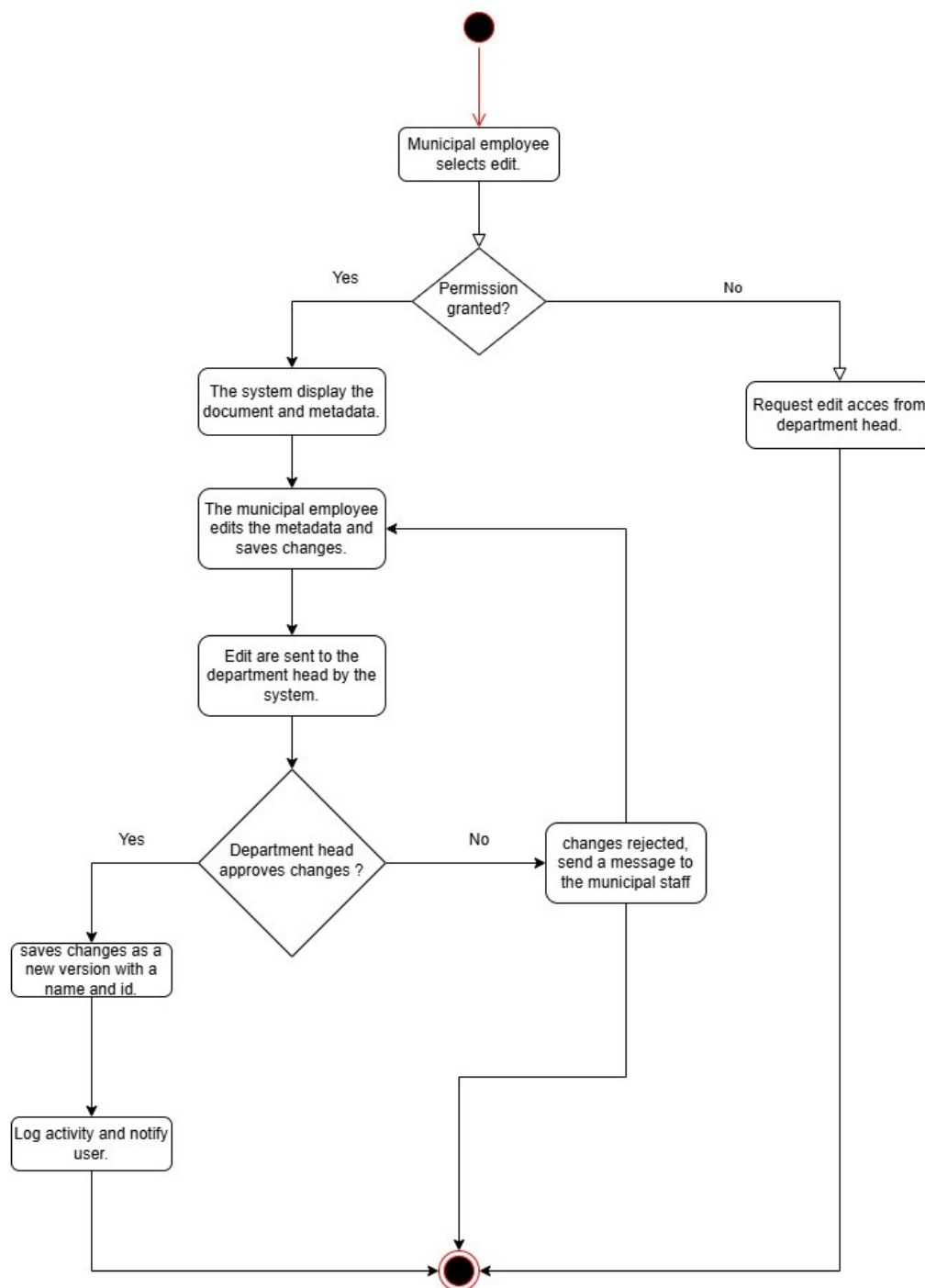


Figure 3.15: Edit activity diagram

Figure 3.15 the edit activity diagram shows the edit steps for editing a document.

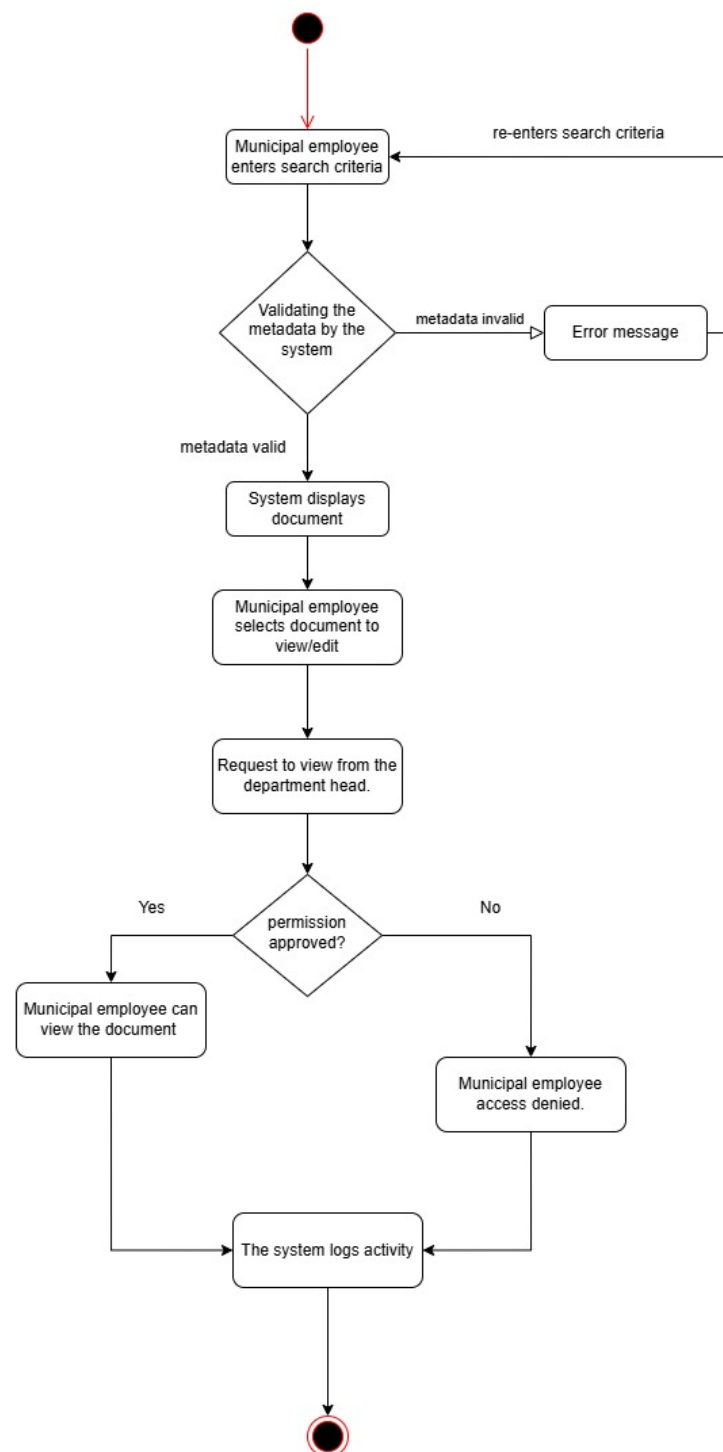


Figure 3.16: Search activity diagram

Figure 3.16 this diagram illustrate the search functionality.

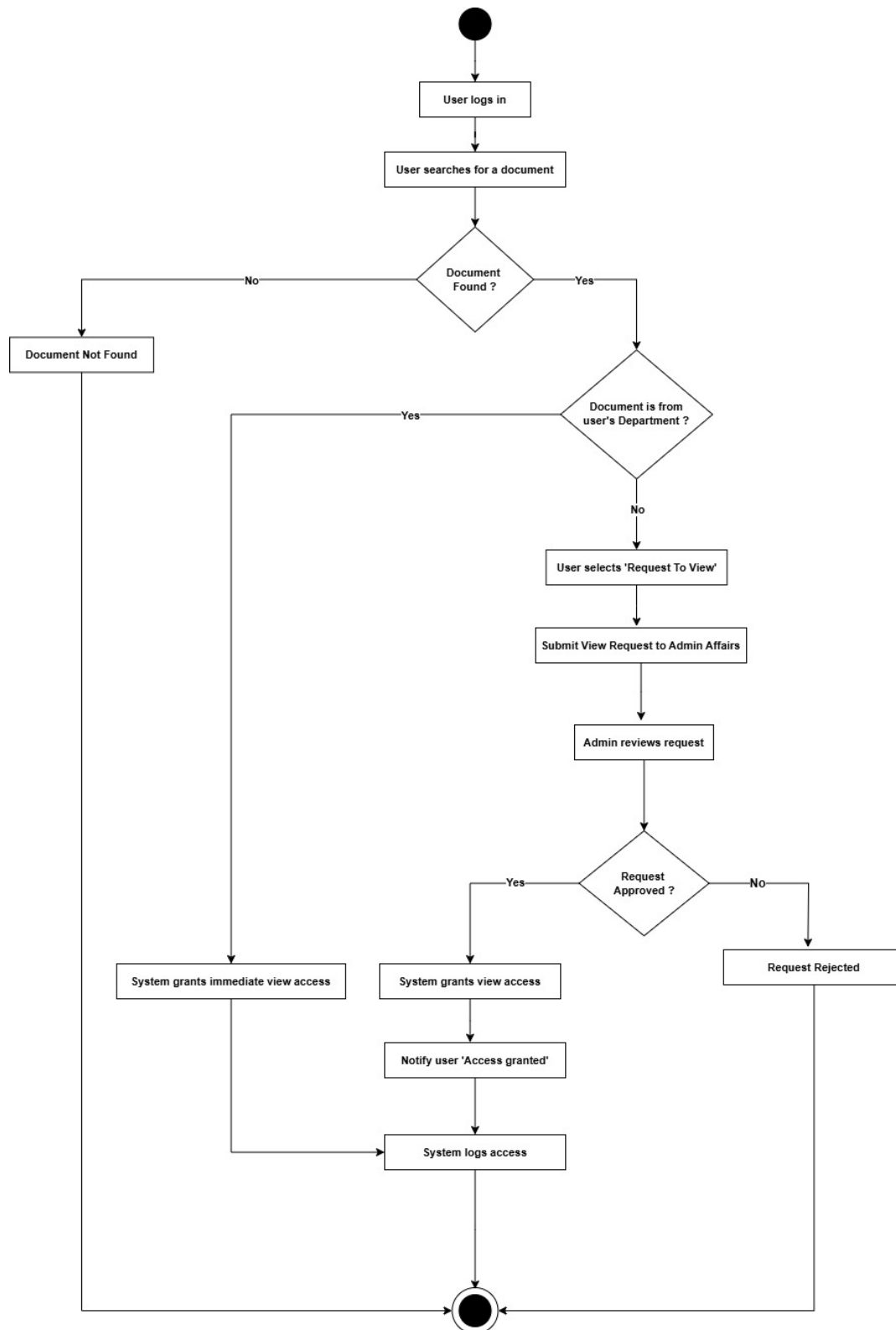


Figure 3.17: View activity diagram

Figure 3.17 the activity diagram to view a document under each category and department.

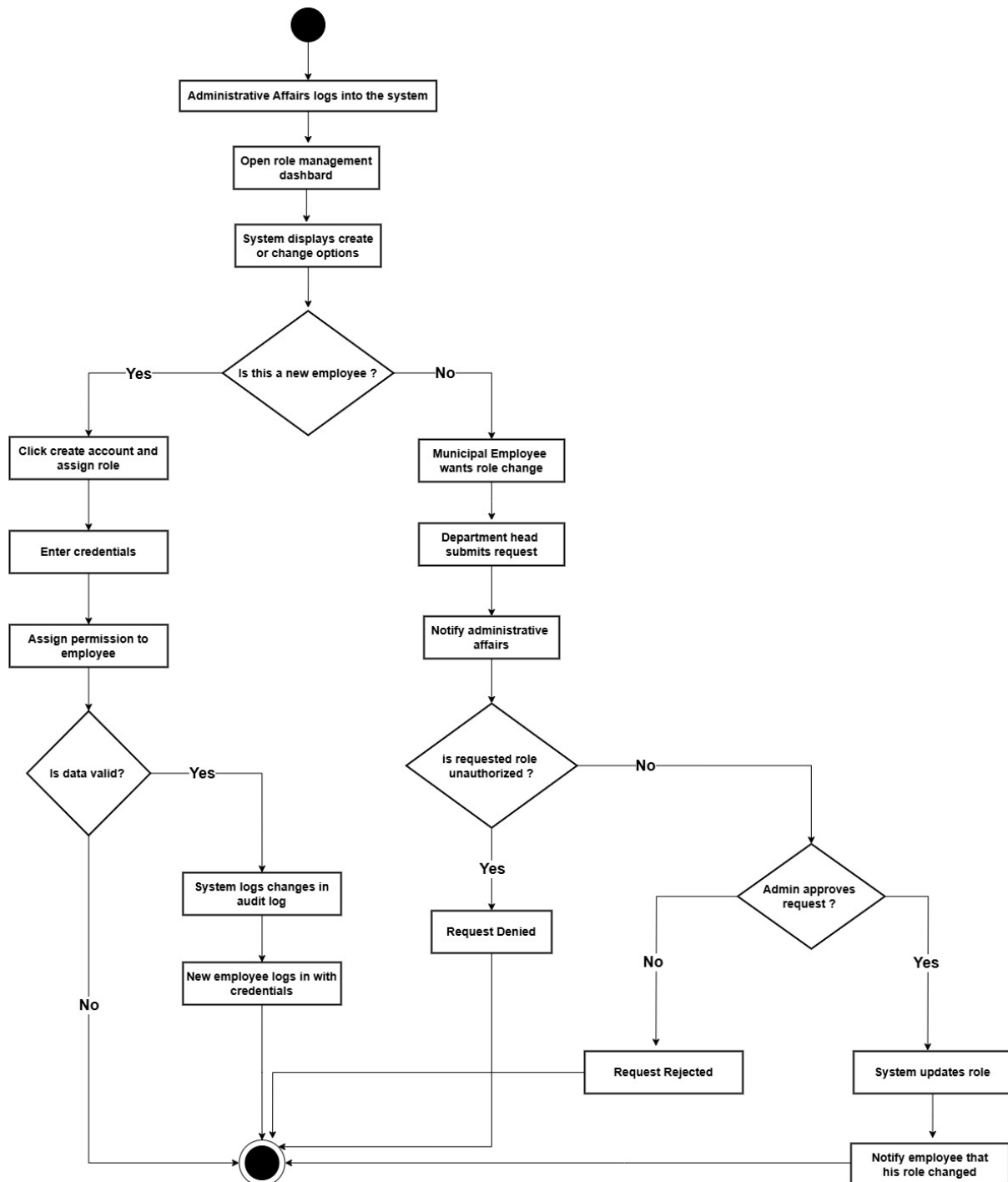


Figure 3.18: Manage user role activity diagram

Figure 3.18 the user role activity diagram shows that how the administrative affairs can create for the new employees accounts and each role has its own dashboard.

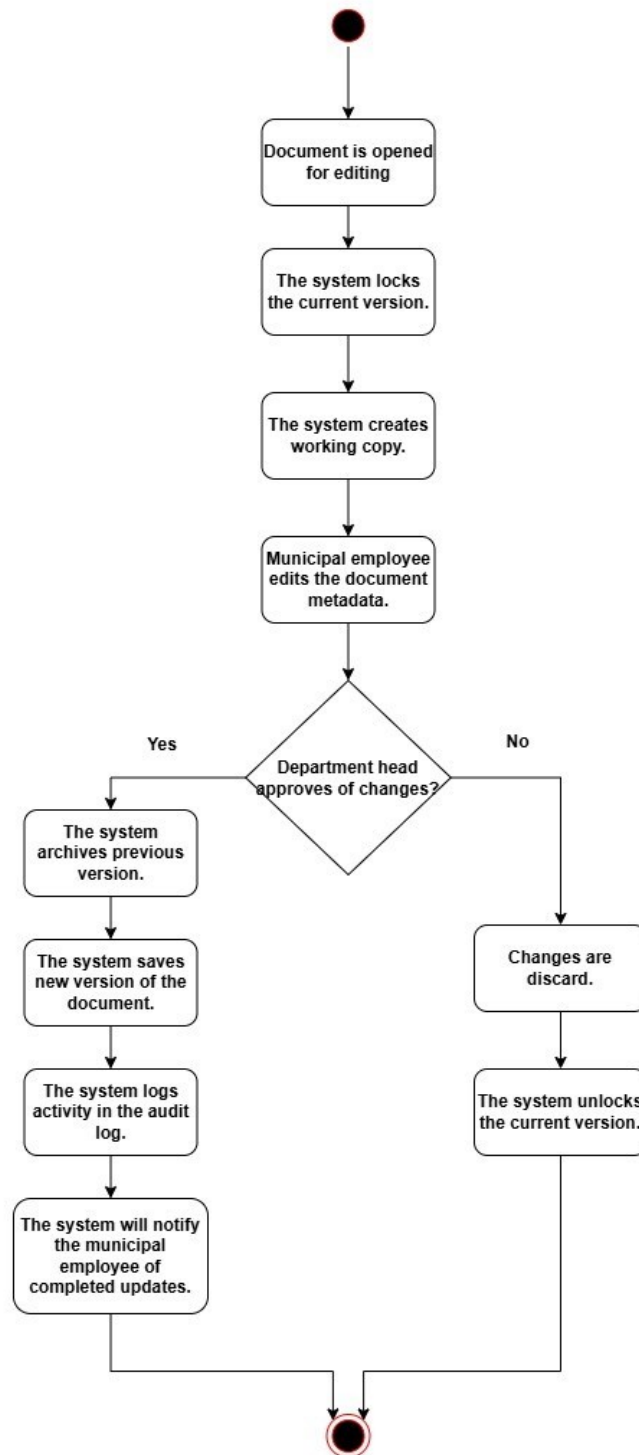


Figure 3.19: version control activity diagram

Figure 3.19 The version control activity diagram explains the steps of document editing and version control workflow. when a document is edited the current version is locked and a new version is created of the document only if the department head approves of the changes.

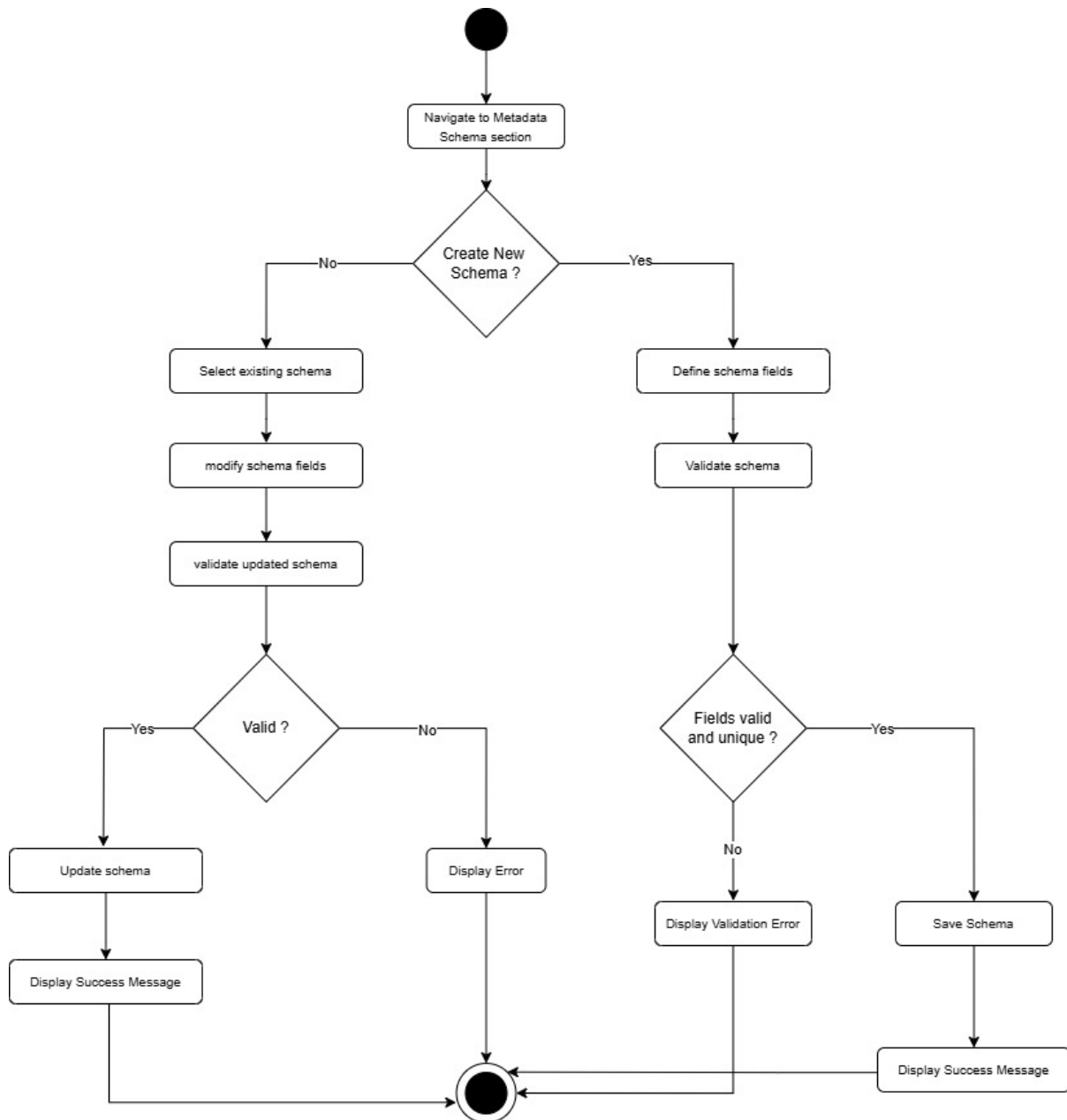


Figure 3.20: Metadata schema management activity diagram

Figure 3.20 This activity diagram of managing the municipality metadata, if a new metadata need to be defined in some category metadata.

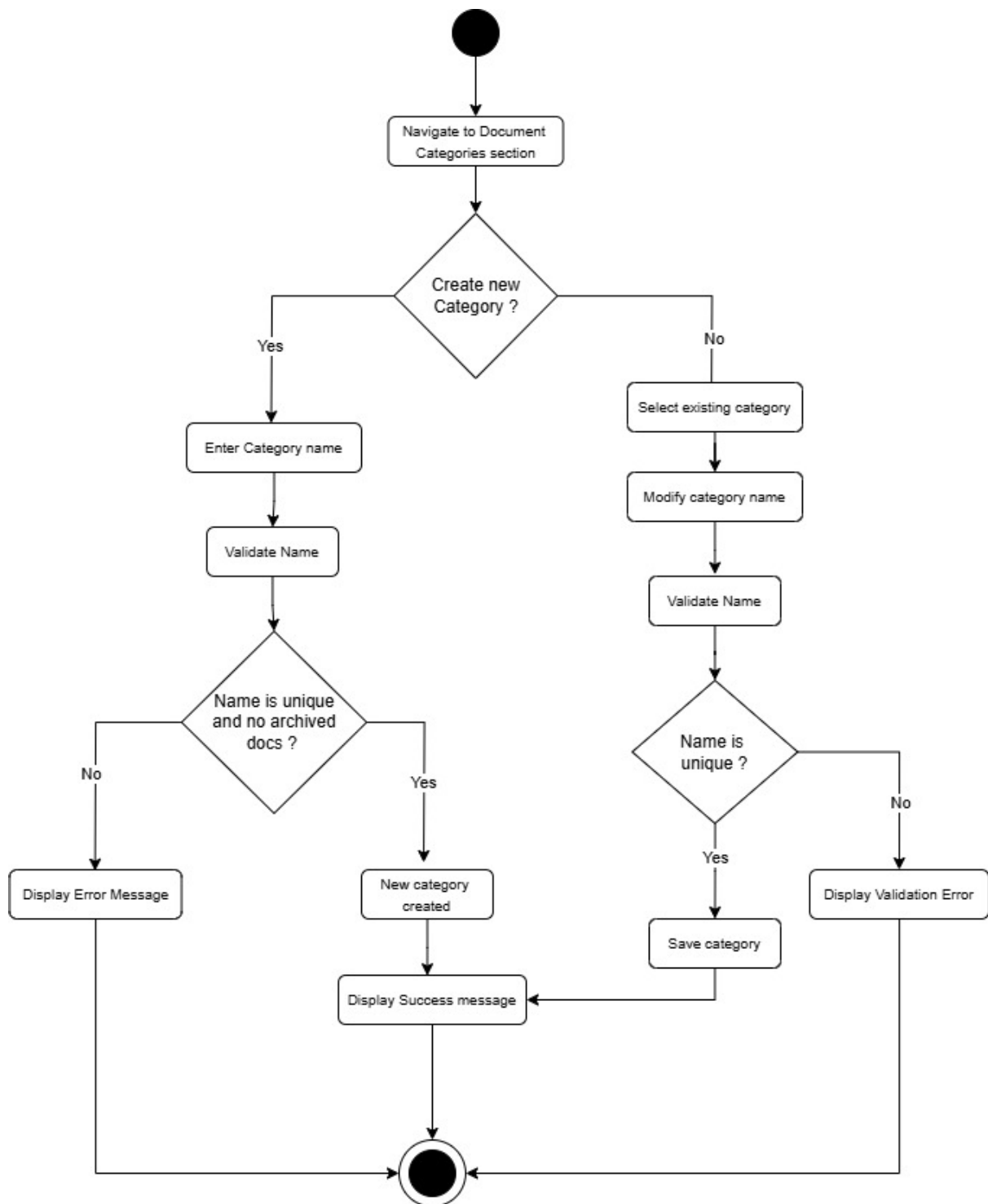


Figure 3.21: Manage document categories activity diagram

Figure 3.21 This activity diagram of managing the municipality departments, if the administrative affairs wants to add a new category to the system or edit the name of the category.

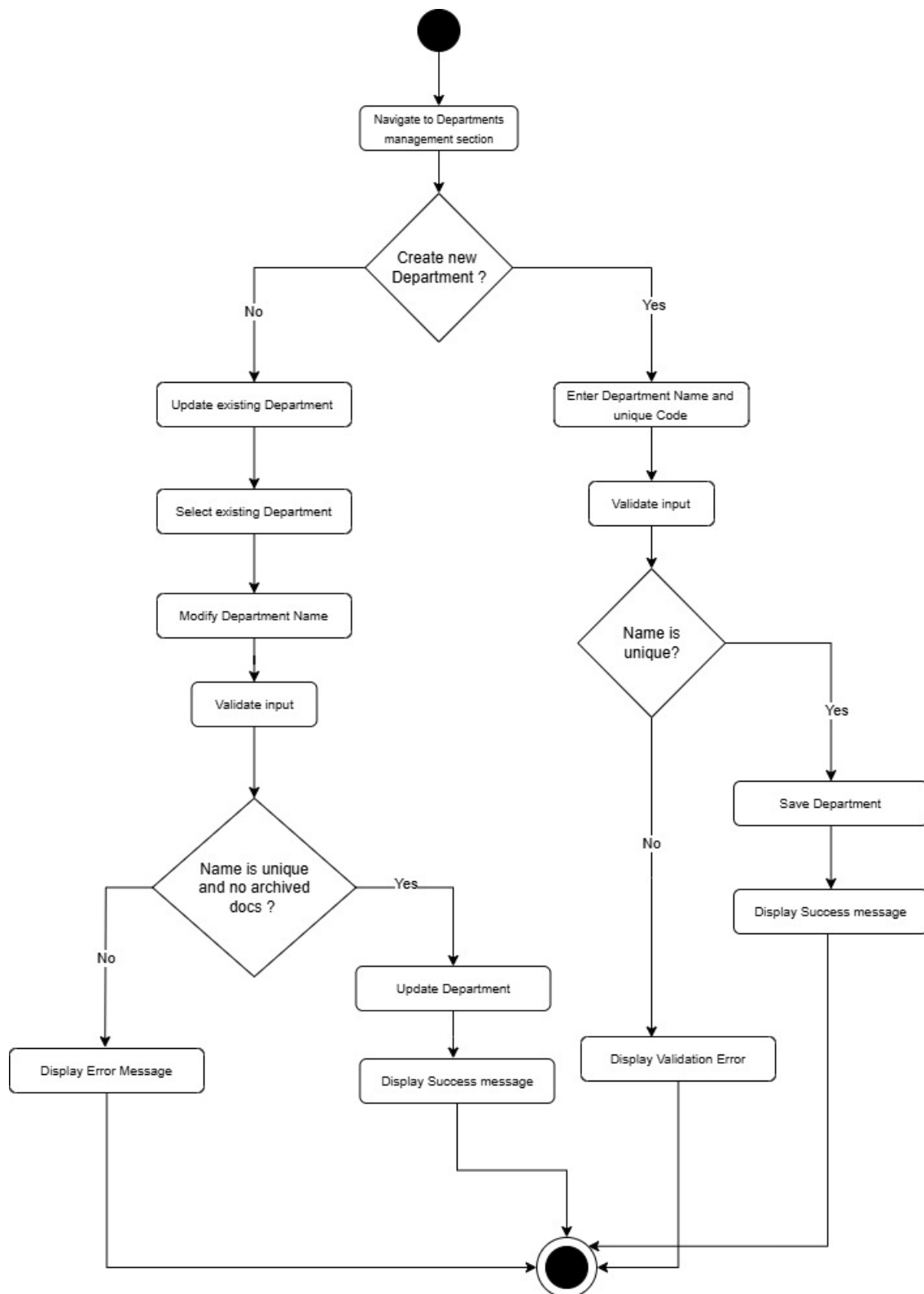


Figure 3.22: Manage municipal departments activity diagram

Figure 3.22 This activity diagram of managing the municipality departments, if the administrative affairs wants to add a new departments to the system.

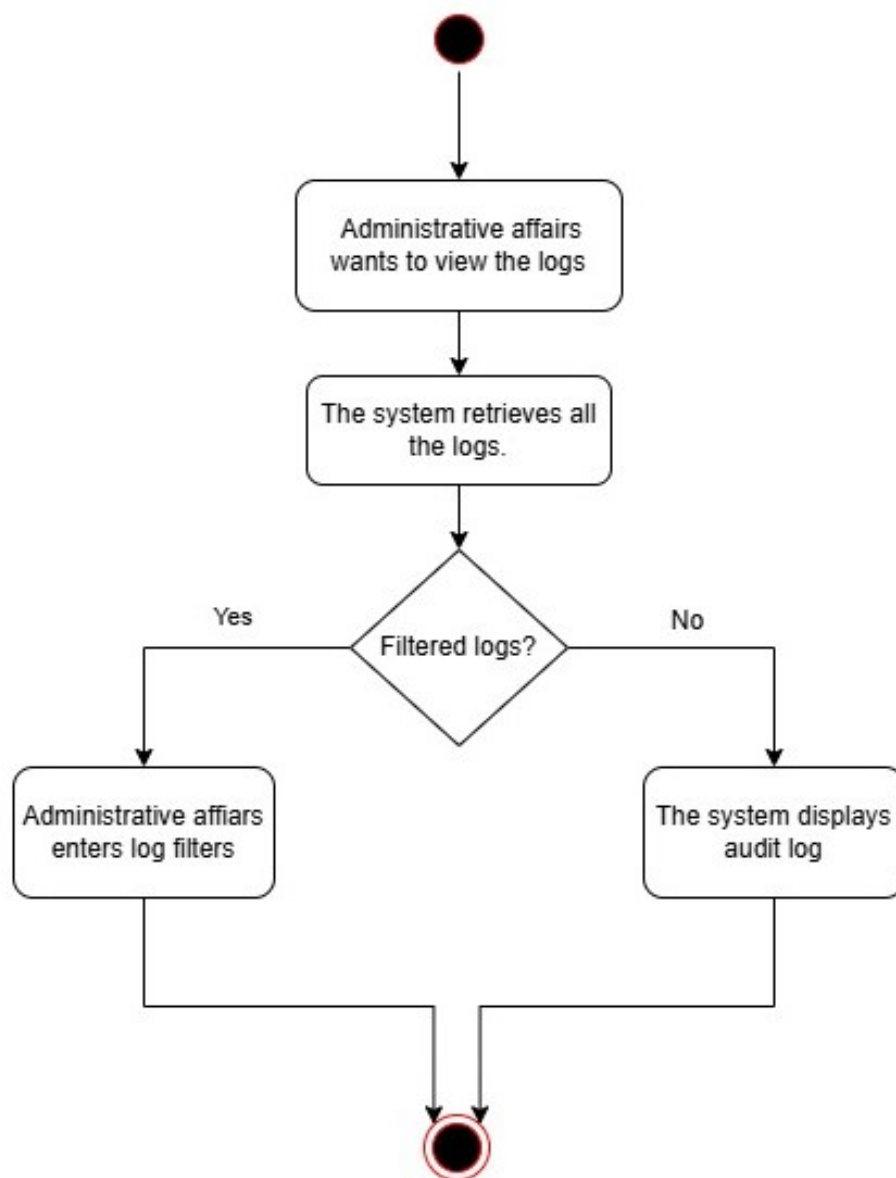


Figure 3.23: Audit log activity diagram

Figure 3.23 This audit log activity diagram shows how the logs are stored and displayed for the administrative affairs.

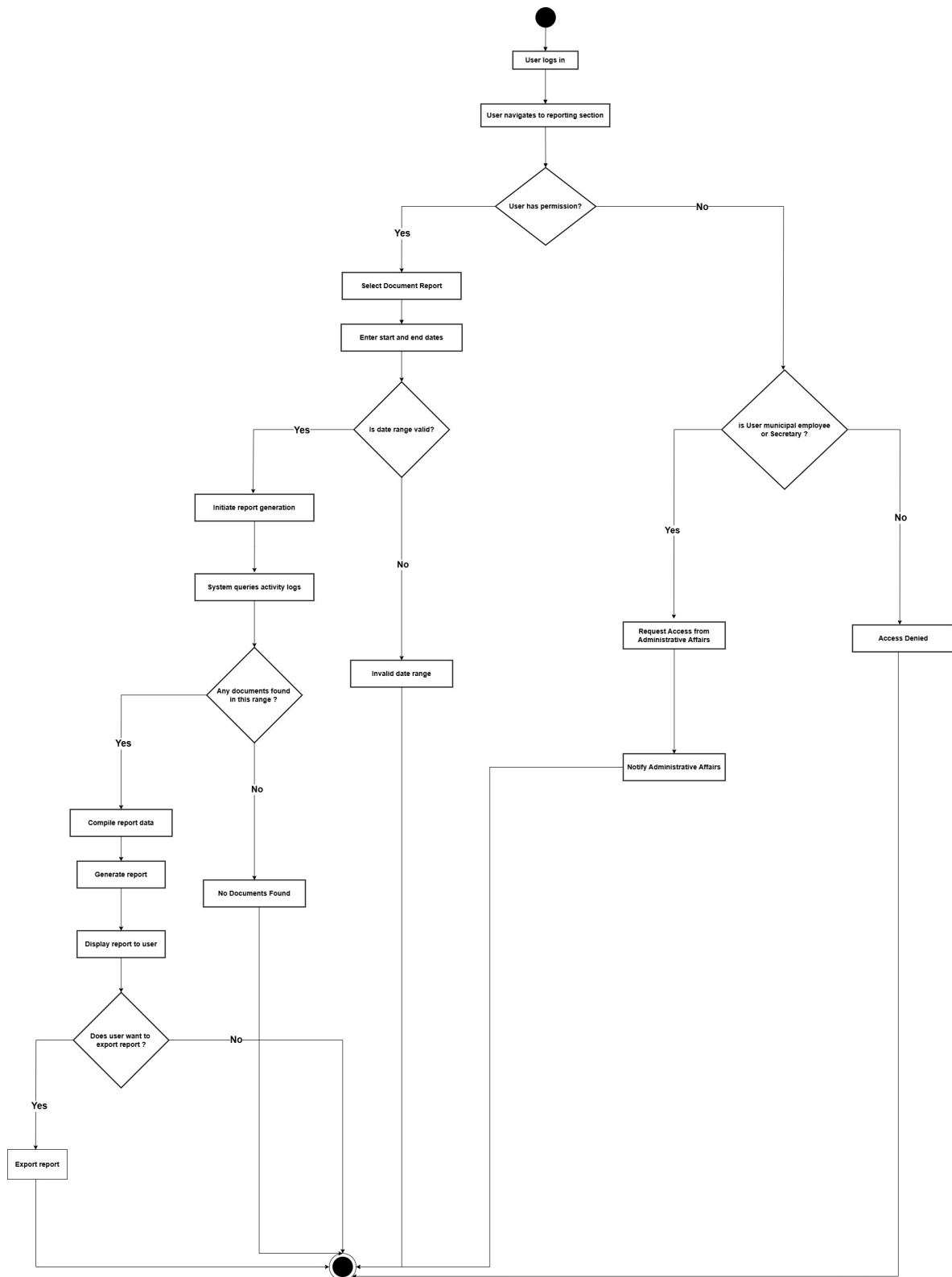


Figure 3.24: Generate report activity diagram

Figure 3.24 This activity diagram shows the steps of generating a report.

3.3.4 State Chart Diagram

Shows the different states an object can be in and the transitions between those states.

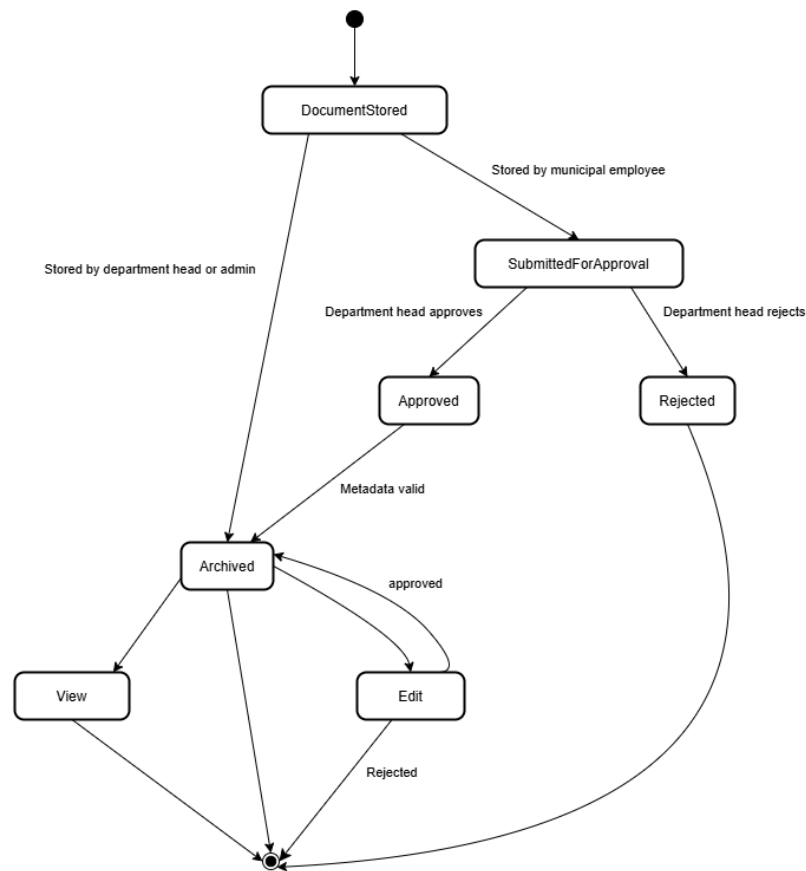


Figure 3.25: Document state chart diagram

Figure 3.25 This state chart of the document presents the state that a new document go through, first if the document archived by a regular municipal employee the document needs approval by the department head and the administrative affairs. If approved, then archived, he document might also be edited or viewed so the document uses the edit/view use case.

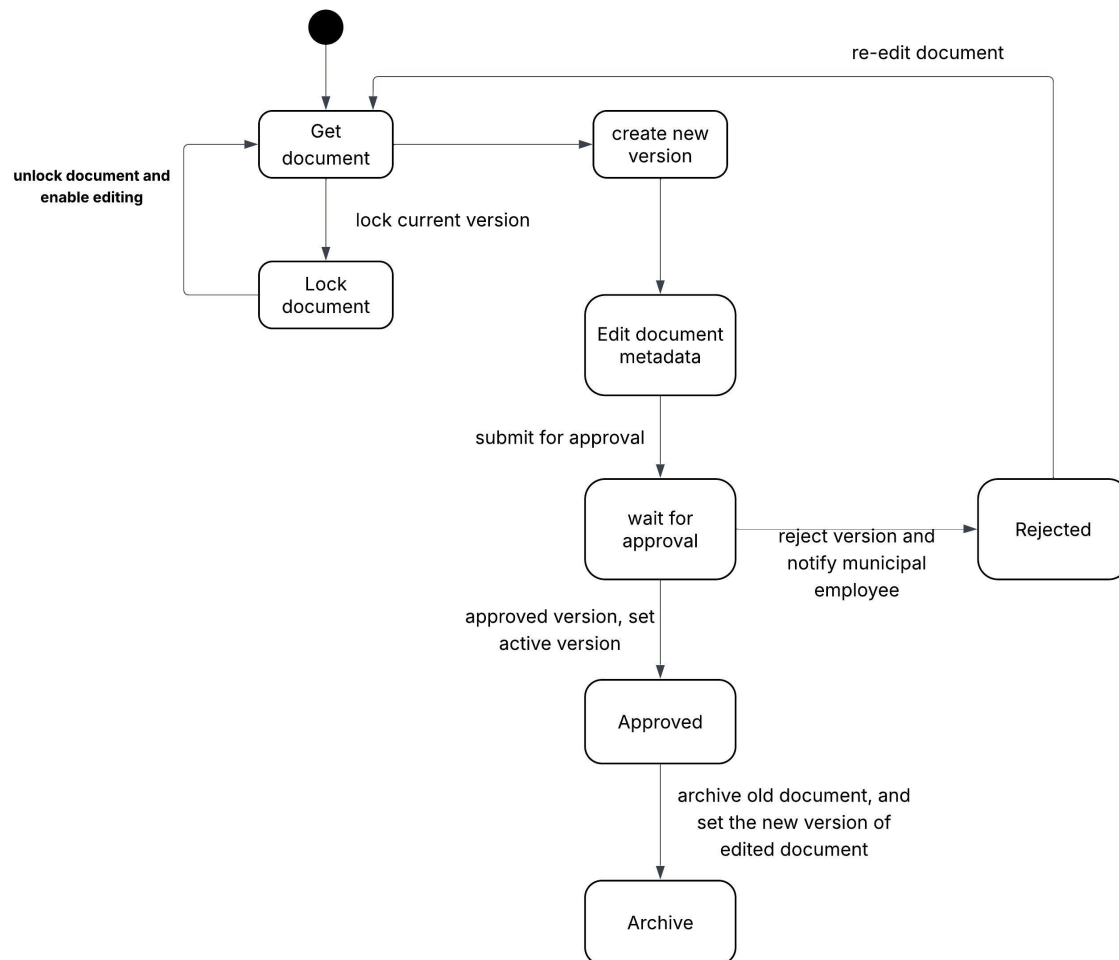


Figure 3.26: Version control state chart diagram

Figure 3.26 The version control state chart diagram explains the steps when a document is edited and approved of changes then the document has a new version.

3.4 System Architecture

3.4.1 Sub-System

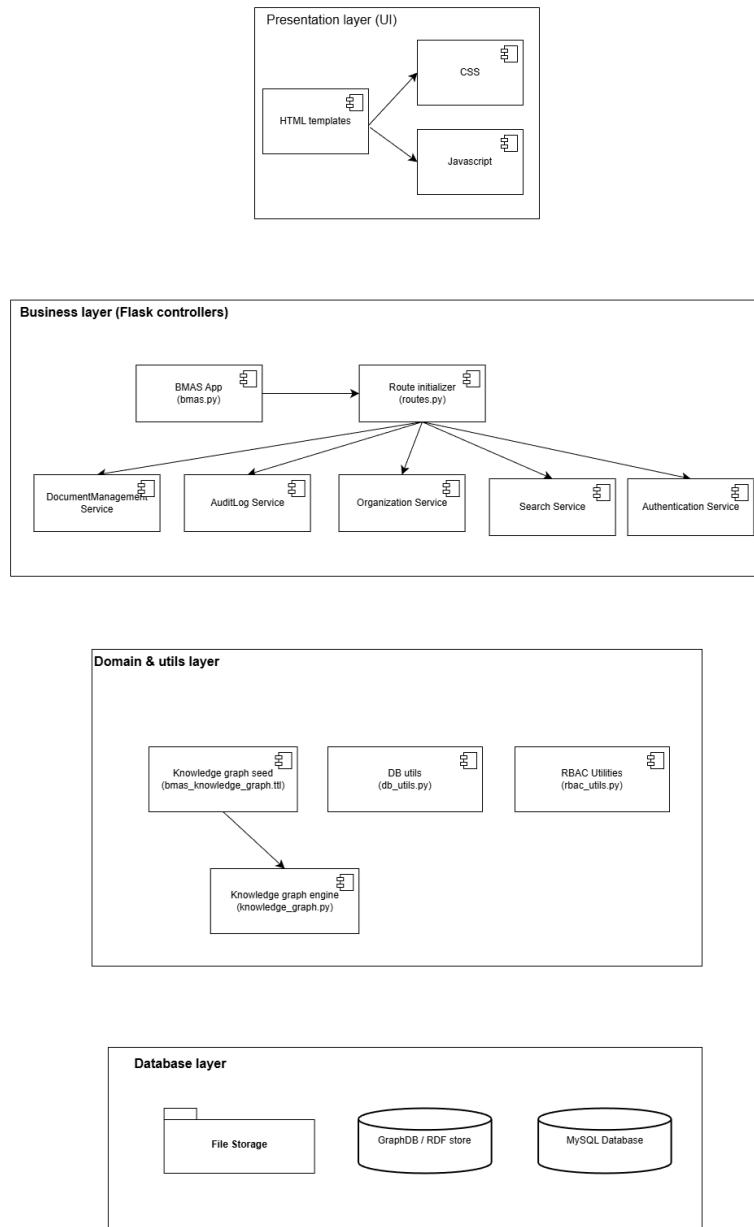


Figure 3.27: Sub System

Figure 3.27 This diagram present to us a high-level layered view of the system with separated layers, the User-Interface layer, business layer and the database layer, This architecture highlights the responsibilities and separation of concerns.

3.4.2 Software Architecture

in this software architecture we will show the set of structures required to analyze a software system and the developing of such structure and systems:

1. **User interface layer:** represents the user interfaces components of the system, such as search UI, edit UI, etc.
2. **Middle layer:** represents the business logic components, controllers and services.
3. **Data layer:** represents the databases interactions.

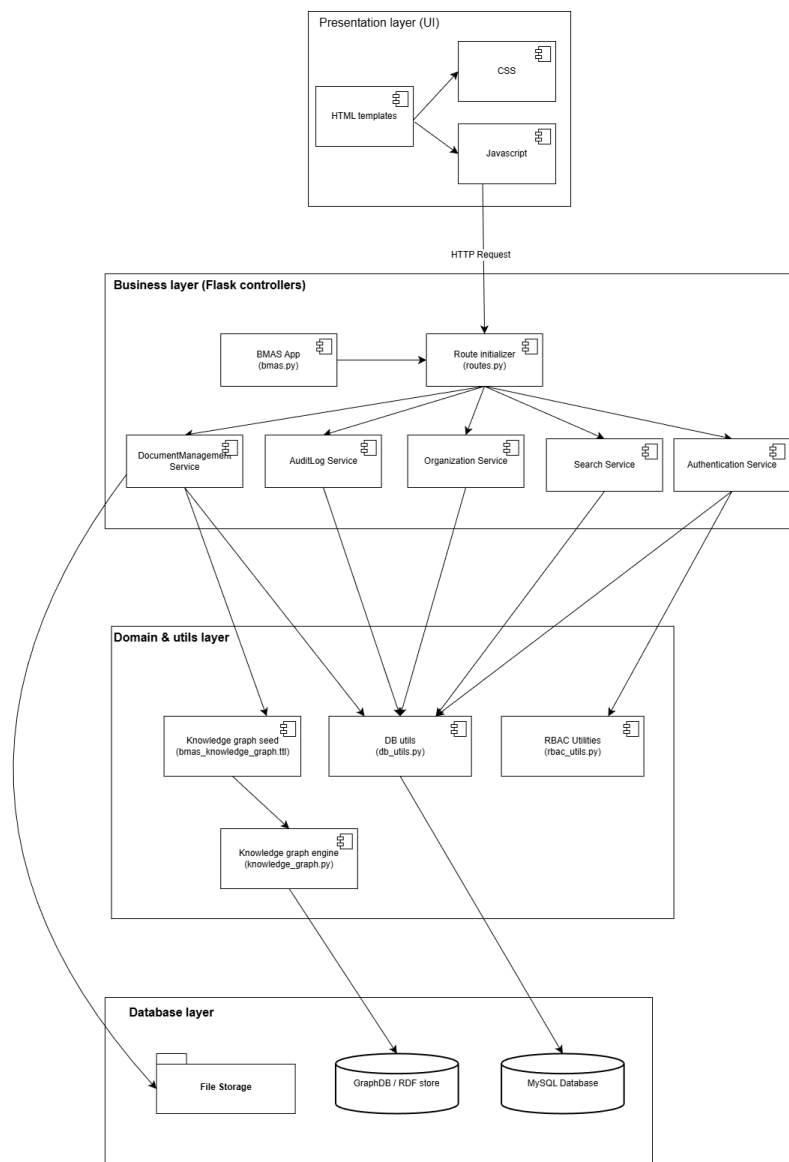


Figure 3.28: Software architecture

figure 3.28 This diagram is more detailed showing the interactions between the UI components, business services, and databases, and how each UI module communicates with specific services and how data flows to MySQL and the graphDB databases.

3.4.3 Deployment diagram

Enables you to demonstrate how software system as well as container instances from the static model are installed on the infrastructure in a specific deployment environment.

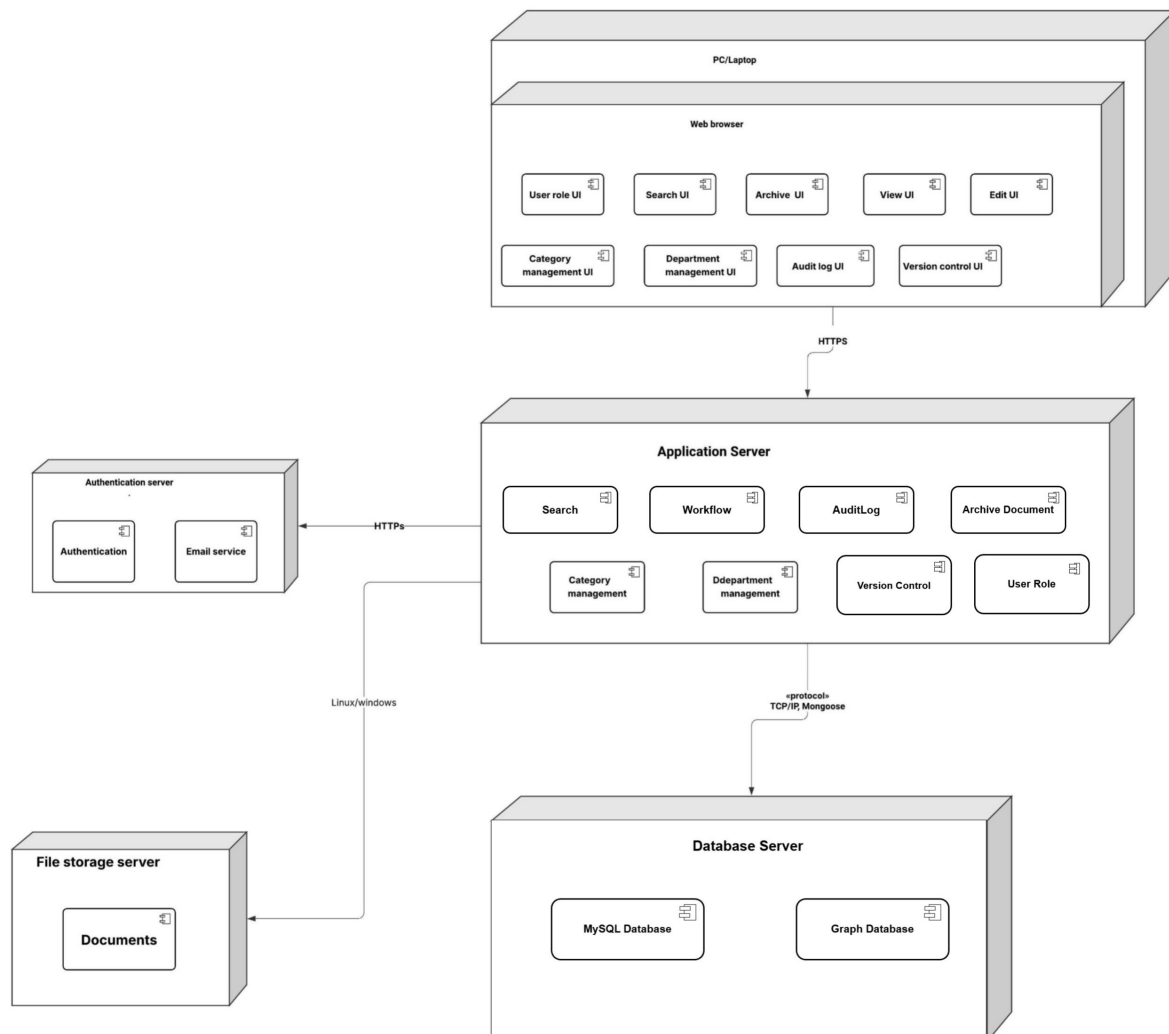


Figure 3.29: Deployment Diagram

figure 3.29 This deployment diagram displays the physical layer of the system, showing how the client communicates securely with the application server, authentication server over HTTPS, and how the application layer communicates with the database server over TCP/IP.

3.5 Data Management and ERD

Models the logical structure of a database, showing entities, their attributes, and relationships.

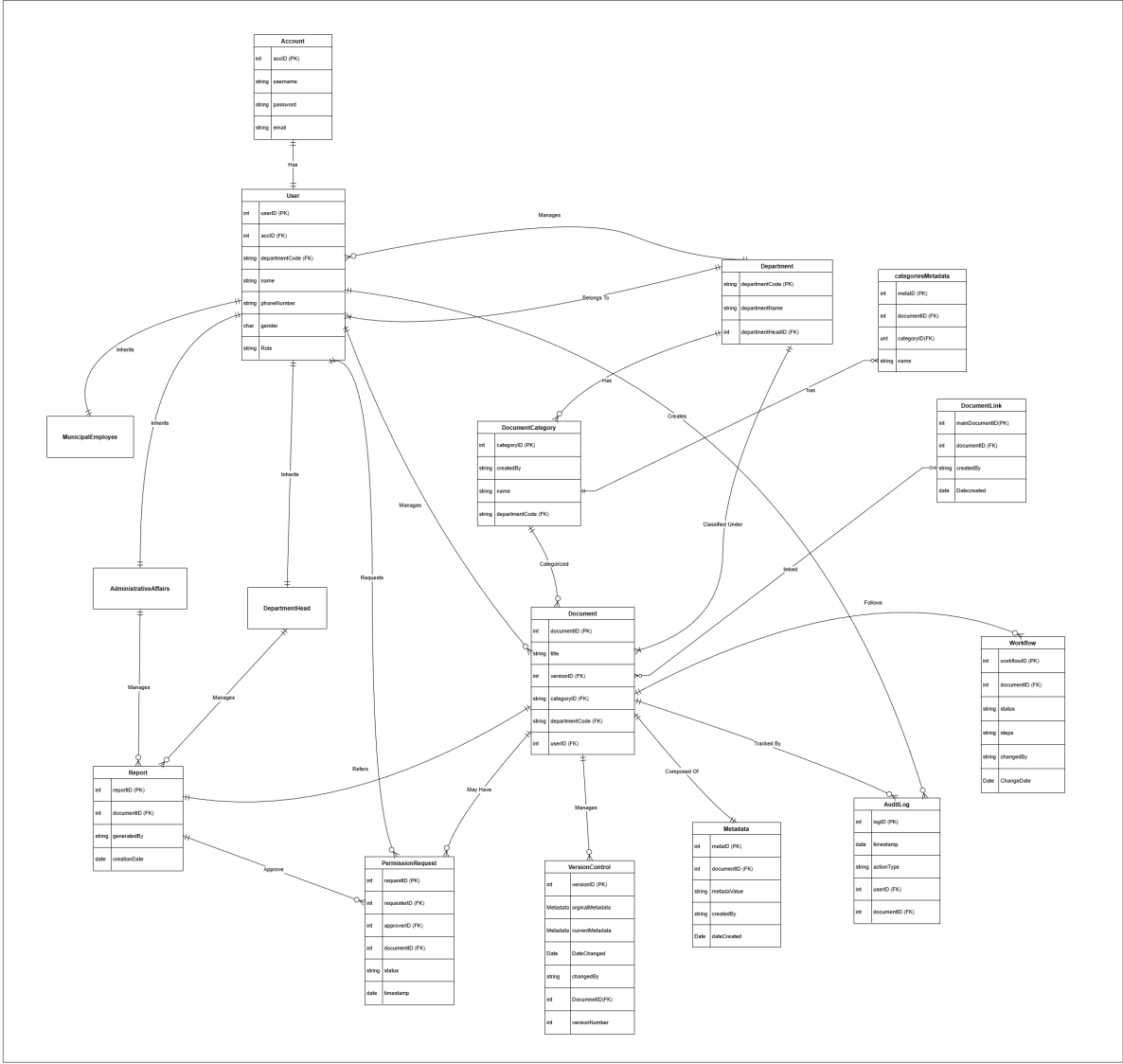


Figure 3.30: ERD

figure 3.30 This diagram shows how Bmas database was structured.

Summary

This chapter presented the complete system design for the Birzeit Municipality Archiving System. We started by defining the system objectives, main features, and operating requirements. We identified five actors (Mayor, Administrative Affairs, Department Head, Municipal Employee, and Secretary) and described thirteen use cases that cover all system functions. We then presented detailed UML diagrams that include class diagrams, sequence diagrams, activity diagrams, and state chart diagrams that show how the system components interact and work together. We also explained the system architecture with its three layers (user interface, middle layer, and data layer), the deployment structure, and the database design through the ERD. This complete design provides a clear blueprint for implementing the system.

Chapter 4

Testing

In this testing phase we present the development of Birzeit municipality archiving system, ensuring that the system works correctly and meets the project requirements. the focus was on checking if the users are able to interact with the system smoothly, and the functionalities work as expected.

The birzeit municipality archiving system includes main features such as semantic search, role based access control, metadata entries, document management and more that have been tested according to several test scenarios, this process helped identify issues early and improve the system reliability.

4.1 Testing methodology

To ensure the Birzeit Municipality Archiving System works correctly and reliably, we used a well-organized testing approach that combines different testing methods. Our strategy included both automated and manual testing techniques to verify that all system features function properly and meet the project requirements. We applied black-box and white-box testing methods to examine the system from multiple perspectives and ensure quality, dependability, and user-friendliness.

4.1.1 Black-Box Testing

Black-box testing helped us evaluate the system's behavior from an external viewpoint, without examining the internal code structure. This method allowed us to identify problems from the user's experience and verify that the system performs according to its specifications. We focused on testing user interactions, validating role-based access restrictions, examining semantic search with different queries, and ensuring approval workflows operate correctly.

4.1.2 White-Box Testing

White-box testing focused on examining the internal code components to confirm their correctness. We conducted unit tests and analyzed the program logic to verify that each function produces the expected results. This included testing the semantic search algorithm, metadata indexing procedures, version control functionality, and database operations.

4.2 Integration testing

Integration testing for Birzeit Municipality Archiving System verified that modules and components work together smoothly. It ensured frontend, backend APIs, databases, and external services interact correctly, confirming the system functions as intended when integrated.

4.2.1 Document Management Section

Archive Document – Positive Flow

TEST CASE TITLE	TEST DATE
Archive Document – Positive Flow	1/21/2026

TEST SCENARIO	TEST DESIGNED BY
Archive document with metadata	Issam Abdelkareem

TEST DESCRIPTION	PRE-CONDITIONS
Verify that users can successfully archive documents with complete metadata, and the system stores them in MySQL, file system, and GraphDB	• User is logged in • User has archive permissions • Document categories and departments exist • PDF file is available

Step	Step Description	Expected Result	Actual Result	Status	Notes
1	User navigates to the archive page from the dashboard	Archive page loads successfully with the document upload form	Yes	Pass	–
2	User selects document category and department	Category and department are displayed correctly in dropdowns	Yes	Pass	–
3	User enters document title and fills in all required metadata fields	All fields are populated successfully	Yes	Pass	–
4	User selects a PDF file and submits the document	System validates inputs and processes the request without errors	Yes	Pass	–

Step	Action Description	Expected Result	Actual Result	Status	Notes
5	System stores document data	Metadata saved in MySQL, PDF stored in file system, and RDF triples created in GraphDB	Yes	Pass	–
6	Success message displayed and document listed	Confirmation message appears and document shows with correct metadata	Yes	Pass	–

Table 4.1: TC-01: Archive Document Test – Positive Flow

Archive Document – Negative Flow (Input Validation)

TEST CASE TITLE	TEST DATE
Archive Document – Negative Flow	1/21/2026

TEST SCENARIO	TEST DESIGNED BY
Archive document validation and error handling	Issam Abdelkareem

TEST DESCRIPTION	PRE-CONDITIONS
Verify that the system properly validates inputs, handles file errors, and permission restrictions during document archiving	• User is logged in • Archive page is accessible

Step	Step Description	Expected Result	Actual Result	Status	Notes
1	User attempts to submit document without selecting category or department	System displays error: "Please select a document category and department"	Yes	Pass	–
2	User leaves document title and required metadata fields empty	System displays error listing all missing required fields	Yes	Pass	–
3	User enters special characters in title field	System sanitizes input or displays validation error	Yes	Pass	–
4	User attempts to upload non-PDF file (e.g., .docx, .jpg)	System displays error: "Only PDF files are allowed"	Yes	Pass	–
5	User attempts to upload file exceeding size limit (>16MB)	System displays error: "File size exceeds maximum limit"	Yes	Pass	–
6	User uploads corrupted or invalid PDF file	System displays error: "Invalid or corrupted file"	Yes	Pass	–
7	User with insufficient permissions attempts to archive	System displays error: "You do not have permission to archive documents"	Yes	Pass	–
8	Regular employee submits document requiring approval	Document marked as "Pending Approval" and department head notified	Yes	Pass	–

Table 4.2: TC-02: Archive Document Test – Negative
Flow (Input Validation)

Archive Document – Negative Flow

TEST CASE TITLE	TEST DATE
Archive Document – Negative Flow	1/21/2026

TEST SCENARIO	TEST DESIGNED BY
Archive document system failure handling	Issam Abdelkareem

TEST DESCRIPTION	PRE-CONDITIONS
Verify that the system handles database failures, network issues, and storage errors gracefully during document archiving	• User is logged in • User has archive permissions • Valid document and metadata ready for upload

Step	Step Description	Expected Result	Actual Result	Status	Notes
1	MySQL database connection fails during submission	System displays error: "Unable to save document. Please try again"	Yes	Pass	–
2	File system storage fails during file upload	System displays error and rolls back database transaction	Yes	Pass	Data consistency maintained
3	GraphDB connection fails during RDF triple creation	System displays error: "GraphDB Connection Error"	Yes	Pass	Semantic search may be limited
4	Network timeout occurs during document submission	System displays timeout error and allows user to retry	Yes	Pass	–
5	Disk space full on file server	System displays error: "Storage space unavailable"	Yes	Pass	–

Table 4.3: TC-03: Archive Document Test – positive Flow

4.2.2 Search and Retrieval Section

Search Document – Positive Flow (Semantic Search)

TEST CASE TITLE	TEST DATE
Testing Semantic Search metadata and keywords– Positive Flow	1/24/2026

TEST SCENARIO	TEST DESIGNED BY
Metadata and keywords search functionality	Rama Shaheen

TEST DESCRIPTION	PRE-CONDITIONS
Verify that the system successfully performs semantic search using the knowledge graphDB and retrieves accurate and relevant documents based on the user entries.	• User is logged in • Search page is accessible. • Documents with semantic metadata exist within the system. • GraphDB service is running and connected.

Step	Step Description	Expected Result	Actual Result	Status	Notes
1	User navigates to search page	Search page loads with input field and filter options	Yes	Pass	–
2	User enters semantic query.	System processes query and queries MySQL and GraphDB	Yes	Pass	–
3	System retrieves matching documents	Results display with title, category, department, and date	Yes	Pass	–
4	User searches by metadata (title, department, category)	System returns all documents matching metadata criteria	Yes	Pass	–
5	User enters partial keyword (e.g., "البناء" for "رخصة البناء")	System returns documents containing partial match	Yes	Pass	Semantic search
6	User enters context-based query (e.g., "permits from engineering")	System understands context and returns relevant documents	Yes	Pass	GraphDB RDF triples

Step	Action Description	Expected Result	Actual Result	Status	Notes
7	User applies filters (department/category)	Results filtered correctly, multiple filters work simultaneously	Yes	Pass	–
8	User clicks on search result	Document details page opens with correct information	Yes	Pass	–
9	Search results accuracy and performance verified	All results match criteria and load within 3 seconds	Yes	Pass	–
10	System logs search activity	Search recorded in audit log with user ID and timestamp	Yes	Pass	–

Table 4.4: TC-04: Search Document Test – Positive Flow

Search Document – Negative Flow (Invalid query input)

TEST CASE TITLE	TEST DATE
Testing Search validation and Error Handling – Negative Flow	1/24/2026

TEST SCENARIO	TEST DESIGNED BY
Search validation and incorrect input handling	Rama Shaheen

TEST DESCRIPTION	PRE-CONDITIONS
Verify that the system properly handles invalid search queries, empty inputs, permission restrictions, connection failures, and edge cases	• User is logged in • Search page is accessible

Step	Step Description	Expected Result	Actual Result	Status	Notes
1	User submits empty search query	System displays: "Please enter a search term"	Yes	Fail	–
2	User searches for non-existent document	System displays: "No results found. Try different keywords"	Yes	Pass	Proper user feedback
3	User enters only whitespace or special characters	System displays: "Invalid search query"	Yes	Fail	–
4	User enters extremely long query (>500 characters)	System truncates or displays: "Search query too long"	Yes	Pass	–
5	user repeats invalid input multiple times.	System remains stable.	Yes	Pass	Stability testing

Table 4.5: TC-05: Search Document Test – Negative Flow (Invalid query input)

Search Document – Positive flow (filter-based search)

TEST CASE TITLE	TEST DATE
Testing Semantic search filters – positive flow	1/25/2026

TEST SCENARIO	TEST DESIGNED BY
Filtered search functionality	Rama Shaheen

TEST DESCRIPTION	PRE-CONDITIONS
Verify that the system allows users to filter search results using metadata attributes such as date, category, and department.	• User is logged in • Search page is accessible. • Document exist within the system.

Step	Step Description	Expected Result	Actual Result	Status	Notes
1	User navigates to search page	Search page loads with input field and filter options	Yes	Pass	–
2	user selects category filter: مخطط المساحة المعتمد	Filtered applied.	Yes	Pass	–
3	user selects date range from 1/1/2026 to 25/1/2026.	Date filter applied	Yes	Pass	–
4	User clicks search.	System returns all documents matching meta-data.	Yes	Pass	–
5	Relevant filtered documents displayed.	Correct results displayed.	Yes	Pass	–

Table 4.6: TC-06: Search Document Test – Positive flow (filter-based search)

4.2.3 Editing and version control

Edit Document – Positive Flow

TEST CASE TITLE	TEST DATE
Editing an existing document and saving a new version– Positive Flow	1/25/2026

TEST SCENARIO	TEST DESIGNED BY
Edit document	Malak Mustafa

TEST DESCRIPTION	PRE-CONDITIONS
Verify that the system allows users to edit an existing document and save the changes successfully	• User is logged in • Documents exist in the system • User has edit permissions • User is on the search page

Step	Step Description	Expected Result	Actual Result	Status	Notes
1	Search for an existing document	Document appears in search results	Yes	Pass	–
2	Click the edit icon next to the document	Edit mode is enabled	Yes	Pass	–
3	Modify document context	Change is accepted	Yes	Pass	–
4	Click ”حفظ التغييرات” button	Change is saved successfully	Yes	Pass	–
5	System creates a new version	New version is added to version history	Yes	Pass	Version number increased

Table 4.7: TC-07: Edit Document Test – Positive Flow

Unauthorized edit - Negative Flow

TEST CASE TITLE	TEST DATE
Editing document without permission – Negative Flow	1/25/2026

TEST SCENARIO	TEST DESIGNED BY
Edit document (unauthorized user)	Malak Mustafa

TEST DESCRIPTION	PRE-CONDITIONS
Verify that the system handles unauthorized users making modifications correctly	• User is logged in. • Document exists in the system. • User does NOT have permission to edit. • User is on the search page

Step	Step Description	Expected Result	Actual Result	Status	Notes
1	Search for an existing document.	Document appears in search result	Yes	Pass	–
2	Edit document content	Content modified	Yes	Pass	–
3	Save changes	New version is created	Yes	Pass	–
4	Try to click edit icon.	User can't click the icon	Yes	Pass	Edit icon disabled

Table 4.8: TC-08: Edit Document Test – Negative Flow

Version history creation after edit - Positive Flow

TEST CASE TITLE	TEST DATE
Verify version history after editing a document – Positive Flow	1/25/2026

TEST SCENARIO	TEST DESIGNED BY
Version control	Malak Mustafa

TEST DESCRIPTION	PRE-CONDITIONS
Checking that a new version is created after editing.	• User is logged in. • Document exists with at least one previous version. • User has permission to edit. • User is on the search page

Step	Step Description	Expected Result	Actual Result	Status	Notes
1	Search for an existing document.	Document appears in search result	Yes	Pass	–
2	Edit document content	Content modified	Yes	Pass	–
3	Save changes	New version is created	Yes	Pass	–
4	Click version icon	All versions are displayed in the versions page	Yes	Pass	Latest version on top

Table 4.9: TC-09: Edit Document Test – Positive Flow

4.3 Testing results and analysis

Throughout the testing phase of the Birzeit Municipality Archiving System, we carefully evaluated how each feature performed in both normal and challenging situations. We were satisfied to see that core features such as document archiving, semantic search, metadata management, and role-based access control worked reliably and consistently. Manual testing helped us understand the platform from the perspective of actual municipal employees. Actions such as archiving documents, searching by department or category, editing metadata, and requesting access to documents from other departments all worked smoothly across different browsers. At the same time, testing was not only about confirming what worked correctly, but also about discovering potential issues. We found several areas that needed improvement, such as error messages that were unclear or validation that needed to be stricter. Negative test cases, like attempting to archive without selecting a category, uploading files larger than 10MB, or trying to edit documents without proper permissions, helped us confirm that the system actively prevents incorrect data and unauthorized actions. These issues were addressed and fixed during the development process. Integration testing between MySQL, GraphDB, and the file system showed that all components work together seamlessly to store documents, metadata, and semantic relationships. Overall, the testing phase gave us real confidence that the Birzeit Municipality Archiving System is not only functional but also secure, reliable, and ready to improve document management at the municipality.

Chapter 5

Implementation and UI

This chapter outlines the practical implementation of the Birzeit municipality archiving system (BMAS), with an explanation of how the system architecture, and functional requirements outlined in the previous chapter were transformed into a fully operational web-based platform, focusing on the implementation strategy, and back-end design, while another section focuses on the user interface.

5.1 Implementation plan

- **Sprint 1:** Collect project requirements, set up development environment, and design basic wireframes.

During this sprint, we met with Birzeit Municipality staff to understand their document management needs and challenges. We set up our development tools including VS Code, GitHub for version control, and XAMPP for local database testing. We also created initial wireframes using Figma to visualize the user interface for different roles like administrative affairs, department heads, and municipal employees.

→ 2/10/2025 - 15/10/2025

- **Sprint 2:** Create login and registration pages for the system.

We built the login interface using HTML, CSS, Bootstrap5, and JavaScript to provide a clean and accessible entry point to the system. The registration form was designed exclusively for administrative affairs to create new employee accounts. We implemented basic client-side validation to check for proper email format and password strength before submission.

→ 16/10/2025 - 29/10/2025

- **Sprint 3:** Build database structure and begin backend development.

This sprint focused on creating the MySQL database schema with tables for users, documents, departments, categories, and metadata. We also set up GraphDB to prepare for semantic search functionality. The backend was initialized using Python and Flask framework, with basic route handlers and database connection utilities established.

→ 30/10/2025 - 12/11/2025

- **Sprint 4:** Build user authentication system and link it to the database.

We implemented secure login functionality with password hashing to protect user credentials. The authentication system validates user credentials against the MySQL database and creates sessions for logged-in users. We also connected user roles and permissions to the database, laying the groundwork for RBAC implementation in later sprints.

→ 13/11/2025 - 26/11/2025

- **Sprint 5:** Add document upload feature with RBAC and approval system.

During this sprint, we built the document archiving functionality that allows users to upload PDF files with metadata. We implemented RBAC to control who can archive documents directly and who needs approval. The approval workflow was created so that municipal employees send documents for department head review, while department heads and administrative affairs can archive documents immediately. File validation ensures only PDF files under 16MB are accepted.

→ 27/11/2025 - 10/12/2025

- **Sprint 6:** Build semantic search using GraphDB.

This sprint was dedicated to implementing the semantic search feature, one of the core functionalities of the system. We integrated GraphDB to store document metadata as RDF triples, creating a knowledge graph that represents relationships between documents and their attributes. The search algorithm combines keyword matching from MySQL with semantic understanding from GraphDB, allowing users to find documents based on meaning and context rather than exact keywords.

→ 11/12/2025 - 24/12/2025

- **Sprint 7:** Add document viewing, editing, version tracking, and report creation.

We implemented the document viewer with a split-screen layout showing the PDF on one side and metadata on the other. The editing feature allows authorized users to modify document metadata, with version control tracking all changes and creating new versions when edits are approved. We also built the report generation module that lets administrative affairs and department heads create statistical reports on document activities within specified time periods.

→ 8/1/2026 - 15/1/2026

- **Sprint 8:** Run final tests, improve user interface, and prepare system for use.

In the final sprint, we conducted comprehensive testing including black-box and white-box testing to verify all features work correctly. We tested various scenarios like invalid inputs, permission restrictions, and system failures to ensure reliability. Based on test results, we refined the user interface for better usability and fixed any bugs discovered during testing. We prepared documentation and training materials for municipality staff.

→ 16/1/2026 - 21/1/2026

5.1.1 System architecture implementation

The Birzeit municipality archiving system was built using a 4-tier architecture, that consists of presentation, business logic, domain and utilities, and database layers. This implementation ensures modularity, scalability, and maintainability of the website and it follows the architectural design presented in chapter 3 (System analysis and design), to make sure that the system is consistent in the planning and development phases.

First, we have the **presentation layer**: This layer was developed using **HTML, CSS, Bootstrap5, and JavaScript** providing a responsive and user-friendly interface, managing user interactions, from submissions, client-side input validation, and dynamic data visualization. prioritizing the usability, accessibility, and clarity to support users with varying levels of technical expertise.

Then, we have **the business logic layer**: The environment where python and flask frameworks handles the core system operations including the main application controller and route initializer, and other services components such as document management, authentication services, search services. Overall, these services encapsulates system logic, and operations between the presentation layer and the lower layers.

The Domain and utilities layer: This layer represents the shared services and the reusable components that support the core backend functionalities. These utilities enable secure access control, efficient databases operations, and semantic indexing while maintaining separation from business logic.

And last thing **The database layer**: where the system's persistent storage infrastructure, this layer includes **MySQL, GraphDB and the local file storage**, ensuring reliability, semantic data management and long-term document preservation.

5.1.2 Security and Access control implementation

To give the system the security needed to protect the documents from damage and loss, and manage the access, The system applies the following:

- Role-based access control: this was implemented to restrict the access to the system functionalities.
- Password hashing: the user credentials are securely stored.
- User role permission request: the operations on the document such as archiving, editing, and search are restricted depending on the role and its permission to do any operation on the document.

These measures ensure confidentiality, integrity, and compliance with municipal data protection policies.

5.1.3 Semantic search implementation

In our system, the semantic search was one of the core features for document retrieval, a hybrid semantic search was implemented using both **MySQL-based filtering and RDF-based semantic search modeling**. During the archiving of a document, meta-data attributes and contextual relationships are extracted and stored as Resource description framework (RDF) triples in the frm (subject, predicate,object) managed within GraphDB, forming a semantic knowledge graph that represents conceptual connections among archived documents.

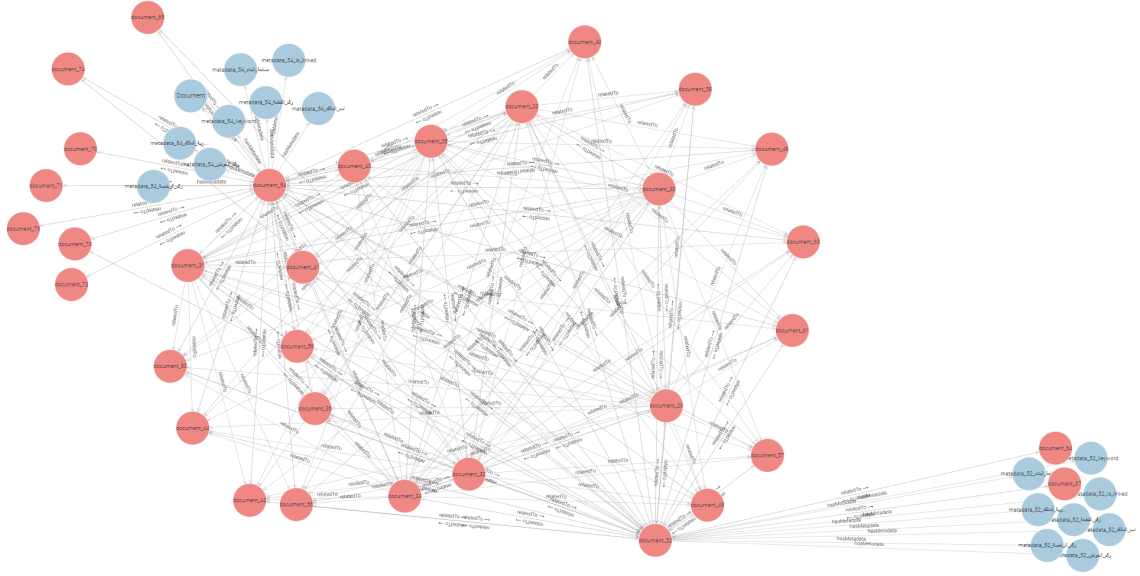


Figure 5.1: RDF-based semantic knowledge graph

Figure 5.1 illustrates the semantic knowledge graph generated for the archived documents in the birzeit municipality archiving system. Each red node in this graph represents a document, while each blue node represents the metadata entities that are associated with the document, the edges represent the relationships extracted from the shared metadata attributes using the RDF triples and SPARQL queries.

When two, three or more documents share identical or semantically related metadata values, a relationship **ns1:relatedTo** is automatically created between them, forming a dense semantic network that captures conceptual similarities and contextual connections across archived documents and establishing the graph we see.

5.2 Implementation of backend

5.2.1 System File Structure

JavaScript Files

The implementation consists of modular JavaScript files organized by functionality as shown in Figure 5.2. Each file handles a specific feature of the system, including admin metadata management, security controls, document archiving, user authentication, and various administrative functions. The files follow a consistent naming convention (`feature_role.js`) to clearly identify their purpose and the user role they serve. Bootstrap bundle files provide the UI framework for responsive design across all pages.

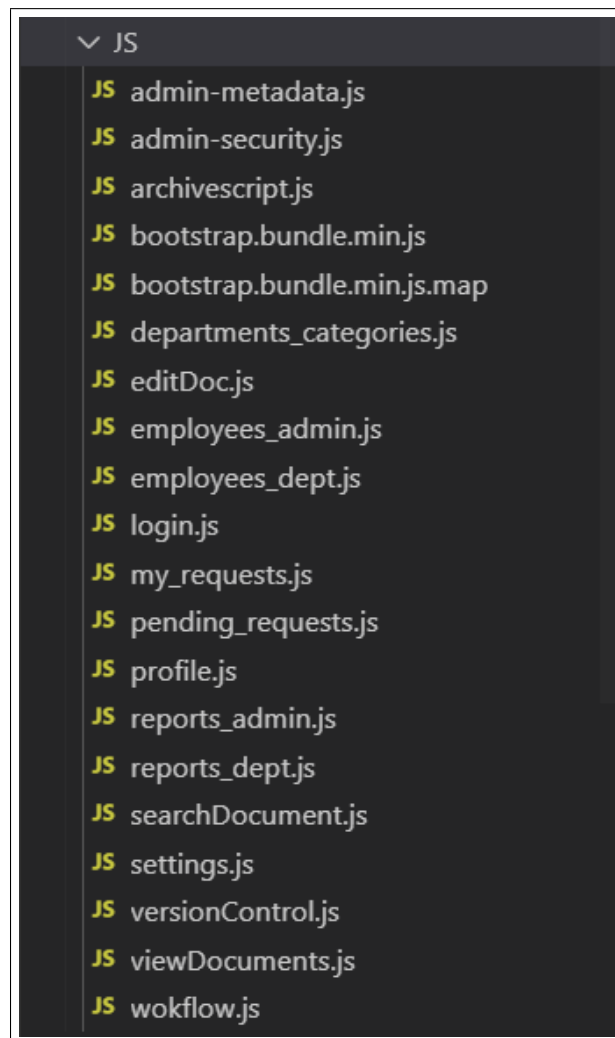


Figure 5.2: JavaScript Files Structure

Python Routes

The backend is organized into separate Python route modules, each managing distinct system functionality as shown in Figure 5.3. Core modules include authentication (`auth_routes.py`), document management (`document_routes.py`), search operations (`search_routes.py`), and workflow management (`workflow_routes.py`). The `db_utils.py` file provides cen-

tralized database connection handling, while `knowledge_graph.py` manages the semantic search integration with GraphDB. The `bmás.py` file serves as the main application entry point, and `routes.py` coordinates all route modules. This modular structure ensures clean separation of concerns and makes the codebase easy to maintain and extend.

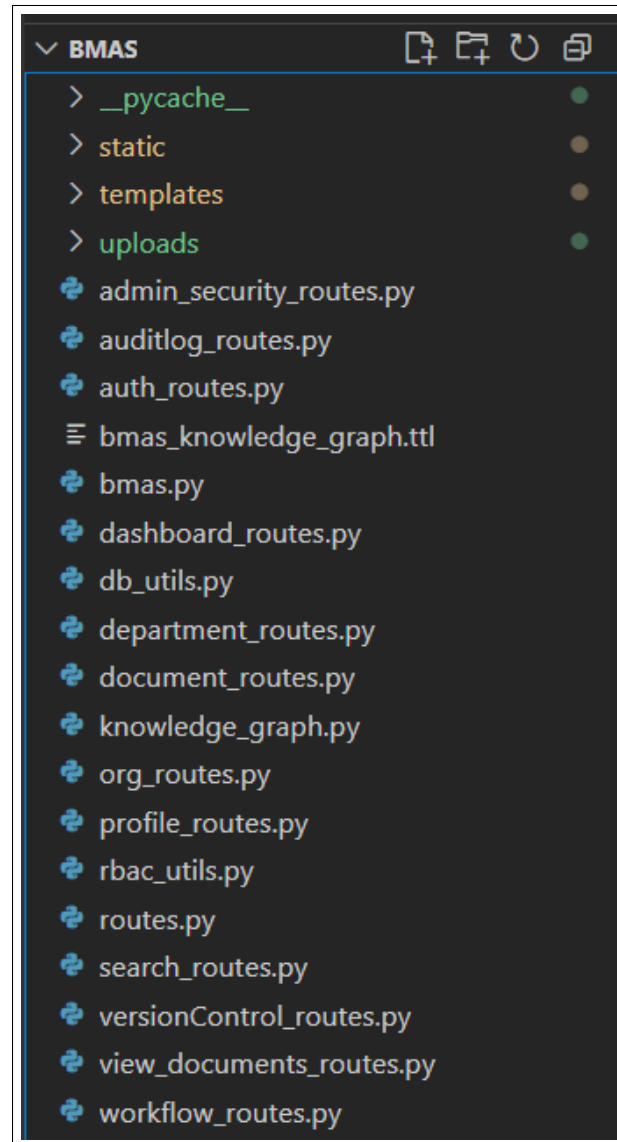


Figure 5.3: Python Backend Files Structure

5.2.2 Back-end architecture

The implementation of the Birzeit municipality archiving system back-end was by using **python and flask** web framework, and by following a modular, service-oriented architecture, each major functionality is managed by a dedicated route module.

The following snippet shows the main backend entry point where the Flask application is initialized and all routes are registered through a single initializer function.

```
from flask import Flask
from routes import init_routes
```

```

app = Flask(__name__)
app.secret_key = "super_secret_key"

# Register routes and database connection
init_routes(app)

if __name__ == '__main__':
    app.run(debug=True)

```

Then, using the following snippet to demonstrate the backend modularization by grouping different functionalities into separate route modules.

```

# routes.py
from auth_routes import init_auth_routes
from dashboard_routes import init_dashboard_routes
from document_routes import init_document_routes
from search_routes import init_search_routes
from auditlog_routes import init_auditlog_routes
from department_routes import init_department_routes
from workflow_routes import init_workflow_routes
from view_documents_routes import init_view_documents_routes
from profile_routes import init_profile_routes
from org_routes import init_org_routes
from admin_security_routes import init_admin_security_routes

def init_routes(app):
    # Initialize all route modules
    init_auth_routes(app)
    init_dashboard_routes(app)
    init_document_routes(app)
    init_search_routes(app)
    init_auditlog_routes(app)
    init_department_routes(app)
    init_workflow_routes(app)
    init_view_documents_routes(app)
    init_profile_routes(app)
    init_org_routes(app)
    init_admin_security_routes(app)

```

5.2.3 Database implementation

The implementation of the relational database was by using **MySQL** to manage structured data, including user account, roles, permissions, documents, metadata, auditlogs, and system reports. This was implemented to centralize connection handling and reduce repetitive connection code across modules.

The following snippet shows how Birzeit municipality archiving system establishes database connectivity through a reusable helper function, supporting error handling to ensure integrity during failures.

```

import mysql.connector

def get_db_connection():
    try:
        db = mysql.connector.connect(
            host="localhost",
            user="root",
            password="",
            database="bmas"
        )
        return db
    except mysql.connector.Error as e:
        print(f"Database connection error: {e}")
        return None

```

5.2.4 File storage management

When archiving a document the only document type that is allowed and stored is **PDF** within a structured hierarchy in the local file system. To ensure the that the archived documents are in the supported formats, the documents are validated before storage. The following code snippet shows server-side validation for file uploads, ensuring only **PDF** documents can be archived.

```

ALLOWED_EXTENSIONS = {'pdf'}
MAX_FILE_SIZE = 16 * 1024 * 1024 # 16MB

def allowed_file(filename: str) -> bool:
    return '.' in filename and filename.rsplit('.', 1)[1].lower()
        in ALLOWED_EXTENSIONS

file = request.files.get('file')
if not file or file.filename == '':
    flash("No file selected.", "danger")
    return redirect(url_for('archive_page'))
if not allowed_file(file.filename):
    flash("File must be in PDF format only.", "danger")
    return redirect(url_for('archive_page'))

original_filename = secure_filename(file.filename)
unique_filename = f"doc_{datetime.now().strftime('%Y%m%d_%H%M%S')}_\
    {original_filename}"
upload_dir = os.path.join(app.root_path, 'uploads', 'documents')
os.makedirs(upload_dir, exist_ok=True)
full_file_path = os.path.join(upload_dir, unique_filename)
file_path = unique_filename
file.save(full_file_path)

```

5.2.5 Semantic layer implementation

GraphDB was integrated into the backend to manage the semantic relationships and enable context-aware document retrieval. During document archiving, the metadata and selected attributes are converted into RDF triples and sent to **GraphDB** using **SPARQL** updates.

```
def save_to_graphdb(doc_id, title, createdBy, doc_date, description,
                    department_code, category_id, keywords):
    endpoint = "http://localhost:7200/repositories/BMAS/statements"
    headers = {'Content-Type': 'application/sparql-update'}
    triples = f"""
PREFIX bmas: <http://birzeitmunicipality.ps/ontology#>
PREFIX xsd: <http://www.w3.org/2001/XMLSchema#>
INSERT DATA {{
    bmas:{doc_id} a bmas:Document ;
    bmas:title "{title}" ;
    bmas:createdBy "{createdBy}" ;
    bmas:date "{doc_date}"^^xsd:date ;
    bmas:description "{description}" ;
    bmas:department "{department_code}" ;
    bmas:category "{category_id}" .
}}
"""
    for kw in keywords:
        pred_name = str(kw.get("name", "")).replace(" ", "_")
                                                .replace("-", "_")
        val = str(kw.get("value", ""))
        triples += f'    bmas:{doc_id} bmas:{pred_name} "{val}" .\n'

    triples += "}\n"
    response = requests.post(endpoint, data=triples.encode('utf-8'),
                             headers=headers)
    if response.status_code in (200, 204):
        print("Data saved to GraphDB")
    else:
        print("Error saving data to GraphDB", response.status_code,
              response.text)
```

This code snippet illustrates how GraphDB was integrated by generating RDF triples and submitting them through a **SPARQL** update endpoint.

5.2.6 Archive document implementation

The archive document functionality enables users to upload PDF files along with their metadata to the system. When a document is submitted, the system first validates the file type and size to ensure only PDF files under 16MB are accepted. The uploaded file is then saved to the local file system with a unique filename that includes a timestamp to prevent naming conflicts.

```
# File validation
ALLOWED_EXTENSIONS = {'pdf'}
MAX_FILE_SIZE = 16 * 1024 * 1024 # 16MB

def allowed_file(filename: str) -> bool:
    return '.' in filename and filename.rsplit('.', 1)[1].lower()
        in ALLOWED_EXTENSIONS

# Secure file storage
original_filename = secure_filename(file.filename)
unique_filename = f"doc_{datetime.now().strftime('%Y%m%d_%H%M%S')}
    _{original_filename}"
upload_dir = os.path.join(app.root_path, 'uploads', 'documents')
os.makedirs(upload_dir, exist_ok=True)
full_file_path = os.path.join(upload_dir, unique_filename)
file.save(full_file_path)

# Insert document record
cursor.execute("""
    INSERT INTO document (title, documentType, categoryID,
                          departmentCode, userID, uploadDate,
                          filePath, status)
    VALUES (%s, %s, %s, %s, %s, NOW(), %s, %s)
""", (title, form_document_type, category_id, department_code,
      user_id, file_path, doc_status))

document_id = cursor.lastrowid
```

The system determines the document status based on the user's role. Administrative users can directly archive documents with an "Archived" status, while department heads can archive documents within their own departments. Municipal employees must submit their documents for approval, which creates a "Pending" permission request that routes to their department head for review.

5.3 User Interface

5.3.1 Login & Register

The login page serves as the entry point to the system, providing **secure authentication** for all municipal employees. Users enter their credentials to access their **role-specific dashboard** and functionalities. Figure 5.4 illustrates the login interface where users authenticate themselves. The registration interface is exclusively accessible to administrative affairs for creating new employee accounts by entering employee information including name, email, password, department, and gender. Figure 5.5 shows the registration form used for creating new employee accounts.

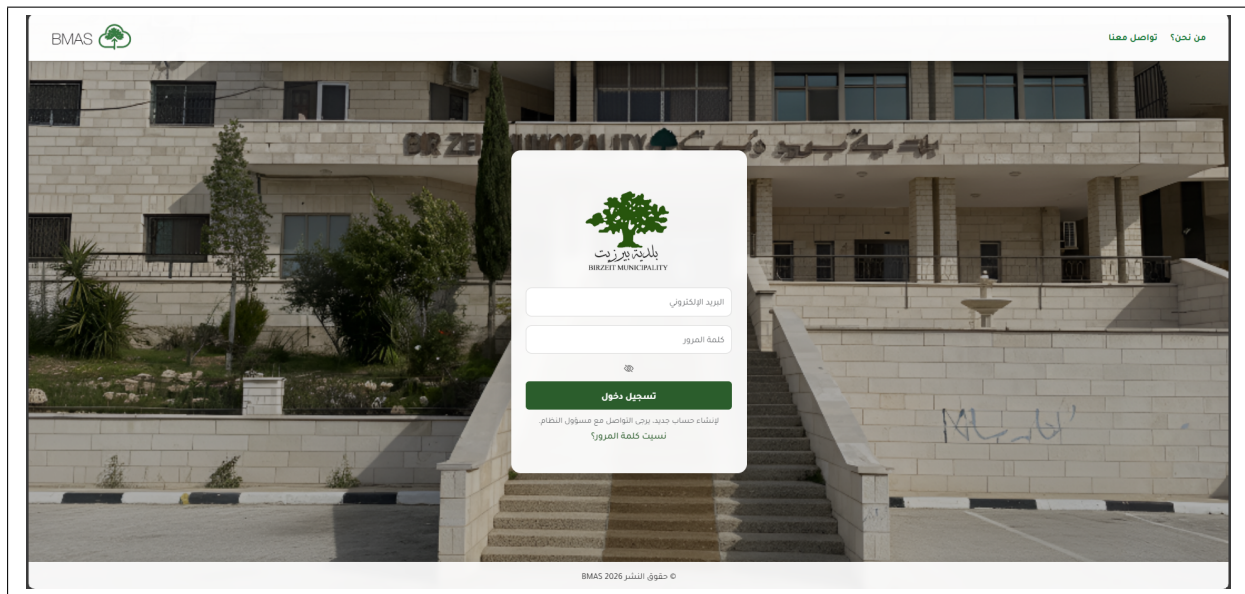


Figure 5.4: Login Page

Figure 5.5: Register Form

5.3.2 Home Pages

Upon successful login, users are directed to their personalized dashboard based on their assigned role within the municipality. Each dashboard displays relevant features and actions according to the user's permissions and responsibilities. Figure 5.6 illustrates the **Administrative Affairs** dashboard which provides comprehensive access to all system functionalities including user management, document approval, and audit logs. **Department Heads** can approve documents and generate reports, as shown in Figure 5.8. Figure 5.9 demonstrates how **Municipal Employees** can archive, search, and edit their own documents. The **Secretary** role allows viewing and searching documents with limited modification rights, as depicted in Figure 5.7.

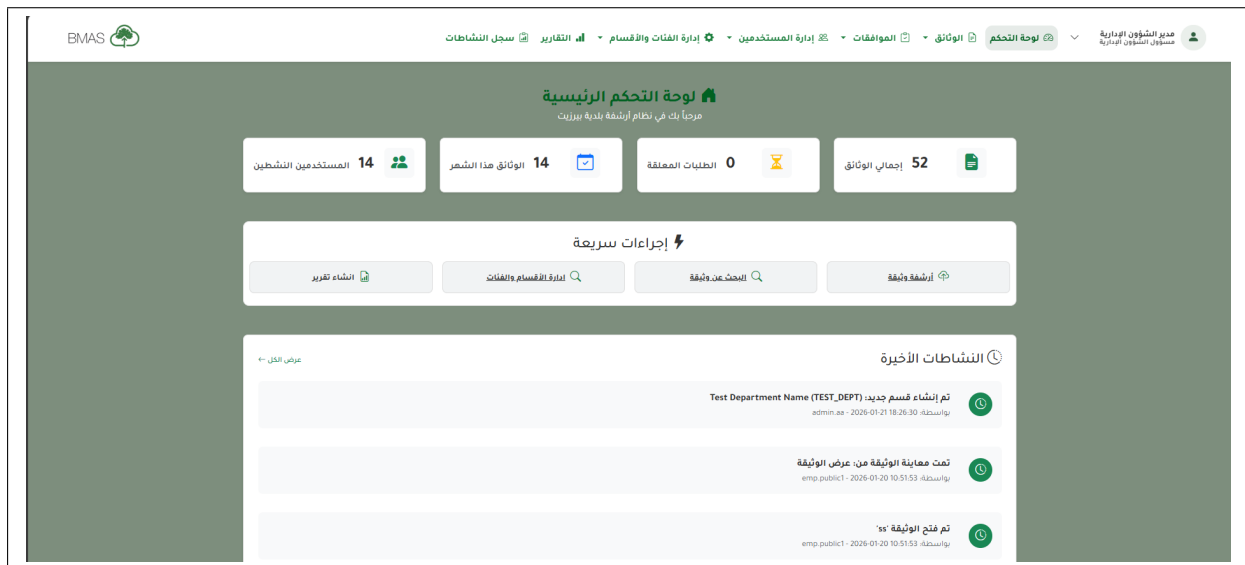


Figure 5.6: Administrative Affairs Dashboard

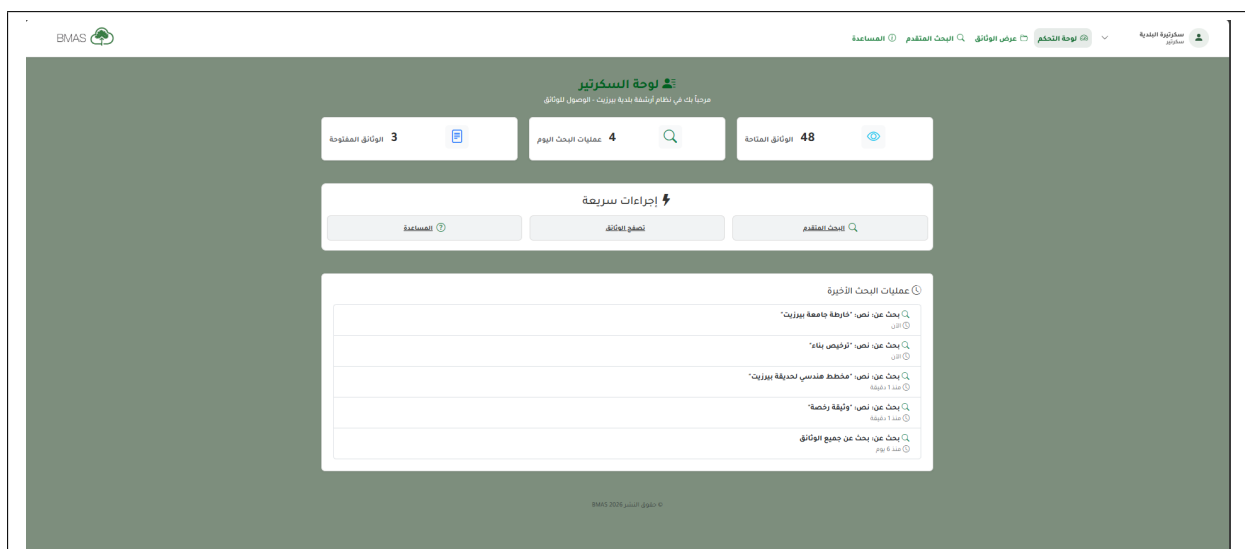


Figure 5.7: Secretary Dashboard

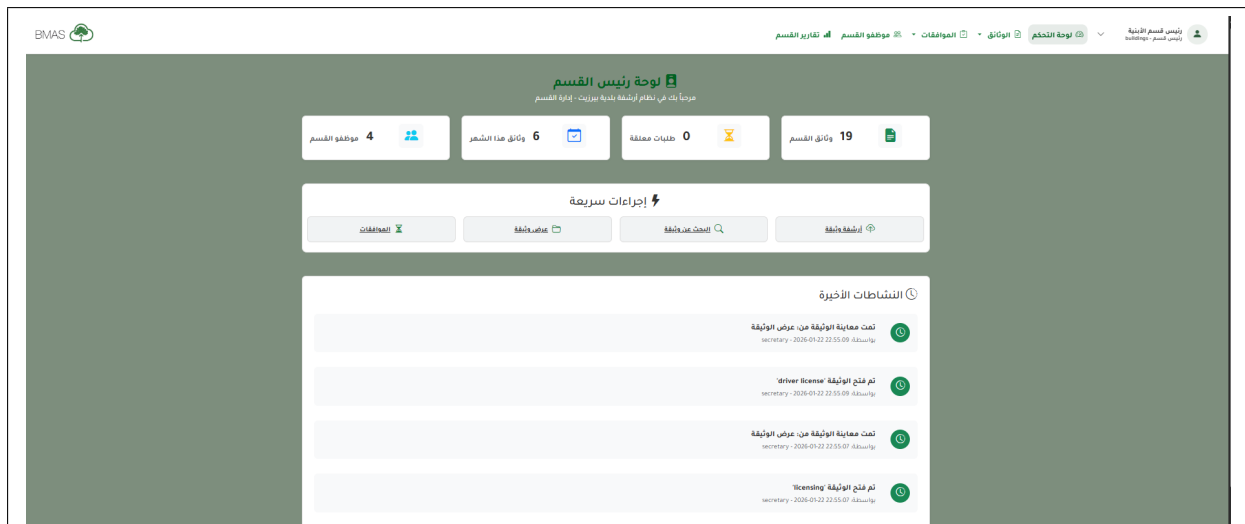


Figure 5.8: Department Head Dashboard

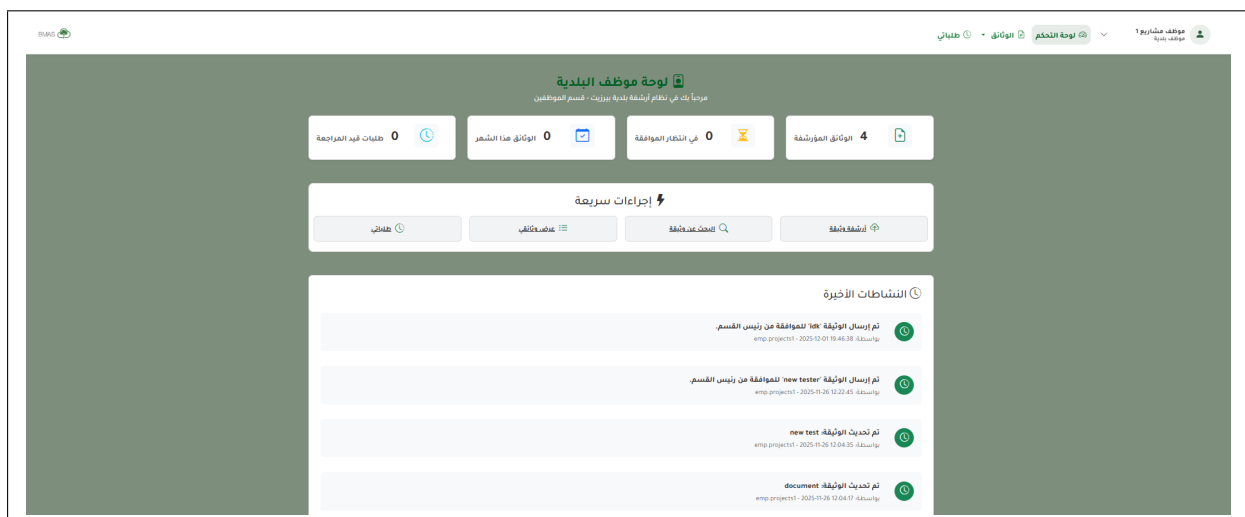


Figure 5.9: Municipal Employee Dashboard

5.3.3 View Documents

Figure 5.10 illustrates the View Documents page which provides a comprehensive interface for browsing and accessing all archived documents in the system. Users can search for documents using the search bar by entering document title or number, and apply **filters** to narrow results by department and category. The page displays documents organized in an expandable accordion format, grouped by department and category for easy navigation. Each document card shows essential information including document number, title, department, category, and upload date.

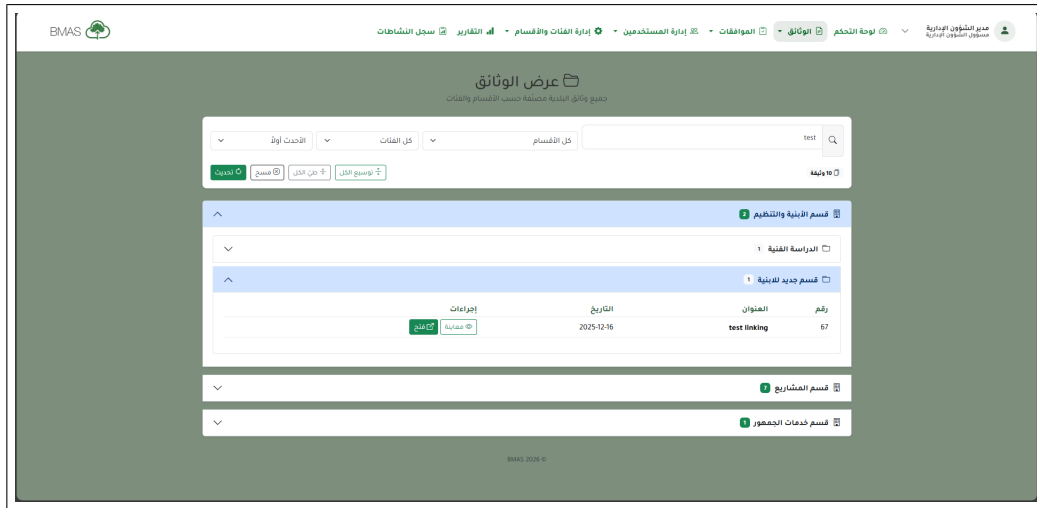


Figure 5.10: View Documents

Users can sort documents by newest first, oldest first, or alphabetically by title. The interface includes action buttons to expand or collapse all sections, clear filters, and refresh the document list. When a document is selected, users can preview its metadata in a modal window or open the full **PDF viewer**. The PDF viewer features a split-screen layout with the document displayed on the left and a metadata panel on the right, showing complete document information and providing a download option. Navigation buttons allow users to move between documents directly within the viewer. **Role-based access control** ensures that administrative affairs and secretaries can view all documents across departments, while department heads and municipal employees see only documents from their own department.

5.3.4 Search For Document

Figure 5.11 demonstrates the Search For Document page which implements an intelligent **semantic search algorithm** that goes beyond simple keyword matching to understand the context and meaning of user queries. Users can enter search terms in natural language, and the system retrieves relevant documents even with partial keywords or related terms by leveraging **SPARQL queries** on RDF triples stored in GraphDB.

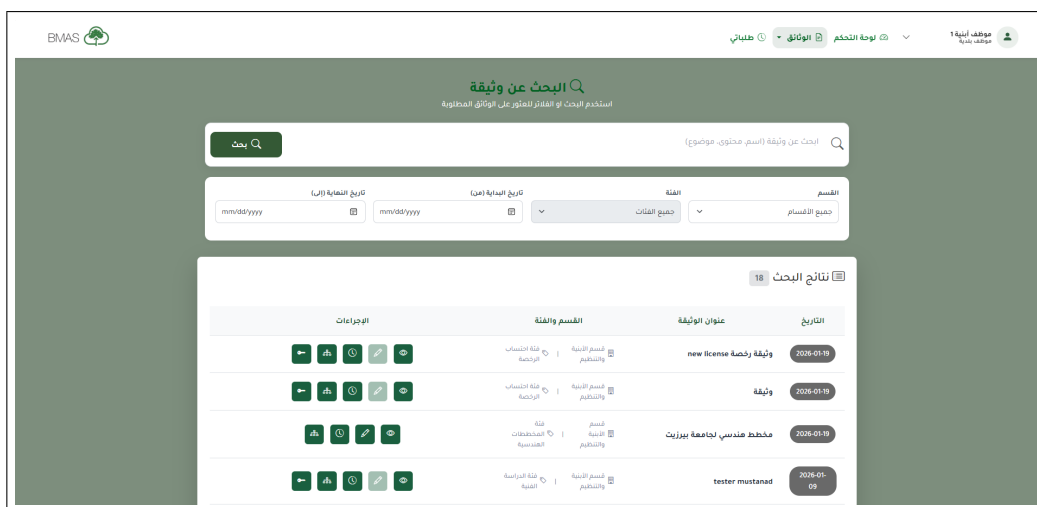


Figure 5.11: Search For Document

The search interface provides filter options by department and category to refine results, and displays matching documents with their complete metadata including title, department, category, and upload date. Each search result card includes action buttons based on the user's role and permissions through role-based access control. Authorized users can edit document metadata directly from the search results, view the complete **version history** and control of documents to track changes over time, open the PDF viewer to read the full document content, and access the workflow status to see approval history and current document state. The search results update in **real-time** as users type or apply filters, providing a fast and efficient document retrieval experience. The semantic search capability ensures that municipal employees can find the documents they need quickly without remembering exact titles or document numbers, significantly improving productivity and reducing time spent searching through archives.

5.3.5 Archive Document

Figures 5.12, 5.13, and 5.14 illustrate the Archive Document page which provides a structured **two-step process** that enables municipal employees to digitize physical documents and store them in the system with complete metadata classification. The interface is designed to simplify the archiving process by breaking it down into manageable steps, with a clear progress indicator at the top of the page that shows users exactly where they are in the process. Visual cues and helpful instructions guide users through each stage, ensuring that all required information is captured accurately before the document is submitted to the system.

Figure 5.12: Step 1: Department and Category Selection

Figure 5.12: Step 1: Department and Category Selection

Figure 5.14: Successful Archiving Confirmation

Figure 5.13: Step 2: Meta-data Entry and File Upload

In the first step, as shown in Figure 5.12, users select the **department** and **document**

category from dropdown menus, establishing the primary classification for the document. The progress indicator shows "Department Information" (معلومات القسم) as step 1 of 2. Once selections are made, clicking "Next" (التالي) advances to the second step.

The second step, depicted in Figure 5.13, presents a comprehensive **metadata entry form** with fields that vary based on the selected document category. Users specify the document type (غير اداري or اداري), enter both of the mutual and exclusive metadata fields to the chosen category. The interface clearly marks required fields with helpful guidance text. At the bottom, users upload the **PDF file** through a drag-and-drop zone or file browser, with validation ensuring only PDF files are accepted. A description field allows additional contextual notes to be added before submission.

Upon successful submission, as illustrated in Figure 5.14, the system displays a confirmation dialog asking whether the archived document is related to another existing document, enabling proper cross-referencing within the system. In these figures, the document was archived directly because the user has administrative affairs privileges. However, for regular municipal employees, the **workflow differs**: if "غير اداري" is selected, the document is sent as a request to the department head for approval; if "اداري" is selected, it is sent to administrative affairs for approval. Only after **approval** the document is officially archived in the system.

5.3.6 Metadata Management

Figure 5.15 shows the Metadata Management page which is exclusively available to users with **administrative affairs privileges**. This page allows administrators to customize and manage the **metadata fields** associated with each document category in the system. By defining category-specific metadata fields, administrators ensure that users are prompted to enter the appropriate information when archiving documents, maintaining consistency and completeness across the archiving system.

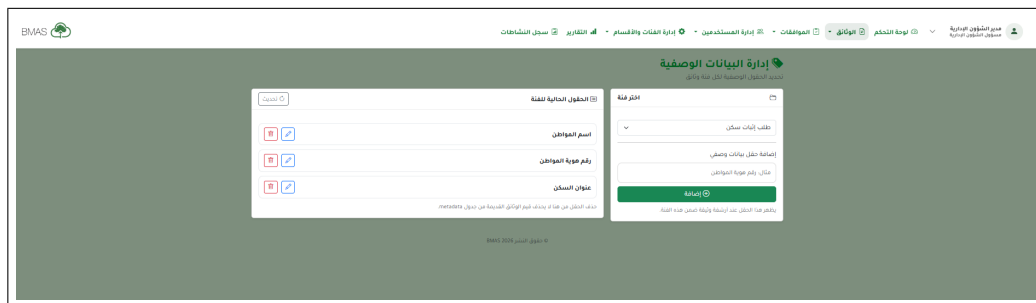


Figure 5.15: Metadata Management Interface

The interface is divided into two main sections. On the right side, administrators select a document category from a dropdown menu to view and modify its associated metadata fields. Below the category selector is a form to **add new metadata fields** by entering a field name (such as "رقم المواطن" or "عنوان السكن") and clicking the "Add" button. The system provides helpful text indicating that newly added fields will appear in the archiving form whenever a user archives a document under the selected category.

The left side displays all current metadata fields defined for the selected category in a clean, organized list. Each field entry includes the field name along with action buttons for **editing** (rename) and **deleting** the field. Administrators can modify field names using the edit button or remove fields that are no longer needed using the delete button. A refresh button at the top of this section allows administrators to reload the field list after making changes. The interface includes a note clarifying that deleting a metadata field from this page does not remove existing metadata values from previously archived documents stored in the database, ensuring **data**

integrity and historical record preservation. This dynamic metadata management capability allows the system to adapt to changing organizational needs and support diverse document types across different municipal departments.

5.3.7 Departments and Categories Management

Figure 5.16 illustrates the Departments and Categories Management page which is an administrative interface exclusively available to users with administrative affairs privileges. This page enables administrators to manage the municipality's **organizational structure** by creating and organizing departments and their associated document categories. Administrators can **add new departments** to the system and define specific document categories within each department, ensuring that the archiving system reflects the actual workflow and document types handled by different municipal units.

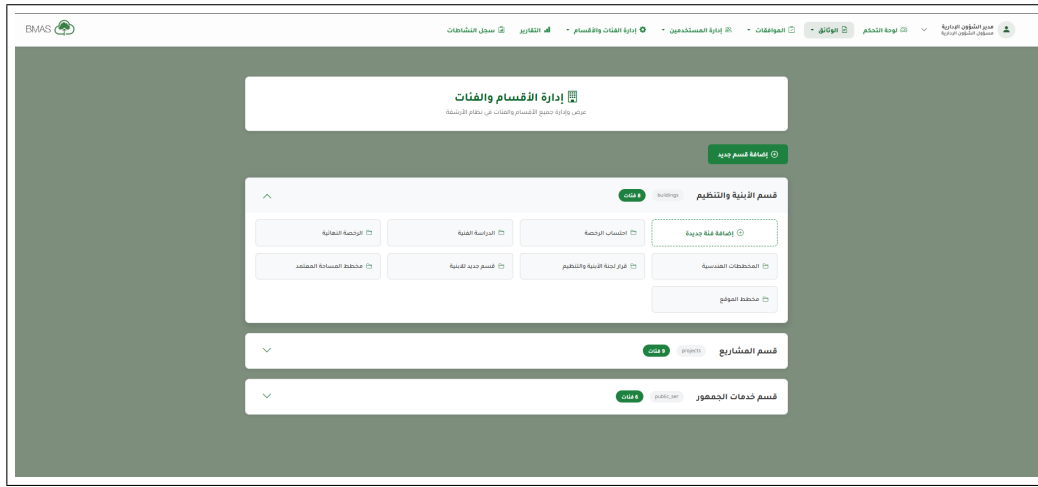


Figure 5.16: Departments and Categories Management Interface

As shown in the figure, the system currently contains **three main departments** with their respective document categories. The "قسم الأبنية والتخطيط" (Buildings and Planning Department) includes categories such as "المخططات الهندسية" (Engineering Plans), "احتساب الرخصة" (License Calculation), and "مخطط الموقع" (Site Plan), among many others. The "قسم المشاريع" (Projects Department) contains categories like "العطاءات المقدمة من الشركات" (Company Submitted Bids), "المخططات المعتمدة للمشروع" (Approved Project Plans), and "دراسة الجدوى للمشروع" (Project Feasibility Study), with additional categories available. The "قسم خدمات الجمهور" (Public Services Department) manages categories including "طلب إثبات سكن" (Residence Proof Request), "طلب براءة ذمة" (Financial Clearance Request), and "طلب كهرباء" (Electricity Request), along with several other service-related categories. This **flexible structure** allows the archiving system to accommodate the diverse document types processed across different municipal departments while maintaining organized classification for efficient retrieval and management.

5.3.8 Requests

Figures 5.17 and 5.18 demonstrate the Requests section which provides different views for tracking document approval workflows. Municipal employees and department heads use these interfaces to monitor submission status and manage approval processes, ensuring transparency and accountability throughout the document archiving workflow.



Figure 5.17: Pending Requests - Department Head View

Pending Requests (Department Head View): Figure 5.17 shows this interface which displays all requests submitted by employees within the department head's department awaiting approval. Each request shows complete details including the employee name ("الموظف المطلب"), category ("الفئة"), department ("القسم"), submission date ("تاريخ الوثيقة"), request date ("تاريخ الطلب"), and document description ("الوصف"). Department heads can review the request details and take action using the "الموافقة" (Approve) button to accept the request or "رفض" (Reject) button to decline it with a reason. The "عرض الوثيقة" (View Document) button allows heads to preview the document before making approval decisions, ensuring informed and appropriate authorization.

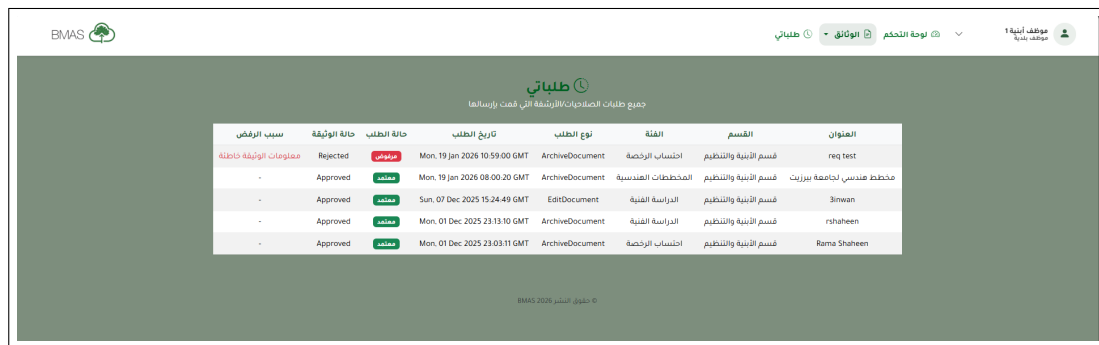


Figure 5.18: My Requests - Employee View

My Requests (Employee/Department Head View): Figure 5.18 demonstrates this page which enables users to **track the status** of all archiving and permission requests they have submitted. The table displays the document title ("العنوان"), department ("القسم"), category ("الفئة"), **request type** ("نوع الطلب"), submission date ("تاريخ الطلب"), **request status** ("حالة الطلب"), and **rejection reason** ("سبب الرفض") if applicable. **Color-coded badges** indicate approval status: green "موافقة" (Approved) and red "مرفوض" (Rejected), allowing users to quickly identify outcomes and take appropriate follow-up actions.

5.3.9 Report Generation

Figure 5.19 illustrates the Report Generation page which provides comprehensive **statistical analytics** and reporting capabilities for monitoring system activity and document management across the municipality. This interface enables administrators and department heads to generate **customized reports** based on specific time periods, offering valuable insights into archiving workflows, document distribution, and departmental performance. Access to reports is **role-based**: administrative affairs can view statistics across all departments, while department heads can only access reports for their own department.

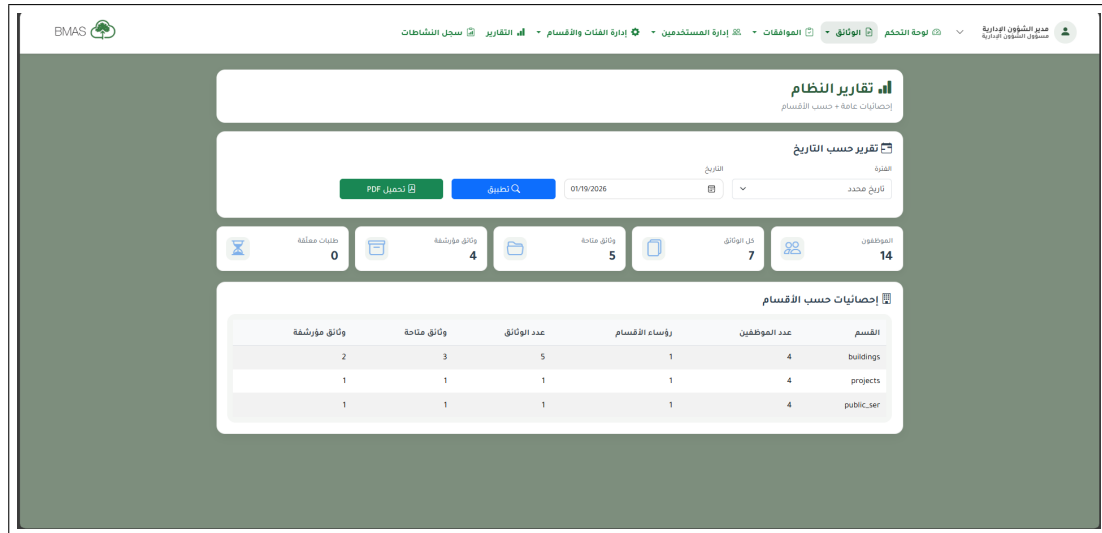


Figure 5.19: Report Generation Interface

As shown in the figure, the interface features a **date-based filtering system** where users can select either a specific date or a **date range** (from-to) using the date picker and generate reports by clicking the "تطبيق" (Apply) button. The system displays key performance indicators through visual cards showing "الموظفون" (Employees) count, "كل الوثائق" (All Documents), "وثائق متاحة" (Available Documents), "وثائق مؤرشفة" (Archived Documents), and "طلبات معلقة" (Pending Requests). Below these summary cards, a detailed table titled "إحصائيات حسب الأقسام" (Statistics by Department) breaks down the data by department, displaying columns for department name ("القسم"), number of employees ("عدد الموظفين"), department heads ("رؤساء الأقسام"), total documents ("عدد الوثائق"), available documents ("وثائق متاحة"), and archived documents ("وثائق مؤرشفة"). Users can export the generated report as a PDF file using the "تحميل PDF" (Download PDF) button for offline analysis, documentation, or presentation purposes, facilitating data-driven decision-making and performance monitoring across the municipality.

Chapter 6

Conclusion

This final chapter reflects on the work completed so far and outlines our plans for the future. We review the project goals, the features we designed, and how our system addresses the municipality's document management challenges. We then present our plan for next semester, where we will move from design to implementation, build the actual web application, test it with real users, and prepare it for deployment at Birzeit Municipality.

6.1 Review of the project

The overall review of the Birzeit municipality archiving system:

- The goal of this project is to develop a digital archiving system that works alongside the current municipalities paper-based system, the objective is to enhance the traditional way of document storage with more effective and searchable digital solution. Employees will be able to manage files more safely across departments, search by metadata, and access documents more quickly with this system.
- We created and put into use important features like workflow approvals, version control, audit logging, and semantic search during the development phase. In order to show how the system works, UML diagrams including class diagrams, sequence diagrams, and state charts were used to support these. Additionally, we ensured that the system accurately reflects the work flow and meets the municipality's actual needs.
- With everything taken into account, the project provides a solid basis for enhancing document archiving within the municipality. Although the paper archive will continue to be used, this digital system provides a more structured and easily accessible method of managing official records, ultimately saving staff time and minimizing errors.

Bibliography

- [1] N. Duhan, A. Sharma, and K. Bhatia, “Semantic search using word2vec and annoy index for large-scale information retrieval,” *International Journal of Advanced Computer Science and Applications*, vol. 11, no. 5, pp. 273–280, 2020.
- [2] M. Terras, “The digital classicist: Building a bridge to digital humanities in classical studies,” *Digital Humanities Quarterly*, vol. 5, no. 1, 2011.
- [3] S. Jaeckel et al., “A web-based open source tool for sharing and visualizing wireless channel measurements,” in *IEEE 86th Vehicular Technology Conference (VTC-Fall)*, IEEE, 2017.
- [4] N. Radzuan et al., “Digital transformation in municipalities,” in *Proceedings of the International Conference on Smart Cities*, 2018.
- [5] P. Nero, *Role-based access in archiving systems*, 2005.

Appendix A

Appendices

A.1 Use Case Specifications

Use case Id and name:	UC-01: Login/Authentication
Actors:	Municipal employees
Trigger:	Municipal employees wants to log in to the system.
Description:	This use case describes how a user logs in to the system. This login mechanism ensures the user's home page is displayed according to their permissions.
Preconditions:	The user has an existing account
Postconditions:	1. On success: The user is granted access to their personalized dashboard or homepage. 2. On failure: The user is notified of the failed login attempt and remains on the login page.
Normal flow:	1.0 Login: 1. The user navigates to the login page. 2. The system opens the login page 3. The user enters a valid username/email and password. 4. The system validates the credentials. 5. The system authenticates the user. 6. The user is redirected to their dashboard or homepage.
Alternative flow:	1.1.A1: Invalid credentials: 1. The user enters an incorrect username or password. 2. The system displays an error message and prompts the user to try again
Exceptions:	1.2.E1: System unavailable: 1. The login service is temporarily unavailable due to server downtime or maintenance. 2. The system displays an appropriate error message and advises the user to try again later.
Assumptions:	1. Users understand how to enter their credentials. 2. The system has access to a reliable database of user credentials.
Other information:	The system may support multiple languages on the login interface

Table A.1: UC-01: Login/Authentication

Use case ID and name:	UC-02: Archive document
Actors:	Administrative affairs, Department head, Municipal employee
Trigger:	New document is received (paper).
Description:	Enable the user to archive a document to make the retrieval of a specific document easier.
Preconditions:	1. The user is logged in the system. 2. The user has permission to archive.
Postconditions:	Document is archived successfully.
Normal flow:	2.0 Archive a document by a municipal employee: 1. The municipal employee selects the type of document to archive 2. The system displays the document upload and metadata form. 3. The user uploads the document and fills in the metadata. 4. The system validates the input and checks role-based permissions. 5. If the user is a regular employee, the system sends the document for review to the department head. 6. If the user is the department head or administrative affairs, the document is archived directly. 7. The system confirms successful archiving and logs the action.
Alternative flow:	2.1.A1: Validation Error: 1. The system detects missing or invalid metadata. 2. An error message is shown, and the user is prompted to correct the input. 2.1.A2: Approval required: 1. If a regular employee archives a document, the system marks it as "Pending Approval." 2. The department head is notified to review and approve the submission. 3. Once approved, the document is archived and confirmed to the user.
Exceptions:	2.2.E1: System error during upload The system displays an error message indicating a failure. The user is prompted to retry or contact support. 2.2.E2: Insufficient permissions The user is notified that they do not have permission to archive documents. The user exits or contacts an authorized actor.
Assumptions:	• Employees are trained on proper metadata entry and document preparation. • Document classification guidelines are in place. • System maintains version control for document updates.
Other information:	none

Table A.2: UC-02: Archive

Use case ID and name:	UC-03: Edit a document
Actors:	Municipal employee, Administrative affairs, Department head
Trigger:	• Document metadata needs updating. • Errors are found in archived document.
Description:	Enable the employee who archived the document to edit the metadata only if the document is marked as editable. Editable documents support versioning. The system ensures changes are tracked, approved, and saved as new versions.
Preconditions:	1. The municipal employee is logged into the system. 2. The document is located by search. 3. The municipal employee has edit permission and the document is editable.
Postconditions:	• For editable documents: 1. Metadata is successfully edited. 2. A new version is created with a new unique ID/metadata. 3. Activity is logged in the audit log. • For non-editable documents: 1. After department head approval, document becomes permanently locked from edits. 2. Activity is logged in the audit log.
Normal flow:	3.0 Edit a document: 1. The municipal employee provides the document ID to edit. 2. The system verifies edit permission and retrieves the document and its metadata. 3. The system checks if the document is editable, if editable: 4. The system locks the current version of the document and creates a working copy. 5. The municipal employee edits the metadata and stores a new document or image if needed. 6. The municipal employee confirms changes. 7. The system sends edit changes to the department head and displays a confirmation message to the municipal employee. 8. The department head reviews the changed metadata and approves the changes. 9. The system saves the changes as a new version of the document and confirms the edit. 10. The system logs the edit activity.

Alternative flow:	A1: Department head rejection: 1. The department head rejects the changes. 2. The system notifies the municipal employee about the rejection. 3. The working copy is discarded and the original version remains unchanged. A2: Document is not editable: 1. The system detects that the document is non-editable. 2. The system displays a message that no further edits can be made.
Exceptions:	3.1.E1: Document not found: • The system notifies the user that the document could not be located. • The user exits the use case or searches again. 3.2.E2: Unauthorized edit: • The system detects invalid or missing input. • An error message is shown, and the user is prompted to correct the input.
Assumptions:	• Municipal employees are trained on update and search procedures. • Document editability status is set correctly upon archiving
Other information:	• The system must handle metadata updates promptly to avoid data conflicts. • Once a non-editable document is approved by the department head, it becomes read-only

Table A.3: UC-03: Edit document

Use case ID and name:	UC-04: Search for a document
Actors:	Municipal employee, Administrative affairs, Department head, Mayor
Trigger:	• User needs to find a specific document. • User wants to browse available documents.
Description:	Enable authorized users to search for documents in the archiving system using metadata. The system retrieves and displays matching documents with options based on view or edit permissions.
Preconditions:	• The user is logged in to the system.
Postconditions:	• Search results are displayed on the screen. • The system logs the search activity.
Normal flow:	4.0 Search for a document: 1. The municipal employee enters metadata search criteria. 2. The system validates the input. 3. The system searches indexed metadata. 4. The system displays matching documents. 5. A municipal employee selects a document to view. 6. The system checks if the municipal employee has permission to view the selected document. 7. If permitted, the document is displayed. 8. If not permitted, the system prompts the municipal employee to request access from the department head. 9. The system logs the search activity.
Alternative flow:	4.1.A1: Department head approval: 1. The municipal employee does not have view permission. 2. An error message is shown, and the municipal employee cannot view or edit the document.
Exceptions:	4.1.E1: No result found: • A "no results found" message is displayed. • The municipal employee validates the inputs or exits the search. 4.1.E2: Validation error: • The system detects invalid or missing input. • An error message is shown, and the municipal employee is prompted to correct the input.
Assumptions:	• Staff are trained on search procedures. • Document metadata is accurately indexed.
Other information:	• Search results must load within 15 seconds. • The system must handle concurrent searches from multiple employees.

Table A.4: UC-04: Search for a Document

Use case ID and name:	UC-05: View document
Actor:	Municipal staff
Trigger:	• User needs access to a document from another department. • User needs access to a document from their own department.
Description:	Allow a staff member to request access to view a document that belongs to a different department. The request must be approved by the administrative affairs officer before access is granted.
Preconditions:	• The user is logged into the system. • The user is authenticated and has permission to request access.
Postconditions:	• The view request is submitted and saved. • The administrative affairs officer is notified of the request. • If approved, the user is granted view access to the requested document. • The action is logged for audit purposes.
Normal flow:	5.0 Same Department Document Access: 1. The municipal employee selects a document that belongs to their own department. 2. The system verifies the department match. 3. The system grants immediate view access. 4. The system logs the access without requiring approval. 5. The system notifies the user that access has been granted.
Alternative flow:	5.1.A2 Request to view document (different department): 1. The user navigates to the “Request to View” section. 2. The user selects a document outside their department. 3. The system submits a view request to the administrative affairs officer. 4. The officer reviews the request. 5. If approved, the system grants view access to the user. 6. The user is notified, and the action is logged. 5.1.A3 Invalid Request Input: 1. The system detects missing or invalid input. 2. The system displays an error message. 3. The user corrects the input or cancels the request.
Exceptions:	5.2.E1 Document not found: • The system notifies the user that the document could not be located. • The user exits the use case or searches again. 5.2.E2 Request Rejected: • The administrative affairs officer rejects the request. • The system notifies the user of the rejection. • No access is granted.

Special requirements:	• Only the administrative affairs officer can approve or reject view requests. • Temporary access must be controlled and revoked automatically if needed. • All access and approvals must be recorded in the system logs.
Assumptions:	• Staff understand the procedures for requesting access to documents from other departments. • The administrative affairs officer has full access to manage document requests.
Notes and issues:	• All document view activities must be tracked, including timestamps, user identity, and approval status. • Notifications should be sent to both the requester and the officer for transparency and follow-up.

Table A.5: UC-05: View Document

Use case ID and name:	UC-06: Manage user roles and permissions
Actors:	Administrative affairs, Department head, Municipal employee
Trigger:	• New employee joins the municipality. • Department head requests a role change for themselves or other municipal employees.
Description:	This use case enables the administrative affairs to create new user roles or change user roles for municipal employees.
Preconditions:	• Administrative affairs is logged into the system. • The employee is either new or existing.
Postconditions:	• A new user role account is created for the new employee. • User role is changed. • Changes are recorded in the audit log.
Normal flow:	6.0 New employee in the municipality: 1. Administrative affairs opens the role management dashboard. 2. The system displays options to create or change user roles and permissions. 3. Administrative affairs clicks on “Create Account” and assigns the employee role. 4. A new username and password are generated for the employee. 5. Administrative affairs assigns permissions for the new employee. 6. The system validates the data. 7. The system logs the changes in the audit log. 8. The new employee logs into the system using the assigned credentials.
Alternative flow:	6.1.A1 Existing employee role change: 1. A municipal employee wants to change role (e.g., to department head or supervisor). 2. The department head submits a role change request to administrative affairs. 3. The system notifies administrative affairs of the new request. 4. Administrative affairs approves or rejects the role change. 5. If approved, the role is updated and the municipal employee is notified. 6. The municipal employee must re-log into the system to confirm the role change. 7. If rejected, the municipal employee is notified. 8. The system logs all changes in the audit log.
Exceptions:	6.2.E1 Unauthorized role change request: 1. A municipal employee requests a role they are not authorized to hold. 2. The system denies the request and notifies administrative affairs.
Other information:	Changes are integrated with the HR workflows.

Assumptions:	• Department heads will request role changes only for their team members. • Roles align with the municipality's departmental hierarchy.
---------------------	---

Table A.6: UC-06: Manage User Roles and Permissions

Use case ID and name:	UC-07: Document version control
Actors:	Administrative affairs, Department head, Municipal employee
Trigger:	User edits an existing editable document.
Description:	The system tracks and manages multiple versions of the same document. Only the latest version of the document is retained as active, while older versions are archived for reference.
Preconditions:	- The user is logged into the system. - The document exists in the system.
Postconditions:	- The old version is archived and marked as inactive. - A new document version is saved with a new ID. - The activity is recorded in the audit log.
Normal flow:	7.0 Version control: - The municipal employee opens a document to make changes. - The system validates if the document is editable. - If editable, the system locks the current version and creates a working copy. - The municipal employee edits the document and confirms edits. - The system marks the old version as inactive. - The system assigns a new unique ID to the new version. - The system saves the new version as the active document. - The system notifies the municipal employee of successful version update. - The system logs the action in the audit log.
Alternative flow:	7.1.A1: Document edit access is temporary: - The document can be edited only before it is approved by the department head. - Once approved, the document becomes archived and non-editable.
Exceptions:	7.2.E1 Save document failure: - If the network disconnects during saving, the document is not saved. - The system displays an error message to the employee who made the changes.
Assumptions:	Archived versions are retained for compliance but are not searchable in active workflows.
Other information:	Old versions are stored in a read-only archive with associated metadata.

Table A.7: UC-07: Document Version Control

Use case ID and name:	UC-08: Metadata management
Actor:	Administrative affairs
Trigger:	• New document type requires custom fields. • Existing metadata fields need modification.
Description:	This use case describes how the user can create, update, and manage metadata schemas within the archiving system. These schemas define standard metadata fields used to tag and classify documents, enabling consistent organization, improved searchability, and regulatory compliance.
Preconditions:	• The user must be logged into the system. • The user must have administrative privileges.
Postconditions:	• Metadata schemas are successfully created or updated. • The schemas are saved in the system for use during document archiving.
Normal flow:	8.0 Metadata schema: 1. Administrative affairs navigate to the “Metadata Schema” management section. 2. The system displays a list of existing metadata schemas. 3. The administrative affairs selects the option to create a new schema. 4. The administrative affairs define schema fields (e.g., field name, type, required/optional). 5. The administrative affairs submits the new schema. 6. The system validates and saves the schema. 7. The system confirms successful creation.
Alternative flow:	8.1.A1 Update Existing Schema: 1. The user selects an existing schema from the list. 2. The user modifies the schema fields. 3. The user submits the updated schema. 4. The system validates and updates the schema. 5. The system confirms successful update.
Exceptions:	8.2.E1: • If a field name is duplicated or invalid, the system displays a validation error. • If the schema is in use and incompatible changes are made, the system shows a warning or blocks the update.
Assumptions:	• Only users with administrative privileges can manage metadata schemas.
Other information:	• Metadata schemas help enforce data consistency and support regulatory compliance. • Clear error messages guide users during schema creation or editing. • Common metadata fields include title, date, author, department, and notes. Some categories may include citizen name, company name, plot number, and basin number.

Table A.8: UC-08: Metadata Management

Use case ID and name:	UC-09: Manage document categories
Actor:	Administrative affairs
Trigger:	• New document classification is needed. • Existing category requires reorganization.
Description:	This use case describes how the user can create and update categories to classify documents within the archiving system. These categories help in organizing documents for better retrieval and metadata consistency.
Preconditions:	• The user must be authenticated and logged into the system.
Postconditions:	• Document categories are successfully updated or created. • Each category is assigned a unique ID.
Normal flow:	9.0 Manage categories: 1. Administrative affairs navigate to the “Document Categories” management section. 2. The system displays a list of existing document categories. 3. Administrative affairs selects the option to create a new category. 4. Administrative affairs enters the category name. 5. Administrative affairs submits the new category. 6. The system validates and saves the new category. 7. The system confirms successful creation.
Alternative flow:	9.1.A1 Update Existing Category: 1. The user selects an existing category from the list. 2. The user modifies the category name. 3. The user submits the update. 4. The system validates and updates the category details. 5. The system confirms successful update.
Exceptions:	9.2.E1: • If the entered category name already exists, the system displays a validation error and prompts the user to enter a unique name. • If there are archived documents under the category, the system will display an error message.
Assumptions:	• Only users with administrative privileges have access to this functionality.
Other information:	• Clear error messages should guide the user in case of invalid inputs or system failures.

Table A.9: UC-09: Manage Document Categories

Use case ID and name:	UC-10: Manage municipal departments
Actor:	Administrative affairs
Trigger:	• New department is created in the municipality. • Department restructuring occurs. • Department naming conventions change.
Description:	This use case describes how the user can create and update municipal departments to classify documents within the archiving system. These departments help in organizing documents for better retrieval and metadata consistency.
Preconditions:	• The user must be authenticated and logged into the system.
Postconditions:	• Departments are successfully created or updated. • Each department is assigned a unique ID.
Normal flow:	10.0 Department management 1. Administrative affairs navigates to the “Departments” management section. 2. The system displays a list of existing departments. 3. Administrative affairs selects the option to create a new department. 4. Administrative affairs enters the department name. 5. Administrative affairs submits the new department and its unique code. 6. The system validates and saves the new department. 7. The system confirms successful creation.
Alternative flow:	10.1 Update existing department: 1. The user selects an existing department from the list. 2. The user modifies the department name. 3. The user submits the update. 4. The system validates and updates the department details. 5. The system confirms successful update.
Exceptions:	10.2.E1 • If the entered department name already exists, the system displays a validation error and prompts the user to enter a unique name. • If there are archived documents under the department, the system will display an error message.
Assumptions:	• Only users with administrative privileges have access to this functionality.
Other information:	• Clear error messages should guide the user in case of invalid inputs or system failures.

Table A.10: UC-10: Manage Municipal Departments

Use case ID and name:	UC-11: Log activity
Actors:	The system
Trigger:	• Automatic system activity logging. • A new action record is performed.
Description:	To store all the system actions performed by the municipality employees.
Preconditions:	• Municipal employees perform an action.
Postconditions:	• All system activities are stored in the log. • Logs are accessible through the audit log feature.
Normal flow:	11.0 Log activities 1. A municipal employee or department head performs any action in the system. 2. The system logs each activity with: date and time, user ID, document ID, role, and action type. 3. The system validates the action and saves the entry into the audit log.
Alternative flow:	None
Exceptions:	11.1.E1 Audit log database not found: • The system fails to retrieve logs due to a database connection failure. • The system generates an alert to the administrative affairs and the mayor. • The system retries accessing the audit logs.
Assumptions:	• Only administrative affairs and the mayor can access audit logs. • System activities are consistently and correctly logged.
Other information:	• Audit logs must include timestamp, user ID, document ID, and action type (view, edit, search, archive). • Logs are read-only and immutable.

Table A.11: UC-11: Log Activity

Use case ID and name:	UC-12: Generate document report
Actors:	Administrative affairs, Department head
Trigger:	• Scheduled compliance reporting. • Departmental audit.
Description:	This use case describes how an authorized user generates reports detailing document-related activities within the Birzeit Municipality Archiving System. Specifically, it focuses on generating reports that show documents uploaded during a user-defined time period.
Preconditions:	• The user must be logged into the system. • The user must have the necessary permissions to access the reporting module and view document activity logs. • The system must have recorded document upload activities.
Postconditions:	• Success: A report summarizing document uploads within the specified time frame is generated and displayed. The user may have options to export the report. • Failure: An error message is displayed if the report cannot be generated.
Normal flow:	12.0 Generate report: 1. The administrative affairs or department head navigates to the reporting section of the archiving system. 2. The user selects the "Document Report" option. 3. The system prompts the user to specify a time period (start date and end date). 4. The user enters the desired start and end dates. 5. The user initiates the report generation process. 6. The system queries the activity logs for all document upload events in the specified range. 7. The system compiles the data, including details like document name, upload date/time, uploading user, and department. 8. The system generates the report in a predefined format. 9. The system displays the generated report to the user.
Alternative flow:	12.1.A1 Municipal employee or secretary requests a report: 1. The municipal employee or secretary must request access from administrative affairs. 2. The system notifies administrative affairs of the request. 12.1.A2 No documents uploaded: 1. If no documents were uploaded in the specified period, the system displays a message indicating that no relevant activity was found.

Exceptions:	12.2.E1 Invalid Date Range: • The system detects an invalid date range. • The system displays an error message and prompts the user to correct the dates. 12.2.E2 System Error: • A system error occurs during data retrieval or report generation. • The system displays an error message indicating failure.
Assumptions:	• The report format is clear and easy to read. • The system can efficiently handle large volumes of data for long date ranges.
Other information:	• Date and time information must be accurate and consistently formatted.

Table A.12: UC-12: Generate Document Report

A.2 Glossary (or data dictionary)

BMAS: Birzeit Municipality Archiving System.

RBAC: Role-Based Access Control.

Semantic Search: An algorithm that retrieves documents based on meaning and context, not just exact keywords.

Indexing: The process of analyzing and organizing content of the document for efficient semantic search.

Metadata: The data that's attached to a document (title, date, author, department, and description) to help with retrieval.

Document Archiving: The process of storing municipal documents in PDF format to ensure long-term preservation.

Version Control: A way to track document versions.

Audit Log: A record that logs all user actions (archive, edit, view) to ensure traceability.

Administrative Affairs: The role with the highest permissions, such as user management, category control, and accessing audit logs.

Department Head: A role with authority to approve requests, edit documents within their department, and generate reports.

Municipal Employee: A user in the BMAS who archives, views, and searches for documents with limited permission.

Departments: Units in the municipality that own and manage different types of documents.

Document Categories: Categories that organize documents into groups based on their type (e.g., Permits, Complaints, Licenses).

Archiving Status: The state of a document (Pending, Approved, Archived, Locked).

Workflow: The sequence of steps that a document goes through, like review and approval.

A.3 Code

In our project, the two main functions that we use are archive and semantic search.

After the user uploads the document and metadata, the system rechecks the user's role. If the user is an employee, then they need to get the approval from the department head, but if the user is a department head or an administrative affairs, then they don't need approval.

Below is a pseudo-code that explains how this algorithm works

```

FUNCTION archiveDocument(userID, documentFile, metadata)
  IF NOT hasPermission(userID, "archive") THEN
    RETURN "Error: User does not have archive permissions"
  END IF
  IF documentFile.type != "pdf" THEN
    RETURN "Error: Only PDF files are accepted"
  END IF
  requiredFields = ["title", "author", "date", "department", "category"]
  FOR EACH field IN requiredFields DO
    IF metadata[field] IS EMPTY THEN
      RETURN "Error: Missing required metadata: " + field
    END IF
  END FOR
  IF getUserRole(userID) == "Municipal Employee" THEN
    status = "Pending Approval"
    approverID = getDepartmentHead(userID)
    sendApprovalRequest(approverID, documentFile, metadata)
  ELSE
    status = "Archived"
  END IF
  docID = generateUUID()
  storeDocument(docID, documentFile)
  storeMetadata(docID, metadata, status, userID)
  logActivity(userID, "Archive", docID, currentDateTIme())
  IF status == "Pending Approval" THEN
    RETURN "Document submitted for approval"
  ELSE
    RETURN "Document archived successfully"
  END IF
END FUNCTION

```

Figure A.1: Archive Pseudo Code

After the document is archived, the user may need to search for a document. The system takes the query from user input and normalizes it, then SPARQL query to check the metadata and displays the result to the user.

Below is a simple pseudo-code that explains how this algorithm works

```
Function: SemanticMetadataSearch (input: query)
  Normalize and clean the query text
  Search for documents where the cleaned query matches any of the following (case-insensitive):
    - Title
    - Department
    - Date
    - Uploader
  Execute the search using a semantic query
  If matching results are found:
    For each result:
      Display the document
      Display the title
      Display the department
      Display the date
      Display the uploader
  If no results are found:
    Display error: "No documents matched your search query"
```

Figure A.2: Semantic Search Pseudo Code

A.4 System modifications

We have made some modifications to the previous introduction to the graduate report, which are:

- Section 1.3 Method.
- Section 1.4 Aim & Objectives.
- Section 1.5 Overview of the technical area.
- Section 1.6 Overview of the report.
- Section 2.3 Tools and Technology.
- Added a summary section after each chapter.
- Added an introductory preface at the beginning of each chapter.
- Added a short description for each figure.
- Functional and Non-functional requirements Section.
- Deleted all functionality of workflow management (dynamic workflow).
- Edit (use case, class, sub-system, software architecture, deployment, and ERD) diagrams.